

THE WORK A VERIFIER NEEDS: CERTIFIED LOW-PRECISION OUTPUT HEADS UNDER A STOCK-KERNEL CONTRACT

Samuel Mausberg

ABSTRACT

A greedy or seeded speculative verifier needs one fact from the target’s output head, the winning token, yet computes every logit. Certifying that winner from a cheap copy of the head is known; we state the contract it must meet inside a production engine and measure its limits. For Qwen3.5-4B served by SGLang on a GH200, that is agreement with the token the engine’s own head kernel returns at the same batch shape, BF16 rounding and first-index tie rule included, and we prove when enclosures of that kernel’s FP32 accumulator, under a named error model, fix it; every other row runs the stock kernel. On 6,005 captured decode states, an int8 copy of the head with a proved envelope re-scores 1.3–1.6 rows per decision on average and leaves 0.35% of rows to the stock kernel under a published model of Hopper tensor-core accumulation, or 1.40% under a conservative model, and no certified token differed from the engine’s; the head-path runtime is pending. Transport, which would reuse a drafter’s per-tile head summaries at the target’s state, fails on both public drafters: their head inputs differ from the target’s by a median 92% and 95%, where transport needs agreement to about 1%, so for every bound family and tiling tested its certificates skip at most 0.65% of the vocabulary.

1 INTRODUCTION

At every position a speculative decoder verifies, the target model projects its hidden state through the output head and scores every token of the vocabulary, although a greedy verifier uses only the index of the maximum, and a sampled verifier with noise fixed in advance only the maximum of a perturbed score. For Qwen3.5-4B the tied head has 248,320 rows of width 2,560, 1.27 GB in BF16: 15% of the weight bytes of a plain decode step and 84% of those of a multi-token-prediction (MTP) draft step (derived from the checkpoint, Table 16). Measured on a GH200, the head’s GEMM, FP32 copy and argmax take 10.3% of a plain decode step at batch 1, 6.1% at batch 128, and 17.2% of an MTP speculative cycle at batch 1 [E31] (Figure 1), so even a free head would speed up a plain step at batch 1 by only about 11%. The question here is narrower than speed: how much of the projection a verifier needs, and against what it must be exact.

The known way to do less computes cheap scores for every

Code, exact references, committed evidence and the source of this paper:

<https://github.com/SamMausberg/verified-progress>.

AI use: the earlier draft was prepared with GPT-6 Astra Pro (OpenAI) and this version with Claude Code (Claude Opus 5.5 agents); the author directed the work and reviewed it as it progressed, including in discussions of specific points with Claude and with OpenAI models, and is responsible for its claims; Appendix G.6 gives details.

Preprint, 30 September 2026; not peer reviewed.

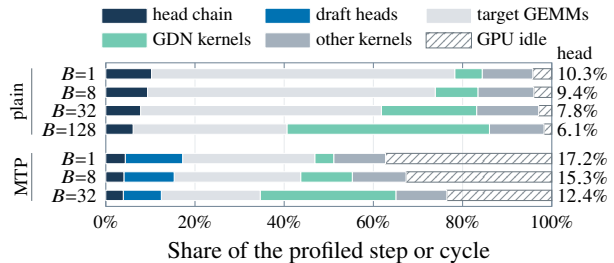


Figure 1. GPU time of a decode step by component, measured under Nsight Systems at a fixed batch B [E30]; an MTP cycle (three draft steps) commits several tokens. Right: the heads’ share of the profiled step or cycle. Tracing inflates the MTP cycle’s idle time; untraced it is about 21%, 20% and 15% (derived, Appendix E.2).

row, bounds their error and re-scores only the rows that could still win; public implementations do this for greedy heads (sparkpipe contributors, 2026; Hawkins, 2026; Layr Labs, 2026; Chamberlain, 2026), and exact sampling from bounded scores is older (Maddison et al., 2014; Musmann et al., 2017). They differ in their reference; inside a production engine it is the engine’s own computation. SGLang’s head is a cuBLAS BF16 GEMM that accumulates in FP32, chooses its kernel by batch shape and rounds to BF16, followed by an argmax that returns the first maximal index. On this model the engine’s two largest logits tie in BF16 at 1.0% of greedy positions, and the real-arithmetic winner differs from the engine’s token at about 0.5% [E1]. Certifying the

real-arithmetic winner certifies a computation the engine does not perform, and a bound that treats FP32 accumulation as exact, or as IEEE additions, does not enclose what tensor cores produce.

We state and test the contract a certified head can meet in such an engine: the token the stock head kernel returns for the same hidden state at the same batch shape. A row is certified only when enclosures of that kernel’s FP32 accumulator, under a named error model, round into BF16 values that fix its argmax and tie rule; every other row runs the stock kernel, so, as long as the enclosures hold (Table 2), a failed certificate costs time and never agreement (Figure 2). The enclosures come from an int8 copy of the head, which reads half its bytes.

We also test *transport*, the mechanism this work set out to build: a drafter that projects through the same head has computed a maximum and a mass for every vocabulary tile, and a Hölder bound can carry them to the target’s state, so the verifier would reuse work already paid for. From the weights alone, transport beats an int8 envelope only if draft and target head inputs agree to about one part in a hundred. On real states they differ by a median 92% for the public DFlash drafter, whose final norm fixes their length at a third of the target’s, and 95% for the model’s own MTP layer, and the certificates skip almost nothing.

1.1 Contributions

1. *A contract against the stock kernel* (Section 2): a gap condition and a sharper interval form, both proved, fix when a certified token equals the stock token at the same batch shape, with the stock kernel as the fallback. The kernel’s error model is an assumption, tested rather than proved; Table 2 lists it with every other numerical assumption.
2. *A certified head that meets it* (Section 4): an envelope sound for tensor-core accumulation under Khattak and Mikaitis’s model of Hopper (Khattak & Mikaitis, 2026), or model-free for an integer pass (an implementation is sound only if its kernel computes the modelled arithmetic), and one refinement for greedy selection, seeded sampling and fixed-noise verification, the last two against the engine’s deterministic-inference mode (proved; measurement [pending D8]). The greedy form is prior art; our claim is narrower (Section 6).
3. *Measurements on the engine’s own states* (Section 4.4): on 6,005 captured plain-decode head inputs an int8 copy re-scores 1.3–1.6 rows per decision and leaves 0.35% of rows to the stock kernel under the Hopper model and 1.40% under the conservative one; no certified token differed from the engine’s. The runtime is [pending D7].
4. *A negative result with its mechanism* (Section 5): on both public drafters the drift is two orders of magnitude above the threshold the weights set, and certified transport skips

at most 0.65% of the vocabulary, no more than a static screen.

2 THE STOCK HEAD AND THE EXACTNESS CONTRACT

A certificate is a statement about a specified computation, so “exact” is incomplete until it names that computation. This section names it for SGLang’s output head.

2.1 What the Engine Computes

Qwen3.5-4B ties its output head to its input embedding: $W \in \mathbb{B}^{V \times D}$ with $V = 248,320$ and $D = 2,560$, where $\mathbb{B}$ denotes the BF16 numbers. At the pinned commit SGLang computes the head as `torch.matmul` of the BF16 head inputs with $W^\top$, a cuBLAS GEMM that accumulates in FP32, rounds each output to BF16 and chooses its kernel by the batch shape M . The logits are widened exactly to FP32, and greedy decoding and greedy verification take `torch.argmax`, which returns the smallest maximal index. BF16 keeps eight significant bits, so logits in [16, 32) are 0.125 apart, and two real logits closer than about one spacing can round to the same value, leaving the index to decide. With no top- k , top- p or min- p the seeded sampler is already a race: after an FP32 softmax and logarithm it returns $\arg \max_i (\log p_i + G_i)$ in FP64, with Gumbel noise G_i computed from a 32-bit hash of (seed, position, token), a *counter-keyed field* that can be recomputed for any token in any order. It exists only with `--enable-deterministic-inference`, whose batch-invariant `mm` runs, at the pin’s defaults on this GPU, DeepGEMM’s deterministic BF16 GEMM (one FP32 wmma accumulator, no split- K , round-to-nearest output); with a seed the stock head is that kernel. Table 1 fixes the terms used below.

2.2 Four References for One Decision

Let $h \in \mathbb{B}^D$ be the head input, $z_i = \langle w_i, h \rangle$ the *real logit* of row i in exact arithmetic, $\hat{y}_i$ the stock kernel’s FP32 accumulator and $L_i = \text{rn}_B(\hat{y}_i)$ its BF16 output, where rn_B rounds to the nearest BF16 value (ties to even) and $\text{ulp}_B(x)$ is the BF16 spacing at $|x|$.

Definition 2.1 (References for one decision). For $T > 0$ and a noise vector $g \in \mathbb{R}^V$, R-real returns $\min\{i : z_i = \max_j z_j\}$; R-bf16 returns $\min\{i : \text{rn}_B(z_i) = \max_j \text{rn}_B(z_j)\}$; R-stock returns $\min\{i : L_i = \max_j L_j\}$; and S-race returns $\min\{i : z_i/T + g_i = \max_j (z_j/T + g_j)\}$.

R-real and R-bf16 depend on (W, h) alone; R-stock is what an unmodified server returns, and it can depend on the reduction order and so on the batch shape. The R-real token differs from the engine’s at about 0.5% of captured greedy positions (Section 3). R-bf16 is batch-invariant but not the engine’s computation either, because the stock kernel rounds

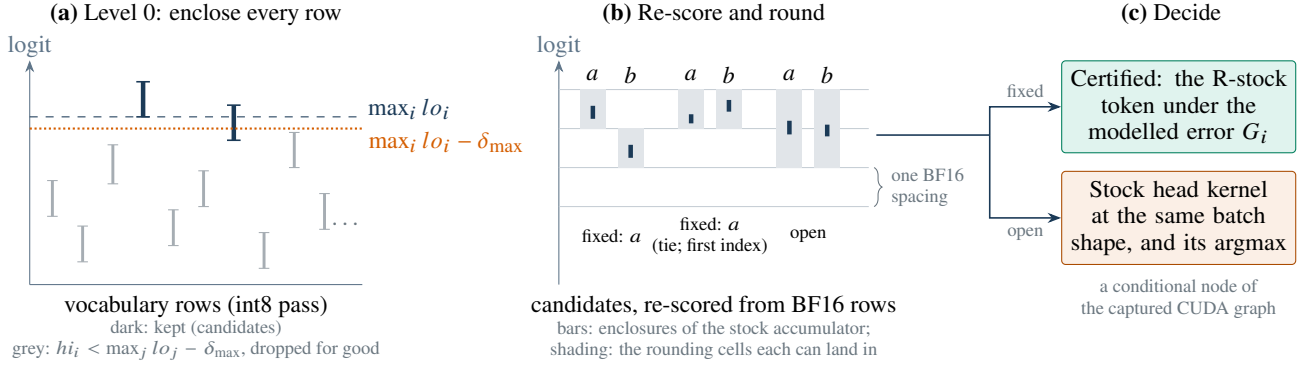


Figure 2. The certified head, schematically (intervals not to scale). (a) An int8 pass encloses every row’s real logit in $[lo_i, hi_i]$. Rows with $hi_i < \max_j lo_j - \delta_{\max}$ lose to every possible winner by more than the stock kernel’s modelled accumulation error plus one BF16 spacing, so, given valid enclosures, they are dropped for good; one or two rows usually survive. (b) The survivors are re-scored from their BF16 rows, and each enclosure of the stock kernel’s FP32 accumulator is rounded into the BF16 values $[L_i^-, L_i^+]$ it can produce (shaded): values in separate rounding cells fix the order, two enclosures within one cell tie and the lower index wins, and two that straddle the same rounding boundary leave the argmax open. (c) A fixed argmax certifies the token; otherwise the stock head runs at the same batch shape.

Table 1. Terminology.

Term	Meaning
Head input h	The BF16 vector the engine feeds to the output head (the final norm’s output).
Real logit z_i	$\langle w_i, h \rangle$ in exact arithmetic over the stored BF16 operands.
Stock head	SGLang’s head at the pin: a cuBLAS BF16 GEMM with FP32 accumulation and BF16 output, then <code>torch.argmax</code> (first maximal index).
Batch shape M	The number of head inputs in one head GEMM call; cuBLAS chooses its kernel by it.
R-real, R-bf16, R-stock, S-race	The four references for one decision (Definition 2.1).
Error model γ , G_i	An assumed bound $G_i = \gamma \sum_j w_{ij} h_j $ on the stock kernel’s accumulation error (Table 2).
Envelope β_i	A proved bound on the error of the low-precision copy’s value for row i .
Candidate	A row the level-0 screen cannot exclude; it is re-scored from its BF16 row.
Fallback	The stock head on the same inputs at the same batch shape; whole-batch or row-wise.
Self-evidence; transport	Enclosures from a low-precision copy of the head at the target’s own input; enclosures carried from a drafter’s per-tile summaries to it.
ρ ; δ_i^*	Drift ratio $\ h^t - h^d\ _2 / \ h^t\ _2$ of a draft pair; the ρ below which transport beats int8 on row i .
Held-out	From prompts that no choice of head, envelope or threshold was fitted on.

its own FP32 accumulation, which can land on the other side of a BF16 boundary; the exact reference has a three-column witness where the two choose different tokens [E17].

Lemma 2.2 (BF16 separation). *For real y_a, y_b , if $y_a - y_b > \text{ulp}_B(\max(|y_a|, |y_b|))$ then $\text{rn}_B(y_a) > \text{rn}_B(y_b)$.*

Theorem 2.3 (Gap condition). *Let $|\hat{y}_i - z_i| \leq G_i$ for every i , and let a be the R-real token. If for all $b \neq a$*

$$z_a - z_b > \delta_{ab} := G_a + G_b + \text{ulp}_B(\max(|z_a|, |z_b|) + \max(G_a, G_b)),$$

then R-stock returns a .

Proof. $\hat{y}_a - \hat{y}_b > \text{ulp}_B(\max(|z_a|, |z_b|) + \max(G_a, G_b)) \geq \text{ulp}_B(\max(|\hat{y}_a|, |\hat{y}_b|))$, since ulp_B is nondecreasing in the magnitude; Lemma 2.2 gives $L_a > L_b$. $\square$

Everything rests on G_i , a bound on a closed-source kernel’s accumulation error, written $G_i = \gamma \sum_j |w_{ij} h_j| + \eta$, where the floor $\eta = 2^{-113}$ covers underflow and flush to zero of products and partial sums (at most $2D \cdot 2^{-126}(1 + \gamma)$), under two named models of γ . The *conservative model* charges $2D$ roundings of relative size 2^{-23} in any order, $\gamma = 6.11 \times 10^{-4}$; it covers any reduction tree, split- K with FP32 partials, and truncating adders. The *Hopper model* takes Khattak and Mikaitis’s measurement-based model of Hopper’s BF16 wgmma (Khattak & Mikaitis, 2026) and adds an FP32 split- K allowance, $\gamma = 1.193 \times 10^{-4}$. Neither is a vendor contract, and neither covers split- K partials stored in BF16, which PyTorch permits by default.

The margin must hold against every competitor, including rows never re-scored. With enclosures $lo_i \leq z_i \leq hi_i$, a competitor j may be excluded only when $hi_j < lo_a - \delta_{aj}$, where $\delta_{aj} \geq \delta_{aj}$ replaces $|z|$ by the largest endpoint magnitude, not merely when $hi_j < lo_a$. Over all rows, $\delta_{\max} = 2G_{\max} + \text{ulp}_B(M_e + G_{\max})$, with M_e the largest endpoint magnitude, bounds every δ_{aj} , so, given valid enclosures, a row with $hi_j < \max_i lo_i - \delta_{\max}$ can be dropped for good (Figure 2a).

2.3 The Interval Form and the Fallback

The gap condition demands a margin above one BF16 spacing even when G is negligible, which 1.95% of captured

positions lack (Section 4.4). The rounding boundaries can be used exactly instead. Rounding is monotone, so an enclosure $[\hat{y}_i^-, \hat{y}_i^+]$ of row i 's accumulator puts L_i in $[L_i^-, L_i^+] = [\text{rn}_B(\hat{y}_i^-), \text{rn}_B(\hat{y}_i^+)]$ (Figure 2b).

Proposition 2.4 (Interval form). *Let C be a set of rows such that every $j \notin C$ has $L_j^+ < \max_{i \in C} L_i^-$, and let $k = \min\{i \in C : L_i^- = \max_{j \in C} L_j^-\}$. If $L_j^+ < L_k^-$ for every $j \in C$ with $j < k$, and $L_j^+ \leq L_k^-$ for every $j \in C$ with $j > k$, then $R\text{-stock}$ returns k .*

Proof. For $j < k$ in C , $L_j \leq L_j^+ < L_k^- \leq L_k$; for $j > k$ in C , $L_j \leq L_j^+ \leq L_k^- \leq L_k$, so a later index can at most tie and the first-index rule returns k ; rows outside C satisfy $L_j < L_k^- \leq L_k$. $\square$

Two rows whose logits round to the same BF16 value under every admissible accumulation are thus certified, the tie broken by index; only a tie the accumulation error could make or unmake stays open.

The contract A certified head claims $R\text{-stock}$ at the batch shape it serves: for each row, the token the stock head returns for the same head input in a batch of the same shape. It certifies a row only when Theorem 2.3 or Proposition 2.4 fixes that token for every accumulator value the named error model admits. Every other row runs the stock kernel. The assumption-free fallback reruns the same `torch.matmul` on the whole batch at the same M , so cuBLAS selects the same kernel; it costs time, never agreement, but a batch with one undecided row pays for the whole stock head. A row-wise mode finishes an undecided row with one stock `matmul` over its gathered candidate rows, which is exact only if a row's BF16 logits do not depend on the rest of the batch or on which head rows are present [pending D3]. Either fallback is a conditional node of the captured CUDA graph, paid only when it runs.

2.4 Numerical Assumptions

Table 2 lists every rounding, accumulation, tie-breaking and error-model assumption and the results that depend on it. Facts about the formats and the stock chain are read from the code; the stock kernel's error model is assumed and only tested; the replay approximates real arithmetic in FP64.

2.5 Comparisons at the Same Batch Shape

Stock configurations that ought to be equivalent already disagree. On one server, plain greedy decoding at client concurrency 1 and 32 (decode batches of up to 16) emits different tokens in 167 of 320 prompts within 256 tokens, 3.42 divergences per 1,000 compared tokens, while repeating either configuration there reproduced every token [E24].

With the prefix cache on, a repeat is not an equality reference in general: with smaller, pinned pools and a cap of 8 running requests instead of 16, a same-server repeat at concurrency 1 differs in 17 of 320 prompts (0.25 per 1,000) and a fresh-server repeat at concurrency 32 in 4, all at exact ties or within one BF16 spacing; the working explanation, not checked at the level of the cache, is history dependence through the prefix cache (Appendix D.4) [E28]. A bit-exact tap of every module output, each pair on two servers with different pools, finds the runs parting at batch-dependent kernels (a gated RMSNorm, a batched prefill GEMM, and decode attention, for which the pools and the prefix cache's history are possible causes too; Appendix D.4), and plain decoding and MTP speculation parting at layer 0's recurrence, whose decode and verify kernels differ; the tie rule and the head GEMM caused none of the divergences [E23]. For plain decoding against MTP, every cache entering every forward agrees up to that recurrence on 40 prompts rerun with cache hashes [E26]; for the concurrency pair, whether the caches agree is [pending D11]. A certified head is therefore compared with the stock head at the same batch shape and configuration, and a speculative arm with stock speculative verification, not with plain decoding, from which MTP speculation diverges at 3.5 to 4.0 per 1,000 compared tokens even with identical pools, always at an exact tie or within two BF16 spacings [E28].

3 EVALUATION SETUP

System One NVIDIA GH200 (96 GB HBM3, compute capability 9.0, 64-core Grace CPU) with PyTorch 2.13 for CUDA 13.0, Triton 3.7.1 and FlashInfer 0.6.18; SGLang (Zheng et al., 2024) at commit bd66ce343e4f with the FlashInfer attention backend. The target is Qwen3.5-4B at revision 851bf6e806ef (Qwen, 2026a): 32 layers, 24 of them Gated DeltaNet linear attention (Yang et al., 2025) and 8 full attention, one native MTP layer, and the tied head of Section 2.1. The drafters are the MTP layer (four draft steps, top- k 1) and the public DFlash block drafter for this target, z-lab/Qwen3.5-4B-DFlash at revision 9a1996cc (Chen et al., 2026a; z-lab, 2026), which drafts blocks of 16 tokens and projects them through the target's head.

3.1 Capture and Replay

A capture-only engine patch stores the head input as BF16 bit patterns and joins it, within the same step, with the tokens the engine chose; under speculation it also stores the draft head inputs, the proposals and the accepted lengths, and pairs each drafted token with the target head input that verifies it (labelled accepted, rejected or unreached). The hooks run in Python between forwards, so the capture servers run without CUDA graphs or the overlap scheduler, with at most 16 running requests (plain decoding), 8 (MTP) or 5 (DFlash).

Table 2. Numerical assumptions and the results that depend on them. “Pin”: code reading of SGLang at bd66ce343e4f with PyTorch 2.13; $D = 2,560$.

Assumption	Status	Results that depend on it
Head weights and inputs are BF16 and are normal or zero. Every product $w_{ij}h_j$, and every product of an int8 code with a BF16 input, is exact in FP32 unless it underflows; underflow and flush to zero of products and partial sums cost less than 2^{-126} each and are charged to the absolute floor η .	Exact in the normal range; normal operands assumed.	Thms. 2.3, 4.2; Prop. 2.4
The stock head rounds its FP32 accumulator to BF16 (nearest, ties to even) and widens it exactly; <code>torch.argmax</code> returns the smallest maximal index in greedy decoding and verification.	Pin.	Def. 2.1; Lem. 2.2; Prop. 2.4
Stock accumulation error $ \hat{y}_i - z_i \leq \gamma \sum_j w_{ij}h_j + \eta$ at every served shape: $\gamma = 6.11 \times 10^{-4}$ (conservative model: $2D$ roundings at 2^{-23} , any order) or $\gamma = 1.193 \times 10^{-4}$ (Hopper model: Hopper wgmma per Khattak and Mikaitis, plus FP32 split- K); $\eta = 2^{-113}$; no split- K partials stored in BF16. The replay omitted η , which can change a decision only if an endpoint lies within 2^{-112} of a BF16 rounding boundary or a gap within 2^{-112} of its threshold.	Assumed; consistent with all replayed rows; kernel error and the implementation’s floor [pending D2, D1].	Certification against R-stock; Table 4
A row’s stock BF16 logits do not depend on the other rows of the batch or on which head rows are present.	Assumed; [pending D3].	Row-wise fallback only
Comparisons are made at the same batch shape and serving configuration.	Required (Section 2.5).	Every agreement claim
The low-precision pass accumulates in FP32 on tensor cores under the conservative model, or exactly in int32 for int8 codes times int8 activation codes.	Model assumed; integer pass exact.	Thm. 4.2; Table 3
The low-precision kernel computes the modelled arithmetic: the scaled products of codes and inputs, accumulated in FP32 within γ or exactly in int32, and the envelope epilogue as analysed.	Required, not proved; validation per compiled variant [pending D1]. A TMA int8 variant produced incorrect envelopes, which the start-up self-test rejects [pending D1].	Tokens certified by a kernel, not by the replay; its runtime
Re-scoring sums the exact BF16 products of each candidate in FP64 on CUDA cores under IEEE rounding, with error at most $\gamma_D (2^{-53}) \sum_j w_{ij}h_j < 3 \times 10^{-13} \sum_j w_{ij}h_j $, which widens its accumulator enclosure.	Design; implementation [pending D1].	Decision by Prop. 2.4
Norms of h are rounded up (ℓ_1 reduction depth at most 2^{13}); interval endpoints are rounded outward; offline constants are exact and stored rounded up.	Proved (Appendix A).	Every computed enclosure
Untruncated seeded noise is a function of (seed, absolute position, token) through a 32-bit hash, evaluated in FP64 after an FP32 softmax and logarithm; statements in law idealize it as i.i.d. continuous Gumbel. The seeded sampler exists only in deterministic-inference mode, whose head is SGLang’s batch-invariant mm, here DeepGEMM’s deterministic BF16 GEMM (one FP32 accumulator, no split- K , round-to-nearest output): the conservative model covers it; the Hopper model is unchecked for it. Undecided sampled rows run the stock sampler.	Pin; the kernel’s error and the rounding slack of the stock chain [pending D2, D8].	Thms. 4.4, 4.5; sampling contract
Replay: real logits are evaluated in FP64 from the stored operands (error below $D 2^{-52} \sum_j w_{ij}h_j $); draft logits are exact, ignoring the BF16 rounding a served draft pass adds, which favours transport.	Replay convention.	Tables 3–5; Figs. 3, 4
Transport is in real arithmetic with one head for both passes; δ^* uses Euclidean envelopes, 64-row tiles of contiguous token indices and tile-mean centres.	Stated; other geometries in Table 5.	Thms. 5.1, 5.2; Fig. 4

The prompts are 320 public prompts in four domains (MT-Bench, No Robots, HumanEval, MBPP, GSM8K, MATH-500, m-ArenaHard and MGSM, 40 each), a third in thinking mode, decoded greedily for 384 tokens and split by prompt into analysis and held-out halves before any fitting; every result is on held-out prompts.

The capture is what the head read In plain decoding the FP64 argmax of the captured head input equals the engine’s token at 99.51% of 105,269 positions, and all 519 disagreements lie within one BF16 spacing of the FP64 winner [E1].

What is compared with what Candidate counts are for the R-real decision on 6,005 held-out plain-decode rows from 160 prompts and 750 decode steps (8.0 held-out rows per step), with real logits in FP64; fallback shares apply the R-stock rules to the same rows, and agreement compares each certified token with the engine’s token at the capture’s batch shape. Drift uses 40,000 held-out pairs of each drafter,

and transport certification 4,020 DFlash-4B pairs from 110 prompts and 16,016 MTP-4B pairs. No served throughput or latency is reported.

4 THE CERTIFIED HEAD

The evidence for a certificate comes from a low-precision copy of the head evaluated at the target’s own head input, which we call *self-evidence*. The copy bounds every row, the rows whose intervals can still matter are re-scored from their BF16 rows, Proposition 2.4 decides, and the stock kernel completes whatever remains (Figure 2).

4.1 An Envelope for Tensor-Core Accumulation

Lemma 4.1 (Accumulation trees). *Let $s = \sum_{\ell} p_{\ell}$ be evaluated by a tree whose internal nodes v return $\hat{s}_v$ with $|\hat{s}_v - \sum_c \hat{s}_c| \leq u_v \sum_c |\hat{s}_c| + \eta_v$ over their children c . With d the largest number of internal nodes on a leaf-to-root path, N the number of internal nodes, $u \geq \max u_v$, $\eta \geq \max \eta_v$ and $du < 1$: $|\hat{s} - s| \leq \gamma_d \sum_{\ell} |p_{\ell}| + N\eta(1 + \gamma_d)$, where*

$$\gamma_d = (1 + u)^d - 1 \leq du/(1 - du).$$

The tree’s shape is all that distinguishes CUDA-core, tensor-core and split- K accumulation. An IEEE addition has $u_v = u$ (Higham, 2002). A tensor-core adder aligns $k + 1$ addends to the largest, truncates each to F fractional bits and rounds once (Fasi et al., 2021), so $u_v = (k+1)2^{-F} + 2^{1-P}$ over depth $\lceil n/k \rceil$. Khattak and Mikaitis model Hopper’s BF16 wgmma as $k = 16$, $F = 25$ with truncation (Khattak & Mikaitis, 2026), which gives $\gamma \approx 1.0 \times 10^{-4}$ at $D = 2,560$; the conservative model, $\gamma = 6.11 \times 10^{-4}$, covers it about sixfold for any reduction order.

Theorem 4.2 (Envelope). *Let row i of the copy have group scales s_{ig} , integer codes and a set of columns kept exact, with dequantized row $\bar{w}_i$ and exact error $e_i = w_i - \bar{w}_i$. If every product is exact in the accumulator format and the pass’s accumulation obeys Lemma 4.1, its value $\tilde{z}_i$ satisfies $|z_i - \tilde{z}_i| \leq \beta_i = Q_i + \gamma_d \|\bar{w}_i\|_2 \|h\|_2 + N\eta(1 + \gamma_d) \max(1, \max_g s_{ig})$, where Q_i is $\|e_i\|_2 \|h\|_2$ or any block-wise Cauchy–Schwarz or Hölder form.*

Proofs are in Appendix A. An int8 pass that also quantizes the activations and accumulates in int32 needs no floating-point accumulation model, at the price of the activations’ quantization error (Theorem A.1); its kernel must still compute the analysed products. The norms of h are rounded up and the endpoints $\tilde{z}_i \mp \beta_i$ rounded outward, so every computed comparison is sound (Lemmas A.2 and A.3), provided the kernel evaluates the analysed operations (Table 2). The envelope assumes worst-case alignment of e_i and h : for per-row int8 its median half-width on captured states is 1.00 logit, 75 times the median realized error of 0.013 [E5].

4.2 Refinement Against the Stock Kernel

Theorem 4.3 (Refinement ladder). *Let levels $\ell = 0, \dots, L$ supply enclosures $lo_i^\ell \leq z_i \leq hi_i^\ell$, exact at level L , and let each f_i be nondecreasing. Starting from rows S , at each level let $\tau = \max_{i \in S} f_i(lo_i^\ell)$ and keep $\{i \in S : f_i(hi_i^\ell) \geq \tau\}$; stop when one row remains, every kept interval is a point, or $\ell = L$, and return the smallest kept index. The result is $\min \arg \max_{i \in S} f_i(z_i)$, whatever the precision or reduction order of each level; $f_i(x) = x$ gives R-real and $f_i(x) = x/T + g_i$ gives S-race.*

Lean proves the certificate and both filter properties in scaled integers, not under IEEE rounding [E18]. For R-stock the filter is not enough: a row dropped because $hi_i < \max_j lo_j$ is known to lose in real arithmetic, not by the margin of Theorem 2.3. The certified head therefore widens the level-0 threshold by $\delta_{\max}$, re-scores the survivors in FP64, widens each by G_i to an enclosure of the stock accumulator, and decides with Proposition 2.4. Given valid enclosures, a dropped row loses by more than $\delta_{\max}$, so the screen changes which rows are re-scored but never which rows the deci-

sion leaves open. How many rows survive depends on the envelope’s width against the logit margins: for a Gumbel race at $T = 1$ the expected number is at most $\min(V, e^{4\beta})$ (Proposition A.4), 55 rows at $\beta = 1$ and 2,981 at $\beta = 2$, so exact sampling through the copy is cheap only for envelopes up to about two logits wide.

4.3 Sampled Decisions Within the Contract

Theorem 4.4 (Certified race). *For a fixed noise vector g and $T > 0$, the ladder with $f_i(x) = x/T + g_i$ returns the token of a dense race with the same g and tie rule. If the g_i are i.i.d. standard Gumbel and independent of z , that token has law $\text{softmax}(z/T)$.*

Proof. Theorem 4.3 and the Gumbel-max identity (Gumbel, 1954; Papandreou & Yuille, 2011); ties have probability zero. $\square$

Bounding scores to skip most perturbations is the principle of A* Sampling and of lazy Gumbel sampling (Madison et al., 2014; Musmann et al., 2017); a fixed field adds pathwise equality with the dense race. That needs a counter-keyed field: in the exact reference, feeding a stream consumed in evaluation order to the candidate rows changed the winner in 247 of 300 cases [E17]. The stock seeded sampler already has the required form.

Theorem 4.5 (Fixed-noise verification). *Let $G_{t,i}$ be keyed by (seed, absolute position t , token i) and i.i.d. standard Gumbel, and let $Y_t = \min \arg \max_i (z_{t,i}(Y_{<t})/T + G_{t,i})$. A speculative decoder that accepts a drafted token if and only if it equals $\arg \max_i (z_{t,i}(x_{<t})/T + G_{t,i})$ at its drafted prefix, and emits that argmax at the first mismatch, outputs Y for every drafter and depth policy, and Y has the target’s autoregressive law.*

This is the drafter-invariant construction of Daliri et al. (2025). Sampled verification becomes an argmax, which Theorem 4.4 certifies without a partition function or a residual sampler. The certified path adopts it with exactly the stock seeded noise. The output is then equal in law to target sampling and independent of the drafter; for SGLang’s default point-mass drafts a draft d is accepted with probability $p(d)$, as under the stock rule; and with a seed and no truncation every committed token is the stock seeded sampler’s token for the verify pass’s own R-stock logits, provided that rows whose race the certificate cannot decide under the stock rounding of the softmax and logarithm run the stock sampler. The reference is the engine in deterministic-inference mode, the only one with a seeded sampler, whose head GEMM (Section 2.1) the conservative model covers and the Hopper model has not been checked for. It equals seeded decoding without speculation only where verify-pass and plain-decode logits agree, which the engine does not guaran-

Table 3. Candidate rows per greedy decision on 6,005 held-out plain-decode positions; conservative model for the copy’s accumulation; row Cauchy–Schwarz envelope for int8 per row, blockwise ℓ_2 otherwise. “Bytes”: the copy relative to the BF16 head; “1 row”: decided without re-scoring. Measured on replayed states [E3].

Copy	Bytes	Candidate rows			Decided with	
		mean	p99	max	1 row	≤ 8
int8, per row	0.50	1.55	8	30	75.4%	99.3%
int8, 128 cols	0.52	1.34	5	23	81.4%	99.8%
int8, 32 cols	0.54	1.26	5	20	84.0%	99.9%
FP8 E4M3, row	0.51	4.89	60	605	56.2%	88.4%
int4, 128 cols	0.27	72,847	248,295	248,320	0%	0%

Table 4. Share of the 6,005 held-out rows, and of the capture’s 750 decode steps (counting each step’s held-out rows, 8.0 on average), that the contract leaves to the stock kernel, by rule and by the model assumed for the stock kernel’s accumulation. The last two rows are sensitivity rows that do not model the stock tensor-core GEMM. Measured on replayed states [E6].

Stock model	γ	Gap (Thm. 2.3)		Interval (Prop. 2.4)	
		rows	steps	rows	steps
Conservative	6.11×10^{-4}	3.03%	21.6%	1.40%	10.9%
Hopper	1.193×10^{-4}	2.18%	16.0%	0.35%	2.8%
Sensitivity	10^{-5}	1.97%	14.9%	0.017%	0.13%
IEEE FP32 tree	1.7×10^{-6}	1.95%	14.8%	0%	0%

tee (Section 2.5). The rounding slack and token agreement on replayed steps are [pending D8]; the standard acceptance rule and residual races are analysed, not built (Appendix B).

4.4 Candidates and Fallback on Captured States

Table 3 and Figure 3a give the candidate sets on 6,005 held-out plain-decode positions for five copies of the head. No envelope was violated, and the exact winner was always a candidate [E2, E3]. An int8 copy with one scale per row needs 1.55 candidate rows on average (median 1, p99 8) and decides 75.4% of positions without re-scoring; with 128-column groups the mean is 1.34 at 0.52 of the BF16 head’s bytes. FP8 E4M3 needs 4.9 rows on average and 60 at p99, and int4 alone is useless. The reason int8 suffices is the margin distribution, not the envelope’s tightness: the envelope is about one logit wide, but the median gap between the two largest logits is 7.3 logits (p10 0.68) [E5]. Tightening the envelope helps little: 16 to 64 exact outlier columns or a 64-dimensional PCA rotation of h lower the mean count by 3–7% for 1–7% more pass bytes, and centring W raises it by 10% [E3, E5]. Races at $T = 1$ and 0.7 behave like greedy selection (mean 1.33 and p99 5 for the 128-group copy). Under the elementwise minimum of these rigorous bounds, the *minimum envelope*, the per-row copy needs 1.54 rows on plain decoding, and 1.57 (DFlash-4B, 1,162 of 4,000 rows) and 1.51 (MTP-4B, 2,901 of 4,010) on the verify rows a greedy verifier needs [E3, E7, E8].

Table 4 gives how often the contract leaves a row to the stock kernel. The gap condition has a floor near 2%; the interval form has none. Under the Hopper model the interval form leaves 0.35% of rows to the stock kernel, and 2.8% of the capture’s decode steps contain such a row; under the conservative model, 1.40% and 10.9%. For every model, no row the interval form certified had a token different from the one the engine returned at the capture’s batch shape [E6]; that is consistent with the error models on these rows, not a proof that cuBLAS stays within the smallest of them.

The shares grow with the batch. If rows fell back independently with probability p , a batch of B rows would contain an undecided row with probability $1 - (1 - p)^B$. At the capture’s mean of 8.0 rows per step this gives 2.8% and 10.7%, close to the measured 2.8% and 10.9%; at a serving batch of 128 it gives 36% and 84% (Figure 3b). A whole-batch fallback would then run the stock head for most large batches, so the certified head’s value at serving batch sizes rests on the row-wise fallback, on the row invariance it needs, and on its cost [pending D3, D5].

4.5 Kernel and Runtime

The path for M head inputs is designed as a short pipeline with grids fixed by M , captured once per batch size with its fallback. An int8 pass accumulates on tensor cores in FP32, evaluates the envelope in its epilogue with directed rounding, and keeps per tile the four largest upper bounds and the largest remaining one instead of writing the $M \times V$ logits; compaction keeps every row above the threshold (all rows of a tile whose remaining bound reaches it), the candidates are re-scored in FP64, Proposition 2.4 decides, and the stock head runs only as the conditional fallback. On capture steps with 5 to 16 held-out rows the 128-group candidates cover 9.5–12.5 distinct head rows, so the gather reads tens of kilobytes beside a 636 MB int8 pass. The path pays off only if

$$T_{\text{pass}} + T_{\text{select}} + T_{\text{rescore}} + P_{\text{fb}}(M) T_{\text{stock}}(M) < T_{\text{chain}}(M), \quad (1)$$

where T_{chain} is the stock GEMM with its FP32 copy and argmax and $P_{\text{fb}}(M)$ the share of batches that fall back. At $M = 1$ the stock GEMM takes 354.5 μs , 95% of the 3.79 TB/s read bandwidth measured over the head’s 1.27 GB, and the chain 369.0 μs [E31, E32], so a pass that reads half the bytes has a floor near 170 μs (derived). The implementation and its head-path times for $M = 1$ to 256, per error model, are [pending D1, D7].

5 TRANSPORT AND WHY IT FAILS

A drafter that projects through the target’s head has already computed each vocabulary tile’s largest score and probability mass. *Transport* carries those summaries to the target’s head input through a Hölder bound, so the verifier projects

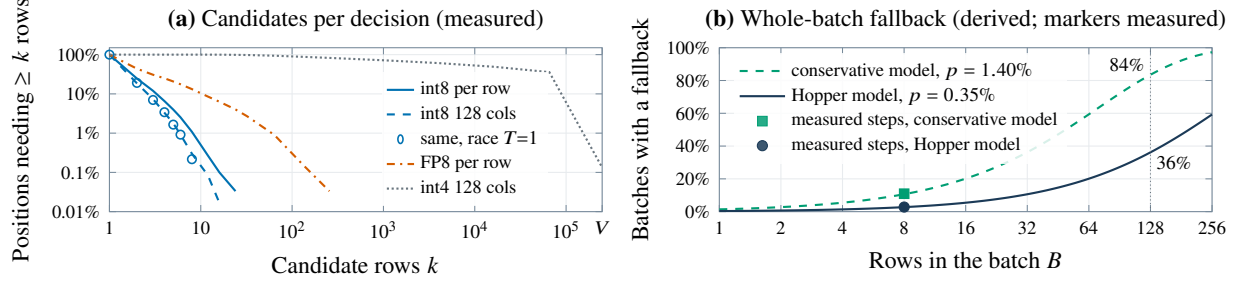


Figure 3. Re-scoring and fallback per decision. (a) Measured share of the 6,005 held-out plain-decode positions that need at least k candidate rows, greedy unless marked [E4]. (b) Share of batches of B rows with a row the interval form leaves to the stock kernel, if rows fell back independently with the per-row shares of Table 4 (derived, $1 - (1 - p)^B$); markers: the measured shares of the capture’s decode steps.

only tiles whose bounds leave its decision open.

5.1 Transported Tile Certificates

Let h^d and h^t be aligned draft and target head inputs (each the final norm’s output) behind the same verified draft token, $\Delta = h^t - h^d$, $z_i^d = \langle w_i, h^d \rangle$ and $z_i^t = \langle w_i, h^t \rangle$. A partition of the vocabulary into tiles c carries, as checkpoint metadata, a centre μ_c and a radius $r_c \geq \max_{i \in c} \|w_i - \mu_c\|_2$. The draft pass emits $M_c^d = \max_{i \in c} z_i^d$ and $Z_c^d = \sum_{i \in c} e^{z_i^d}$.

Theorem 5.1 (Transported certificates). *In real arithmetic, with $a_c = \langle \mu_c, \Delta \rangle$ and $\epsilon_c = r_c \|\Delta\|_2$, every $i \in c$ satisfies $|z_i^t - z_i^d - a_c| \leq \epsilon_c$. Hence $\max_{i \in c} z_i^t \leq M_c^d + a_c + \epsilon_c =: U_c$ and $e^{a_c - \epsilon_c} Z_c^d \leq Z_c^t \leq e^{a_c + \epsilon_c} Z_c^d$.*

Proof. $z_i^t - z_i^d - a_c = \langle w_i - \mu_c, \Delta \rangle$, which Cauchy–Schwarz bounds by ϵ_c . Maximizing over c gives the first bound; exponentiating and summing gives the second. $\square$

Hölder, coordinate, grouped and low-rank radii trade metadata for width (Appendix C). A greedy verifier may skip tile c when $U_c < \tau$ for an exactly evaluated target score τ , such as the draft token’s.

5.2 What Transport Must Beat

Self-evidence’s error for row i is $\langle e_i, h^t \rangle$, bounded by $\|e_i\|_2 \|h^t\|_2$ with e_i known offline; transport’s is $\langle w_i - \mu_c, \Delta \rangle$, bounded by $r_c \|\Delta\|_2$. Each bound is attained by some admissible vector, so each is optimal for the information it uses (Proposition C.2), and the comparison reduces to one ratio, the *drift ratio*

$$\rho = \|h^t - h^d\|_2 / \|h^t\|_2. \quad (2)$$

Theorem 5.2 (Crossover). *For every tile and centre, $r_c \geq \frac{1}{2} \text{diam}_2(c)$. Transport’s envelope for row $i \in c$ is narrower than self-evidence’s, $r_c \|\Delta\|_2 < \|e_i\|_2 \|h^t\|_2$, if and only if $\rho < \delta_i^* := \|e_i\|_2 / r_c$.*

The weights alone fix δ^* . On the Qwen3.5-4B head, with 64-row tiles of contiguous token indices and tile-mean centres, δ^* has a median of 0.85% over all rows (p10 0.67%, p90 1.15%) against int8 with one scale per row, 0.61% against 128-column groups and 15.5% against int4 [E10]. Transport can compete with an int8 copy only if draft and target head inputs agree to about one part in a hundred, which a drafter trained to predict tokens has no reason to achieve. The tile radius itself exceeds the row norm (median $r_c / \|w_i\|_2 = 1.14$), so a static tile bound $\langle \mu_c, h \rangle + r_c \|h\|_2$ is wider than the plain row bound, and transport is no tighter than that static screen once the drift is as large as h (Proposition C.5).

5.3 The Measured Drift

For the public DFlash drafter at block size 16, over 40,000 held-out pairs, ρ has a median of 0.917 (p10 0.875, p90 0.964, smallest 0.823) [E11]. For the model’s own MTP layer with four draft steps, over 40,000 held-out pairs, the median is 0.954 (p10 0.767, p90 1.174) [E14]; both medians lie two orders of magnitude above that of δ^* (Figure 4b). Accepted DFlash drafts are no closer than rejected ones (medians 0.909 and 0.891). Transport’s envelope is the narrower one only on 0.08% of rows, three tiles of identical unused-token rows with zero radius.

Why the drift is large Two properties of the DFlash drafter set its ρ (Figure 4a) [E15]. Its final RMSNorm has unit gain, so its head input always has norm $\sqrt{2,560}$, a third of the target’s, and the triangle inequality alone gives $\rho \geq 0.68$; the angle between them is large as well, so even the best scalar rescaling of h^d would leave ρ near 0.91. The drafter is trained to rank the right token first under Wh^d , not to reproduce the target’s head input, and its logits are softer (median top-token probability 0.17 against 0.69). Nor does the drift hide in directions the head ignores: after removing each position’s mean logit, $\|W\Delta\|_2 / \|Wh^t\|_2$ has a median of 0.85, and the largest realized transport error in a tile is a median 77 times the largest realized int8 error [E11]. The

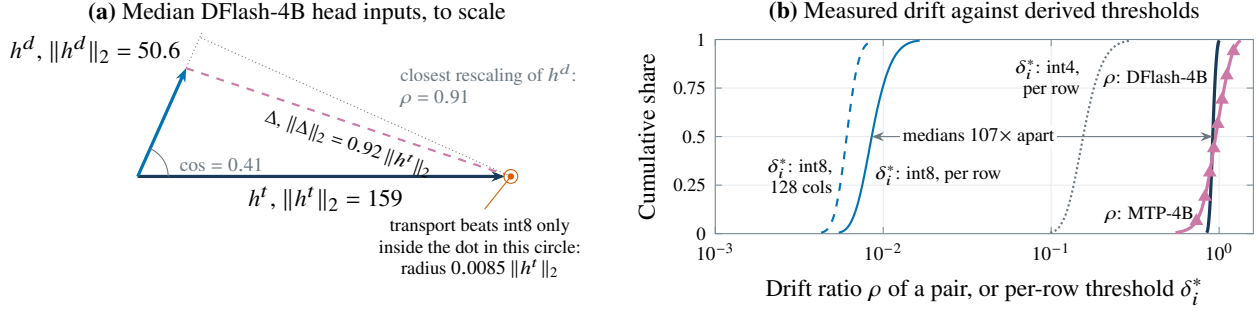


Figure 4. Why transport fails. (a) Median DFlash-4B head inputs, drawn to scale in the plane they span. Transport’s envelope beats a per-row int8 envelope only if h^d lies in the ball of radius $\delta^* \|h^t\|_2$ around h^t , the dot at the tip of h^t for the median δ^* (Theorem 5.2). (b) Cumulative distributions of ρ over 40,000 aligned held-out pairs of each drafter, and of the threshold δ_i^* over the 248,320 head rows (64-row contiguous tiles). Measured drift and medians, derived thresholds [E10, E11, E14, E12, E15].

Table 5. Transport on held-out draft pairs with the best certified bound family per tiling (the elementwise minimum of all certified radii). “Skipped”: mean share of the vocabulary a greedy verifier skips, by transport (T), by the static screen with the same geometry (S) and by oracle radii (O), the realized per-tile deviations that no certified geometry can undercut. “Width”: median partition width $\log(Z^+/Z^-)$ in nats; $P_?$: mean probability, over proposals $x \sim q$ at $T = 1$, that the acceptance test stays undecided. Measured [E12, E13].

Tiling	Skipped (greedy)			Sampled	
	T	S	O	Width	$P_?$
<i>DFlash-4B, block 16: 4,020 pairs, 110 prompts</i>					
Contiguous 64	0.08%	0.08%	89%	133	0.995
Contiguous 256	0%	0%	84%	213	1.000
Contiguous 1,024	0%	0%	77%	244	1.000
Random 256	0%	0%	76%	238	1.000
k -means 64	0.59%	0.58%	84%	134	0.964
k -means 256	0.10%	0.10%	77%	138	0.982
k -means 1,024	0.02%	0.02%	68%	146	0.991
<i>MTP-4B, four steps: 16,016 pairs, 160 prompts</i>					
Contiguous 64	0.08%	0.08%	98.5%	146	1.000
Contiguous 256	0%	0%	96.4%	233	1.000
Contiguous 1,024	0%	0%	92.9%	268	1.000
Random 256	0%	0%	94.7%	255	1.000
k -means 64	0.64%	0.64%	96.0%	146	0.998
k -means 256	0.11%	0.11%	92.3%	152	0.999
k -means 1,024	0.02%	0.02%	86.8%	162	1.000

MTP layer’s head input is slightly longer than the target’s (median norms 176 and 160), so its drift is mainly one of direction [E13].

What certification achieves Table 5 applies every certified bound family of Appendix C, and their elementwise minimum, to contiguous, random and k -means tiles. For greedy verification the best certified family skips at most 0.59% of the vocabulary on average on DFlash-4B pairs and 0.64% on MTP-4B pairs (0.65% for a row-level variant), no

more than the static screen, which does not use the draft at all [E12, E13]. Oracle radii would skip 68–98.5% for one pair, but a verifier that reuses weights across a batch reads the union of the tiles its rows need: over the capture’s real verify batches every certified family and tiling needs at least 99.48% (DFlash-4B) and 99.4% (MTP-4B) of the vocabulary. For sampled acceptance the partition interval spans 133–268 nats at its median, and the test stays undecided with probability 0.964 or more on average.

A drift-scaling replay, $h^d(s) = h^t - s\Delta$, an extrapolation, not a measured drafter, shows how far a drafter would have to move: with the best certified family on k -means 256-row tiles, greedy transport skips 49% (DFlash-4B) and 80% (MTP-4B) of the vocabulary at $s = 0.1$, while sampled acceptance stays undecided with mean probability 1.00 there and 0.66 at $s = 0.01$ for DFlash-4B [E12, E13].

The verdict On both drafters and for every bound family and tiling tested, transport skips almost nothing, no more than a static screen that ignores the draft, and the batch union erases the rest. The cause is geometric: the head inputs differ by order one, not one part in a hundred. On the same DFlash-4B states the int8 copy needs 1.57 candidate rows (Section 4.4).

6 RELATED WORK

Descriptions follow our literature review [E70]; numbers quoted from other work are the authors’ own.

Certified low-precision heads The greedy form of the certified head is public several times over. sparkpipe screens with an E4M3 shadow whose per-group certificate covers CUDA-core FMA accumulation and re-scores survivors exactly, for decode and MTP argmax rows of a model with the same 248,320-row vocabulary (sparkpipe contributors, 2026). dgpp bounds logits from the high bit planes of an int8

head and takes the quantized head as its reference (Hawkins, 2026); a Metal implementation re-scores with a replica of the stock kernel (Layr Labs, 2026); knlp reports an argmax match of 0.998 at 14B parameters (Chamberlain, 2026). Du et al. show why the reference must be named: BF16 and FP16 greedy decoding diverge on 49–100% of prompts (Du et al., 2026).

Screening and sampling Cheap scores with bounds and exact refinement are old in similarity search (Weber et al., 1998; Li et al., 2017; Teflioudi et al., 2015; Ram & Gray, 2012), where dense multiplication often wins end to end (Abuzaid et al., 2019). SVD-Softmax and HiRE recompute a fixed top set with no certificate (Shim et al., 2017; L et al., 2024); CSV-Decode certifies top- k with cluster bounds $\langle \mu_c, h \rangle + R_c \|h\|$ (Liu et al., 2025), the static screen of Table 5, from which transport differs only in starting at the draft state. FlashSampling fuses Gumbel-max sampling into the head over every tile, exact in distribution with tiling-dependent noise (Ruiz et al., 2026). Batch-invariant kernels make outputs reproducible across batches (He & Thinking Machines Lab, 2025; The SGLang Team, 2025), which changes the kernel a certificate must match.

Speculative decoding Drafter-invariant decoding couples draft and target through shared Gumbel noise (Daliri et al., 2025; Rowan et al., 2025), which Section 4.3 uses to make sampled verification a race; DSpark and D-cut schedule verification depth and state the non-anticipating condition for lossless sampled speculation (Cheng et al., 2026; Liu et al., 2026b). Drafters such as EAGLE, native MTP and DFlash project through a head at every draft step (Li et al., 2024a; Gloeckle et al., 2024; Chen et al., 2026a), and draft-vocabulary reduction trades their acceptance for draft cost (Zhao et al., 2025; Goel et al., 2025; Williams et al., 2026); a certified head keeps the full-vocabulary token and also applies to the target.

What is and is not new Norm or quantization bounds, bound-then-refine search, certified greedy screening of an LM head, sampling from bounded scores, drafter-invariant verification and the non-anticipating condition are prior art. For the certified head we claim only an envelope sound for tensor-core accumulation (for a kernel that computes the modelled arithmetic), a contract against a closed-source stock kernel with that kernel as the fallback, the extension to seeded sampling and fixed-noise verification, and measurements on the engine’s own states. Transport of draft-side tile summaries is the one mechanism here without a precedent we found, and it is a negative result.

7 DISCUSSION AND LIMITATIONS

The error model is an assumption Agreement with R-stock rests on a model of a closed-source kernel’s accumulation, which no vendor documents and which does not cover BF16 split- K partials. It was tested on replayed rows (Section 4.4), not proved; the kernel’s measured error is [pending D2], and the whole-batch fallback needs no model.

Replay, not serving Candidate sets and fallback shares come from offline replay of states captured at batches of up to 16. On speculative verify rows, including those after a rejection, only the gap condition was evaluated: it leaves 5.7% (Hopper model) to 8.6% (conservative model) of DFlash-4B rows and 2.8% to 4.0% of MTP-4B rows to the stock kernel, against 2.2% and 3.0% on plain decoding [E7, E8], so verification may fall back more often. The main open question is whether the complete certified head beats the stock chain once it pays for screening, compaction, re-scoring and fallback (Equation (1)); low candidate counts do not settle it. If rows fell back independently, a whole-batch fallback would run in 36% (Hopper model) to 84% (conservative model) of batches of 128 rows (Figure 3b), so at serving batch sizes the case rests on the row-wise fallback [pending D3, D5]; the head-path runtime [pending D7] answers part of this, the served frontiers [pending D34] the rest.

Scope Transport was refuted for two drafters and the bound families and tilings of Table 5, in a replay that favours it; a drafter trained to reproduce the target’s head input is untested. The measurements concern one model, engine commit and GPU, with text-only prompts. The contract and proofs carry over to any head that rounds an FP32 accumulation to BF16 and breaks ties by index, but the error model and fallback shares must be re-established per kernel and GPU. Secondary investigations are in Appendices A.9, E and F.

8 CONCLUSION

A verifier that must agree with a production engine needs less than a full projection of the output head but more than the real-arithmetic winner: the token the engine’s own kernel, with its rounding and tie rule, returns at the same batch shape. On this engine’s states an int8 copy of the head supplies that with one or two rows re-scored per decision and the stock kernel as a model-free fallback; transported draft summaries do not, because draft head inputs are far from the target’s.

REFERENCES

- Abdelfattah, A., Dongarra, J., Fasi, M., Mikaitis, M., and Tisseur, F. Analysis of Floating-Point Matrix Multiplication Computed via Integer Arithmetic, 2025. URL <https://arxiv.org/abs/2506.11277>. arXiv:2506.11277v3, 8 May 2026 (v1 12 Jun 2025).
- Abuzaid, F., Sethi, G., Bailis, P., and Zaharia, M. To Index or Not to Index: Optimizing Exact Maximum Inner Product Search. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, pp. 1250–1261, 2019. doi: 10.1109/ICDE.2019.00114. URL <https://arxiv.org/abs/1706.01449>. arXiv:1706.01449v3, 15 Mar 2019 (v1 5 Jun 2017).
- An, Z., Liu, T., Liu, Z., Li, D., Liu, R., and Barsoum, E. PARD-2: Target-aligned parallel draft model for dual-mode speculative decoding, 2026. URL <https://arxiv.org/abs/2605.08632v1>. arXiv:2605.08632v1, 9 May 2026.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>.
- Ben Shoham, O. Balancing Coverage and Draft Latency in Vocabulary Trimming for Faster Speculative Decoding, 2026. URL <https://arxiv.org/abs/2603.05210>. arXiv:2603.05210v1, 5 Mar 2026.
- Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 5209–5235. PMLR, 2024. URL <https://arxiv.org/abs/2401.10774>. arXiv:2401.10774v3, 14 Jun 2024 (v1 19 Jan 2024).
- Cai, Y., Liang, X., Wang, X., Ma, J., Liang, H., Luo, J., Zuo, X., Duan, L., Yin, Y., and Chen, X. FastMTP: Accelerating LLM Inference with Enhanced Multi-Token Prediction, 2025. URL <https://arxiv.org/abs/2509.18362>. arXiv:2509.18362v1, 16 Sep 2025.
- Chamberlain, L. Certified LM-head decode: read a quarter of the output head, losslessly. `mcgrof/knlp`, `docs/lm-head-decode.md`, first committed 23 June 2026 (commit `1dd92463092d`), 2026. URL <https://github.com/mcgrof/knlp/blob/main/docs/lm-head-decode.md>. MIT licence. Read 30 September 2026.
- Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating Large Language Model Decoding with Speculative Sampling, 2023. URL <https://arxiv.org/abs/2302.01318>. arXiv:2302.01318v1, 2 Feb 2023.
- Chen, J., Liang, Y., and Liu, Z. DFlash: Block diffusion for flash speculative decoding. In *International Conference on Machine Learning (ICML)*, 2026a. URL <https://arxiv.org/abs/2602.06036v2>. arXiv:2602.06036v2, 28 May 2026 (camera-ready). Code: <https://github.com/z-lab/dflash>.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- Chen, Z., May, A., Svirschevski, R., Huang, Y., Ryabinin, M., Jia, Z., and Chen, B. Sequoia: Scalable, Robust, and Hardware-aware Speculative Decoding. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/ea1f5f0878d43ff4fb8bf64ef4a2326c-Abstract-Conference.html. arXiv:2402.12374v3, 5 Jul 2025 (v1 19 Feb 2024). Proceedings title: “Sequoia: Scalable and Robust Speculative Decoding”.
- Chen, Z., Xu, D., Zhang, Y., Wang, C., Xu, M., and Ma, Y. NanoSpec: Accelerating speculative decoding using minimalist in-context vocabularies, 2026b. URL <https://arxiv.org/abs/2605.26444v2>. arXiv:2605.26444v2, 1 June 2026. Version 1 (8 April 2026) was titled “MicroSpec: Accelerating Speculative Decoding with Lightweight In-Context Vocabularies”.
- Cheng, X., Yu, X., Shao, C., Li, J., Xiong, Y., Qian, Y., Zhu, J., Ma, S., Zhang, X., Ye, J., Chen, Q., Deng, C., Yu, J., Dai, D., Zhang, Z., Wei, Y., Tan, Y., Yang, W., Xu, R., Wu, Y., Xu, Z., Wang, X., Chen, M., Tian, R., Bi, X., Hao, Z., Chen, S., Cao, H., Zhang, W., Xu, A., Zhang, H., Zhao, D., and Liang, W. DSpark: Confidence-scheduled speculative decoding with semi-autoregressive generation, 2026. URL <https://arxiv.org/abs/2607.05147v1>. arXiv:2607.05147v1, 6 July 2026.

- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Daliri, M., Musco, C., and Suresh, A. T. Coupling without Communication and Drafter-Invariant Speculative Decoding. In *2025 IEEE International Symposium on Information Theory (ISIT)*, pp. 1–6, 2025. doi: 10.1109/ISIT63088.2025.11195663. URL <https://arxiv.org/abs/2408.07978>. arXiv:2408.07978v4, 20 Aug 2025 (v1 15 Aug 2024).
- DeepSeek-AI. DeepSeek-V3 Technical Report, 2024. URL <https://arxiv.org/abs/2412.19437>. arXiv:2412.19437v2, 18 Feb 2025 (v1 27 Dec 2024). The arXiv record lists DeepSeek-AI and about 200 named authors.
- Du, G., Khan, A. N., Zhou, R., Liu, X., Chakrabarti, D., Suya, F., and Li, X. Greedy decoding is not precision-invariant: Cross-precision output divergence in LLM inference. *Transactions on Machine Learning Research*, 2026. URL <https://arxiv.org/abs/2609.26621v1>. arXiv:2609.26621v1, 22 September 2026; accepted by TMLR per the arXiv comment.
- Dutson, M., Li, Y., and Gupta, M. Eventful transformers: Leveraging temporal redundancy in vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 16911–16923, 2023. URL https://openaccess.thecvf.com/content/ICCV2023/html/Dutson_Eventful_Transformers_Leveraging_Temporal_Redundancy_in_Vision_Transformers_ICCV_2023_paper.html. Verified on the CVF open access page.
- Fasi, M., Higham, N. J., Mikaitis, M., and Pranesh, S. Numerical behavior of NVIDIA tensor cores. *PeerJ Computer Science*, 7:e330, 2021. doi: 10.7717/peerj-cs.330.
- Frieder, O., Mele, I., Muntean, C. I., Nardini, F. M., Perego, R., and Tonello, N. Caching Historical Embeddings in Conversational Search. *ACM Transactions on the Web*, 18(4), 2024. doi: 10.1145/3578519. URL <https://arxiv.org/abs/2211.14155>. arXiv:2211.14155v1, 25 Nov 2022.
- Fu, Y., Bailis, P., Stoica, I., and Zhang, H. Break the Sequential Dependency of LLM Inference Using Lookahead Decoding. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 14060–14079. PMLR, 2024. URL <https://arxiv.org/abs/2402.02057>. arXiv:2402.02057v1, 3 Feb 2024.
- Ghantasala, S. M. TreeWY: Speculative Verification for Gated DeltaNet Hybrids, 2026. URL <https://arxiv.org/abs/2608.20961>. arXiv:2608.20961v1, 21 Aug 2026.
- Gloeckle, F., Idrissi, B. Y., Rozière, B., Lopez-Paz, D., and Synnaeve, G. Better & Faster Large Language Models via Multi-token Prediction. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 15706–15734. PMLR, 2024. URL <https://arxiv.org/abs/2404.19737>. arXiv:2404.19737v1, 30 Apr 2024.
- Goel, R., Agrawal, S., Gagrani, M., Park, J., Zao, Y., Zhang, H., Liu, T., Yang, Y., Yuan, X., Lu, J., Lott, C., and Lee, M. VOCABTRIM: Vocabulary Pruning for Efficient Speculative Decoding in LLMs, 2025. URL <https://arxiv.org/abs/2506.22694>. arXiv:2506.22694v2, 3 Jul 2025 (v1 28 Jun 2025). ICML 2025 Workshop on Efficient Systems for Foundation Models, according to the arXiv comment.
- Gu, G., Heo, B., Jun, H., Kang, Y., Lee, S., Yun, S., and Han, D. Verification-aware training for speculative decoding, 2026. URL <https://arxiv.org/abs/2608.30135v1>. arXiv:2608.30135v1, 31 Aug 2026.
- Gumbel, E. J. *Statistical Theory of Extreme Values and Some Practical Applications*. Number 33 in National Bureau of Standards Applied Mathematics Series. National Bureau of Standards, 1954. Bibliographic record checked; the text was not read.
- Hawkins, S. Head bit-plane exact argmax: read 25–33 % fewer head bytes on every draft/greedy head read, all families. GitHub issue 69, HawkBearPig/dgpp, opened 29 September 2026; implementation in `src/kernels/packq_head.cu`, 2026. URL <https://github.com/HawkBearPig/dgpp/issues/69>. Apache-2.0. Issue, comments and code read 30 September 2026 at commit `a23d8b1`. The issue is filed by GitHub user HawkBearPig; the commit author is Stephen Hawkins.
- He, H. and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Thinking Machines Lab: Connectionism, September 2025. URL <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>. Published 10 September 2025. Code: https://github.com/thinking-machines-lab/batch_invariant_ops.

- Higham, N. J. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, Philadelphia, PA, second edition, 2002. doi: 10.1137/1.9780898718027.
- Hu, L., Kou, S., Fu, Y., Rajbhandari, S., Rosing, T., He, Y., Deng, Z., and Zhang, H. Fast and accurate causal parallel decoding using Jacobi forcing. In *International Conference on Machine Learning (ICML)*, 2026. URL <https://arxiv.org/abs/2512.14681>. Verified on the arXiv abstract page (v1); ICML 2026 per OpenReview record ORy2hyPd4G.
- Huang, K., Guo, X., and Wang, M. SpecDec++: Boosting Speculative Decoding via Adaptive Candidate Lengths. In *Second Conference on Language Modeling (COLM)*, 2025. URL <https://arxiv.org/abs/2405.19715>. arXiv:2405.19715v3, 11 Jul 2025 (v1 30 May 2024). Venue from the arXiv comment.
- Inco AI. DFlash 2: Keep drafting parallel. Inco AI blog, August 2026a. URL <https://inco.ai/blog/dflash2/>. Published 18 August 2026; read 30 September 2026.
- Inco AI. Qwen3.8-27B-DFlash2 model card. Hugging Face, revision 015e795645c74b1a0eeef3b570031fb62e769bc5, 2026b. URL <https://huggingface.co/incoai/Qwen3.8-27B-DFlash2/tree/015e795645c74b1a0eeef3b570031fb62e769bc5>. Created 18 August 2026; read 30 September 2026.
- Jiang, W., Chitty-Venkata, K. T., Flynn, M., Meyerson, R., Qi, Z., Wu, T., Kurtic, E., Yu, M., and Marques, A. H-Spec: Parallel speculative decoding without a drafter-side KV cache, 2026. URL <https://arxiv.org/abs/2609.24197>. Verified on the arXiv abstract page (v1).
- Karimi, A., Gao, C., and Hassanpour, N. DBLAST: Dependent block drafting for stochastic speculative decoding, 2026. URL <https://arxiv.org/abs/2608.05448v1>. arXiv:2608.05448v1, 5 Aug 2026.
- Khattak, F. A. and Mikaitis, M. Accurate Models of NVIDIA Tensor Cores. *ACM Transactions on Architecture and Code Optimization*, 23(3):1–25, 2026. doi: 10.1145/3830409. URL <https://arxiv.org/abs/2512.07004>. arXiv:2512.07004v4, 11 Jun 2026 (v1 7 Dec 2025).
- Kim, J., Lee, J., and Kim, B.-K. Quantize the Target, Quantize the Drafter: Efficient Inference with Qwen3.5-4B, 2026a. URL <https://arxiv.org/abs/2607.04244>. arXiv:2607.04244v2, 7 Jul 2026 (v1 5 Jul 2026). Efficient Qwen Competition, ICML 2026 AdaptFM workshop, according to the arXiv comment. Checkpoints nota-ai/Qwen3.5-4B-QAD-W4A16 and nota-ai/Qwen3.5-4B-DFlash-GPTQ-W4A16 (Apache-2.0).
- Kim, S., Koo, J., Kim, H., Bang, I., Lee, S., Oh, H., Park, B., and Lee, D. BV loss: Block verification-aware loss for block diffusion speculative decoding, 2026b. URL <https://arxiv.org/abs/2609.34832v1>. arXiv:2609.34832v1, 28 Sep 2026.
- Kocabay, Ş. T., Akkuş, T. R., and Yuksel, K. A. ARCHead: Activation-Metric Residual Correction for Large Language Model Output Heads, 2026. URL <https://arxiv.org/abs/2608.02703>. arXiv:2608.02703v1, 3 Aug 2026. Submitted to ACL Rolling Review.
- Koo, J., Woo, S., Kil, J., Lee, J., Lee, S., Jung, K., and Kim, M. Carryover drafting: Recycling rejected states for speculative decoding, 2026. URL <https://arxiv.org/abs/2609.14717>. Verified on the arXiv abstract page (v1).
- Kool, W., van Hoof, H., and Welling, M. Stochastic Beams and Where to Find Them: The Gumbel-Top-k Trick for Sampling Sequences Without Replacement. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 3499–3508. PMLR, 2019. URL <https://arxiv.org/abs/1903.06059>. arXiv:1903.06059v2, 29 May 2019 (v1 14 Mar 2019).
- Köpf, A., Kilcher, Y., von Rütte, D., Anagnostidis, S., Tam, Z.-R., Stevens, K., Barhoum, A., Duc, N. M., Stanley, O., Nagyfi, R., ES, S., Suri, S., Glushkov, D., Dantuluri, A., Maguire, A., Schuhmann, C., Nguyen, H., and Mattick, A. OpenAssistant conversations – democratizing large language model alignment. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023. URL <https://arxiv.org/abs/2304.07327>.
- Kou, S., Hu, L., He, Z., Deng, Z., and Zhang, H. CLLMs: Consistency Large Language Models, 2024. URL <https://arxiv.org/abs/2403.00835>. arXiv:2403.00835v4, 13 Jun 2024 (v1 28 Feb 2024). ICML 2024 according to the arXiv comment.
- Kumar, T., Dao, T., and May, A. Speculative speculative decoding. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://arxiv.org/abs/2603.03251>. Verified on the arXiv abstract page (v3); OpenReview record aL1Wnml9Ef, ICLR 2026 poster.

- L, Y. S. B., Yerram, V., You, C., Bhojanapalli, S., Kumar, S., Jain, P., and Netrapalli, P. HiRE: High Recall Approximate Top- k Estimation for Efficient LLM Inference, 2024. URL <https://arxiv.org/abs/2402.09360>. arXiv:2402.09360v1, 14 Feb 2024.
- Layr Labs. Laguna LM-head prune (Sources/MLXFastModel/LagunaLmHeadPrune.swift). Source code, Layr-Labs/mlxfast-challenge, commit 4f4f68959cfe (28 July 2026), 2026. URL <https://github.com/Layr-Labs/mlxfast-challenge>. MIT licence. Read 30 September 2026.
- Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 19274–19286. PMLR, 2023. URL <https://proceedings.mlr.press/v202/leviathan23a.html>. arXiv:2211.17192.
- Li, H., Chan, T. N., Yiu, M. L., and Mamoulis, N. FEXIPRO: Fast and exact inner product retrieval in recommender systems. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*, pp. 835–850, 2017. doi: 10.1145/3035918.3064009.
- Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 28935–28948. PMLR, 2024a. URL <https://arxiv.org/abs/2401.15077>. arXiv:2401.15077v3, 4 Mar 2025 (v1 26 Jan 2024).
- Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 7421–7432, 2024b. doi: 10.18653/v1/2024.emnlp-main.422. URL <https://arxiv.org/abs/2406.16858>. arXiv:2406.16858v2, 30 Jun 2024 (v1 24 Jun 2024).
- Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025. URL <https://arxiv.org/abs/2503.01840>. arXiv:2503.01840v3, 23 Apr 2025 (v1 3 Mar 2025). <https://neurips.cc/virtual/2025/poster/119930>.
- Lin, L., Zeng, Z., Peng, S., Liu, S., and Ji, R. ReTrace: Rejected-trajectory conditioning for speculative decoding, 2026. URL <https://arxiv.org/abs/2608.29748>. Verified on the arXiv abstract page (v2).
- Liou, Z.-W. and Dao, T. ReplaySSM: Cache SSM inputs, not state. Dao AI Lab blog, June 2026. URL <https://dao-lab.ai/blog/2026/replayssm/>. Published 15 June 2026. Code: <https://github.com/Johnny-Liou/ReplaySSM>.
- Liu, A., Meng, J., Liu, F., and Chen, Y. CURE: Local uncertainty repair for block-parallel speculative decoding, 2026a. URL <https://arxiv.org/abs/2608.00531>. Verified on the arXiv abstract page (v1).
- Liu, D., Wang, S., Yu, Y., Wang, H., and Lengerich, B. CSV-Decode: Certifiable Sub-Vocabulary Decoding for Efficient Large Language Model Inference, 2025. URL <https://arxiv.org/abs/2511.21702>. arXiv:2511.21702v2, 26 Jul 2026 (v1 16 Nov 2025). Code: <https://github.com/FastLM/CSV-Decode> (no licence file).
- Liu, T., Shen, Y., Cen, R., Shi, J., Zhang, J., Qin, G., Liu, H., Liu, S., Yu, G., and Zhu, J. D-cut: Adaptive verification depth pruning for batched speculative decoding, 2026b. URL <https://arxiv.org/abs/2607.14647v1>. arXiv:2607.14647v1, 16 July 2026.
- Luo, X., Wang, Y., Zhu, Q., Zhang, Z., Zhang, X., Yang, Q., and Xu, D. Turning trash into treasure: Accelerating inference of large language models with token recycling. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6816–6831, 2025. doi: 10.18653/v1/2025.acl-long.338. URL <https://aclanthology.org/2025.acl-long.338/>. Verified on the ACL Anthology page and the arXiv abstract page (v3).
- Ma, Z. GDN Tree-Scan: Served Tree Verification for Recurrent-Hybrid Language Models, 2026. URL <https://arxiv.org/abs/2609.23900>. arXiv:2609.23900v1, 20 Sep 2026.
- Maddison, C. J., Tarlow, D., and Minka, T. A* sampling. In *Advances in Neural Information Processing Systems 27 (NIPS 2014)*, pp. 3086–3094, 2014. URL https://proceedings.neurips.cc/paper_files/paper/2014/hash/937debc749f041eb5700df7211ac795c-Abstract.html. arXiv:1411.0030.

- Mussmann, S. and Ermon, S. Learning and inference via maximum inner product search. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pp. 2587–2596. PMLR, 2016. URL <https://proceedings.mlr.press/v48/mussmann16.html>.
- Mussmann, S., Levy, D., and Ermon, S. Fast amortized inference and learning in log-linear models with randomly perturbed nearest neighbor search. In *Proceedings of the Thirty-Third Conference on Uncertainty in Artificial Intelligence (UAI 2017)*, 2017. URL <https://arxiv.org/abs/1707.03372>. arXiv:1707.03372.
- NVIDIA. NVIDIA Hopper tuning guide. CUDA 13.0.2 documentation archive, 2025. URL <https://docs.nvidia.com/cuda/archive/13.0.2/hopper-tuning-guide/index.html>. Read 30 September 2026.
- NVIDIA. AIPerf metrics reference. AIPerf documentation, 2026a. URL <https://docs.nvidia.com/aiperf/reference/ai-perf-metrics-reference>. Read 30 September 2026.
- NVIDIA. Floating point and IEEE 754. CUDA 13.4 documentation, 2026b. URL <https://docs.nvidia.com/cuda/floating-point/index.html>. Read 30 September 2026.
- NVIDIA. Nsight Compute profiling guide. Nsight Compute documentation, 2026c. URL <https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html>. Read 30 September 2026.
- NVIDIA. Parallel thread execution ISA, version 9.4. CUDA documentation, 2026d. URL <https://docs.nvidia.com/cuda/parallel-thread-execution/index.html>. Sec. 9.7.17 (warpgroup MMA, requires sm_90a) and Sec. 9.7.18 (tcgen05); read 30 September 2026.
- Pan, Y., Xi, H., Zhu, K., Li, X., Wu, Y., Mishra, M., Zhang, H., Zheng, W. X., Kasikci, B., Han, S., Keutzer, K., Iyer, R., and Stoica, I. LeapQuant: Efficient Linear Attention with Accurate Recurrent State Quantization, 2026. URL <https://arxiv.org/abs/2609.38166>. arXiv:2609.38166v1, 29 Sep 2026.
- Papandreou, G. and Yuille, A. L. Perturb-and-MAP random fields: Using discrete optimization to learn and sample from energy models. In *2011 International Conference on Computer Vision (ICCV)*, pp. 193–200, 2011. doi: 10.1109/ICCV.2011.6126242.
- Parger, M., Tang, C., Twigg, C. D., Keskin, C., Wang, R., and Steinberger, M. DeltaCNN: End-to-end CNN inference of sparse frame differences in videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12497–12506, 2022. URL https://openaccess.thecvf.com/content/CVPR2022/html/Parger_DeltaCNN_End-to-End_CNN_Inference_of_Sparse_Frame_Differences_in_Videos_CVPR_2022_paper.html. Verified on the CVF open access page.
- Ponnusamy, P., Sahni, S., Wang, J., and Dao, T. SonicSampler: Unified tile-aware kernels for LLM sampling and speculative verification, 2026. URL <https://arxiv.org/abs/2607.20475v1>. arXiv:2607.20475v1, submitted 24 May 2026.
- PyTorch contributors. Numerical accuracy. PyTorch 2.14 documentation, 2026. URL https://docs.pytorch.org/docs/2.14/notes/numerical_accuracy.html. Read 30 September 2026.
- Qwen. Qwen3.5-4B model card. Hugging Face, revision 851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a, 2026a. URL <https://huggingface.co/Qwen/Qwen3.5-4B/tree/851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a>. Read 30 September 2026.
- Qwen. Qwen3.5-4B model configuration (config.json). Hugging Face, revision 851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a, 2026b. URL <https://huggingface.co/Qwen/Qwen3.5-4B/blob/851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a/config.json>. Read 30 September 2026.
- qwen38-challenge_senpai contributors. E142: The verify readout streams 715 MB per round to answer a two-element question. GitHub pull request 142, morganmcg1/qwen38-challenge_senpai, opened 22 August 2026, 2026. URL https://github.com/morganmcg1/qwen38-challenge_senpai/pull/142. Final report comment of 23 August 2026 records the screens as refuted. Read 30 September 2026.
- Ram, P. and Gray, A. G. Maximum Inner-Product Search Using Cone Trees. In *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 931–939, 2012. doi: 10.1145/2339530.2339677. URL <https://arxiv.org/abs/1202.6101>. arXiv:1202.6101v1, 28 Feb 2012. arXiv title: “Maximum Inner-Product Search using Tree Data-structures”.
- Rowan, J., Phan, B., and Khisti, A. List-Level Distribution Coupling with Applications to Speculative Decoding and Lossy Compression. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025. URL <https://arxiv.org/abs/2506.05632>. arXiv:2506.05632v3, 11 Jan 2026 (v1 5 Jun 2025).

- Ruiz, T., Qin, Z., Zhang, Y., Shen, X., Zhong, Y., and Wang, M. FlashSampling: Fast and memory-efficient exact sampling, 2026. URL <https://arxiv.org/abs/2603.15854v3>. arXiv:2603.15854v3, 25 September 2026 (v1 16 March 2026, v2 12 May 2026). Code: <https://github.com/FlashSampling/FlashSampling>.
- Rump, S. M. Verification methods: Rigorous results using floating-point arithmetic. *Acta Numerica*, 19:287–449, 2010. doi: 10.1017/S096249291000005X.
- Rusanovsky, M., Miron, Y., Uziel, R., Belhasin, O., Guo, H., Zilberstein, R., Ashkenazi, M., and Elad, M. LiLiCorr: Lightweight likelihood correlation of parallel drafts for speculative decoding, 2026. URL <https://arxiv.org/abs/2608.20530v2>. arXiv:2608.20530v2, 22 September 2026.
- Santilli, A., Severino, S., Postolache, E., Maiorca, V., Mancusi, M., Marin, R., and Rodolà, E. Accelerating transformer inference for translation via parallel decoding. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. URL <https://arxiv.org/abs/2305.10427>.
- SGLang contributors. GDN ReplaySSM speculative fold (python/sclang/kernels/ops/attention/fla/gdn_replayssm_spec_fold.py). Source code, commit bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824, 2026a. URL https://github.com/sgl-project/sclang/blob/bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824/python/sclang/kernels/ops/attention/fla/gdn_replayssm_spec_fold.py. Lines 1–95 read 30 September 2026.
- SGLang contributors. [feature] dynamic verification for DFlash2. SGLang pull request 36136, opened 24 August 2026 by GitHub user EthanWang555, 2026b. URL <https://github.com/sgl-project/sclang/pull/36136>. Open and unmerged on 30 September 2026; head c0235e70214f1de5044be25f795faf5b755e450.
- SGLang contributors. DFlash model and candidate-head implementation (python/sclang/srt/models/dflash.py). Source code, commit bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824, 2026c. URL <https://github.com/sgl-project/sclang/blob/bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824/python/sclang/srt/models/dflash.py>. Read 30 September 2026.
- SGLang contributors. Sampler (python/sclang/srt/layers/sampler.py). Source code, commit bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824, 2026d. URL <https://github.com/sgl-project/sclang/blob/bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824/python/sclang/srt/layers/sampler.py#L746-L800>. Lines 594–611, 746–800 and 878–902 read 1 October 2026.
- Shim, K., Lee, M., Choi, I., Boo, Y., and Sung, W. SVD-Softmax: Fast softmax approximation on large vocabulary neural networks. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/4e2a6330465c8ffcaa696a5a16639176-Abstract.html.
- So, J., Kook, H., Jang, C., and Park, E. Speculative coupled decoding for training-free lossless acceleration of autoregressive visual generation. In *International Conference on Machine Learning (ICML)*, 2026. URL <https://arxiv.org/abs/2510.24211>. Verified on the arXiv abstract page; v1 was titled MC-SJD: Maximal Coupling Speculative Jacobi Decoding; ICML 2026 per the arXiv comment and OpenReview.
- Song, Y., Meng, C., Liao, R., and Ermon, S. Accelerating feedforward computation via parallel nonlinear equation solving. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021. URL <https://arxiv.org/abs/2002.03629>.
- sparkpipe contributors. Certified screened LM head (SparkLmHeadCertifiedFp8QuantizeKernel, SparkLmHeadCertifiedFp8ScoreKernel). Source code, model-families/common/include/sparkpipe/spark_lm_kernels.cuh, commit 84efd5b63d8d (16 August 2026); pull request 744 (merged 29 August 2026), 2026. URL <https://github.com/sparkpipe/sparkpipe/pull/744>. No licence file. Read 30 September 2026.
- Sun, Z., Suresh, A. T., Ro, J. H., Beirami, A., Jain, H., and Yu, F. SpecTr: Fast Speculative Decoding via Optimal Transport. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/6034a661584af6c28fd97a6f23e56c0a-Abstract-Conference.html. arXiv:2310.15141v2, 18 Jan 2024 (v1 23 Oct 2023).
- Sun, Z., Mendlovic, U., Leviathan, Y., Aharoni, A., Ro, J. H., Beirami, A., and Suresh, A. T. Block Verification Accelerates Speculative Decoding. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2403.10444>. arXiv:2403.10444v3, 10 Apr 2025 (v1 15 Mar 2024).
- Tao, C., Liu, Q., Dou, L., Muennighoff, N., Wan, Z., Luo, P., Lin, M., and Wong, N. Scaling Laws with Vocabulary: Larger Models Deserve Larger Vocabularies. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/cf5a019ae9c11b4be88213ce3f85d85c-Abstract-Conference.html. arXiv:2407.13623v3, 1 Nov 2024 (v1 18 Jul 2024).

- Teflioudi, C., Gemulla, R., and Mykytiuk, O. LEMP: Fast retrieval of large entries in a matrix product. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, pp. 107–122, 2015. doi: 10.1145/2723372.2747647.
- Teng, Y., Shi, H., Liu, X., Ning, X., Dai, G., Wang, Y., Li, Z., and Liu, X. Accelerating auto-regressive text-to-image generation with training-free speculative Jacobi decoding. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2410.01699>. Verified on the arXiv abstract page; ICLR 2025 poster per OpenReview record LZfjxvqw0N.
- The SGLang Team. Towards deterministic inference in SGLang and reproducible RL training. LMSYS Org blog, September 2025. URL <https://www.lmsys.org/blog/2025-09-22-sglang-deterministic/>. Published 22 September 2025, updated 24 September 2025.
- Triton contributors. triton.autotune. Triton documentation, 2026. URL <https://triton-lang.org/main/python-api/generated/triton.autotune.html>. Read 30 September 2026.
- Urbán, O., Kwon, Y. D., Venieris, S. I., and Mascolo, C. Margins, Not Windows: Training-Free Per-Step Lossy Speculative Decoding, 2026. URL <https://arxiv.org/abs/2609.02897>. arXiv:2609.02897v2, 28 Sep 2026 (v1 3 Jul 2026).
- Vo, Q., Nguyen, C.-D., Srey, P., Tuan, L. A., and Nguyen, T. TIGER: Text-conditioned visual gated routing with acceptance alignment for multimodal speculative decoding, 2026. URL <https://arxiv.org/abs/2607.11131v1>. arXiv:2607.11131v1, 13 Jul 2026.
- Wang, J., Wonka, P., and Ye, J. Lasso screening rules via dual polytope projection. *Journal of Machine Learning Research*, 16:1063–1101, 2015. URL <https://jmlr.org/papers/v16/wang15a.html>.
- Wang, J., Paliotta, D., May, A., Rush, A. M., and Dao, T. The Mamba in the Llama: Distilling and Accelerating Hybrid Models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/723933067ad315269b620bc0d2c05cba-Abstract-Conference.html. arXiv:2408.15237v4, 27 Jun 2025 (v1 27 Aug 2024).
- Wang, J., Su, Y., Li, J., Xia, Q., Ye, Z., Duan, X., Wang, Z., and Zhang, M. OPT-Tree: Speculative decoding with adaptive draft tree structure. *Transactions of the Association for Computational Linguistics*, 13:188–199, 2025. doi: 10.1162/tac1_a_00735. URL <https://aclanthology.org/2025.tac1-1.8/>. arXiv:2406.17276.
- Wang, L., Su, Y., Wu, X., You, C., Liu, Y., Qiu, Z., Zhang, J., Zheng, J., Liu, F., Zhang, J., Tian, C., and Huan, C. Bole: Efficient Tree Speculation for Hybrid-Attention Language Models, 2026a. URL <https://arxiv.org/abs/2608.01651>. arXiv:2608.01651v1, 3 Aug 2026.
- Wang, Z., Han, X., Yang, Z., Liu, F., Li, X., Gu, R., Zhong, S., and Tian, C. SpecLA: Efficient Speculative Decoding for Linear-Attention Models, 2026b. URL <https://arxiv.org/abs/2607.16673>. arXiv:2607.16673v1, 18 Jul 2026.
- Weber, R., Schek, H.-J., and Blott, S. A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces. In *Proceedings of the 24th International Conference on Very Large Data Bases (VLDB 1998)*, 1998. URL <https://www.vldb.org/conf/1998/p194.pdf>.
- Williams, M., Kwon, Y. D., Li, R., Kouris, A., and Venieris, S. I. Speculative decoding with a speculative vocabulary. In *Findings of the Association for Computational Linguistics: ACL 2026*, 2026. URL <https://arxiv.org/abs/2602.13836v2>. arXiv:2602.13836v2, 17 July 2026.
- Wu, T., Yao, Y., Qi, Z., Zheng, H., Wang, Z., Ma, H., Liao, L., Lakkaraju, H., Li, J., and Du, Y. D-PACE: Dynamic position-aware cross-entropy for parallel speculative drafting, 2026. URL <https://arxiv.org/abs/2605.18810v1>. arXiv:2605.18810v1, 12 May 2026.
- Wu, Y., Qin, Z., Wong, A., and Soatto, S. STree: Speculative Tree Decoding for Hybrid State-Space Models. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025. URL <https://arxiv.org/abs/2505.14969>. arXiv:2505.14969v2, 27 Oct 2025 (v1 20 May 2025). <https://neurips.cc/virtual/2025/poster/117297>.
- Xia, T., Ganesan, M., Feng, Y., Wang, H., Egger, M., and Zhang, S. Q. Acceptance-aware draft model training for speculative decoding, 2026. URL <https://arxiv.org/abs/2609.24150v1>. arXiv:2609.24150v1, 21 Sep 2026.
- Xing, S., Ye, Z., Hou, B., Ceze, L., and Chen, T. Sorting-free GPU kernels for LLM sampling. FlashInfer blog, March 2025. URL <https://flashinfer.ai/2025/03/10/sampling.html>. Published 10 March 2025.

- Yang, S., Wang, B., Zhang, Y., Shen, Y., and Kim, Y. Parallelizing Linear Transformers with the Delta Rule over Sequence Length. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/d13a3eae72366e61dfdc7eea82eeb685-Abstract-Conference.html. arXiv:2406.06484v6, 15 Jan 2025 (v1 10 Jun 2024).
- Yang, S., Kautz, J., and Hatamizadeh, A. Gated delta networks: Improving Mamba2 with delta rule. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=r8H7xhYPWz>. arXiv:2412.06464v3.
- Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., and Jiang, J. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, pp. 94–109, 2025. doi: 10.1145/3689031.3696098. URL <https://doi.org/10.1145/3689031.3696098>. Verified via the Crossref record and the arXiv abstract page (v3).
- Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., Chen, T., Kasikci, B., Grover, V., Krishnamurthy, A., and Ceze, L. FlashInfer: Efficient and customizable attention engine for LLM inference serving. In *Proceedings of the Eighth Conference on Machine Learning and Systems (MLSys 2025)*, 2025. URL <https://arxiv.org/abs/2501.01005>. arXiv:2501.01005v2.
- Yuan, J., Li, H., Ding, X., Xie, W., Li, Y.-J., Zhao, W., Wan, K., Shi, J., Hu, X., and Liu, Z. Understanding and Mitigating Numerical Sources of Nondeterminism in LLM Inference. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025. URL <https://arxiv.org/abs/2506.09501>. arXiv:2506.09501v2, 24 Oct 2025 (v1 11 Jun 2025). Version 1 was titled “Give Me FP32 or Give Me Death? Challenges and Solutions for Reproducible Reasoning”.
- z-lab. Qwen3.5-4B-DFlash model card. Hugging Face, revision 9a1996ccf887b79ab3af4fcbf8c1d1f4b5658bcf, 2026. URL <https://huggingface.co/z-lab/Qwen3.5-4B-DFlash/tree/9a1996ccf887b79ab3af4fcbf8c1d1f4b5658bcf>. Apache-2.0; created 5 March 2026, updated 19 June 2026; mirrored at [modal-labs/Qwen3.5-4B-DFlash](https://huggingface.co/z-lab/Qwen3.5-4B-DFlash). DFlash (v1) drafter for Qwen/Qwen3.5-4B with SGLang launch instructions and B200 benchmarks against MTP. Read 30 September 2026.
- Zhang, C. Exact Adaptive Hybrid Retrieval Without Fixed Top-L Cutoffs, 2026. URL <https://arxiv.org/abs/2608.07152>. arXiv:2608.07152v1, 7 Aug 2026.
- Zhang, J., Ullah, N., Schultheis, E., and Babbar, R. DynaSpec: Context-aware dynamic speculative sampling for large-vocabulary language models, 2025. URL <https://arxiv.org/abs/2510.13847v3>. arXiv:2510.13847v3, 3 February 2026 (v1 11 October 2025).
- Zhang, S., Qiu, H., He, H., and Dai, Y. Cost-aware diffusion draft trees for speculative decoding, 2026a. URL <https://arxiv.org/abs/2606.01813v1>. arXiv:2606.01813v1, 1 Jun 2026.
- Zhang, T., Tan, J., Sun, P., Yu, Y., Jiang, Z., Xie, Y., Cai, X., and Zeng, Z. DAMP: Decay-Aware Mixed-Precision Recurrent-State Quantization, 2026b. URL <https://arxiv.org/abs/2608.27513>. arXiv:2608.27513v1, 27 Aug 2026.
- Zhang, Y., Zhang, L., Cui, B., and Miao, X. DFlow: Enabling Verifier Information Flow in Block Diffusion Speculative Decoding, 2026c. URL <https://arxiv.org/abs/2609.06498>. arXiv:2609.06498v1, 6 Sep 2026.
- Zhao, W., Pan, T., Han, X., Zhang, Y., Sun, A., Huang, Y., Zhang, K., Zhao, W., Li, Y., Zhou, J., Zhou, H., Wang, J., Liu, Z., and Sun, M. FR-Spec: Accelerating Large-Vocabulary Language Models via Frequency-Ranked Speculative Sampling. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3909–3921, 2025. doi: 10.18653/v1/2025.acl-long.198. URL <https://arxiv.org/abs/2502.14856>. arXiv:2502.14856v2, 11 Mar 2025 (v1 20 Feb 2025).
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023. URL <https://arxiv.org/abs/2306.05685>.
- Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pp. 62557–62583, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html. arXiv:2312.07104.

Zhou, E., Sheng, K., Chen, H., and He, X. CARD: A cache-assisted parallel speculative decoding framework via query-and-correct paradigm for accelerating LLM inference, 2025. URL

<https://arxiv.org/abs/2508.04462>. Verified on the arXiv abstract page (v2; v1 had a different title).

Zou, X., Li, J., Huang, J., and Zhou, P. Variational speculative decoding: Rethinking draft training from token likelihood to sequence acceptance, 2026. URL

<https://arxiv.org/abs/2602.05774v6>.
arXiv:2602.05774v6, 12 Aug 2026 (v1 5 Feb 2026).

A PROOFS FOR THE CONTRACT AND THE CERTIFIED HEAD

This appendix proves the results of Sections 2.2 and 4 whose proofs are not in the main text, and states and proves the secondary results they rely on: the constants of the accumulation models, the integer pass, the computed norms and outward rounding that make an evaluated envelope an enclosure, the bounds on how many rows a decision re-scores, and the directional margin. Appendix A.9 applies the directional margin to a certified decoder tail, which its kill test rejected. Appendix A.10 separates what is machine-checked from what is tested and what is proved only on paper.

A.1 The BF16 separation lemma

Proof of Lemma 2.2. Rounding is nondecreasing, so if $\text{rn}_B(y_a) \leq \text{rn}_B(y_b)$ while $y_a > y_b$, both values round to the same v and lie in its preimage. When $|v|$ is not a power of two, the preimage has length $\text{ulp}_B(v)$ and both values lie in the binade of v . When $|v| = 2^e$ the preimage is $[v - \text{ulp}_B(v)/4, v + \text{ulp}_B(v)/2]$ for $v > 0$ (symmetrically for $v < 0$): two values on one side of v are within half the spacing of their binade, and two values on both sides are within $\frac{3}{4} \text{ulp}_B(v) \leq \text{ulp}_B(y_a)$. At $v = 0$ the preimage has the subnormal spacing. In every case $y_a - y_b \leq \text{ulp}_B(\max(|y_a|, |y_b|))$, contradicting the hypothesis. $\square$

A.2 Accumulation trees and the constants of the error models

Proof of Lemma 4.1. For a node v and a leaf ℓ below it, let $P_{v\ell} = \prod (1 + u_x)$ over the internal nodes x on the path from ℓ up to v , and for an internal node w below v let P'_{vw} be the same product over the nodes strictly above w up to v , with $P'_{vv} = 1$. By induction on height,

$$|\hat{s}_v| \leq \sum_{\ell} |p_{\ell}| P_{v\ell} + \eta \sum_w P'_{vw}, \quad |\hat{s}_v - s_v| \leq \sum_{\ell} |p_{\ell}| (P_{v\ell} - 1) + \eta \sum_w P'_{vw}.$$

Leaves satisfy both with equality. For an internal node, $|\hat{s}_v| \leq (1 + u_v) \sum_c |\hat{s}_c| + \eta$ and $|\hat{s}_v - s_v| \leq \sum_c |\hat{s}_c - s_c| + u_v \sum_c |\hat{s}_c| + \eta$; substituting the hypotheses for the children multiplies every P and P' by $(1 + u_v)$ and adds $P'_{vv} = 1$. Finally $P_{v\ell} \leq (1 + u)^d$, $P'_{vw} \leq (1 + u)^{d-1}$, and there are N internal nodes. $\square$

Depths and node units An IEEE rounded addition has $u_v = u$, since $|a + b| \leq |a| + |b|$ (Higham, 2002). Sequential accumulation from zero has depth $d = n$, pairwise summation $\lceil \log_2 n \rceil$, blocks of B summed as a tree and then accumulated $\lceil n/B \rceil + \lceil \log_2 B \rceil$, and split- K adds the number of partials. A multi-term adder that aligns $k + 1$ addends to the largest, truncates each to F fractional bits below its leading bit, adds exactly and rounds once has $u_v = (k + 1)2^{-F} + 2^{1-p}$ with truncation, and the accumulation path has $\lceil n/k \rceil$ such nodes. Tensor cores behave this way rather than as IEEE adders (Fasi et al., 2021). A node that multiplies its only child by a constant and rounds adds one level of depth.

The constants used in this paper Each model below except the IEEE one is an assumption about hardware that no vendor documents; Table 2 lists where each is used. The values are for $D = 2560$.

- *The conservative model*, $\gamma = 6.11 \times 10^{-4}$. At most $2D$ roundings, each of relative size 2^{-23} of the magnitudes it combines, in any order: $\gamma_{2D}(2^{-23}) = 6.107 \times 10^{-4}$. It covers any reduction tree, split- K with FP32 partials, and multi-term adders that keep FP32's 24 fractional bits and round to nearest or truncate. The committed replay analyses use 6.11×10^{-4} , the largest value over the exact reference's fused-adder models with blocks of 2 to 64 products, rounding to nearest or toward zero, plus one scale multiply [E2, E6]; the two agree to the precision the paper quotes.
- *The Hopper model*, $\gamma = 1.193 \times 10^{-4}$. Khattak & Mikaitis (2026) model Hopper's BF16 wgmma with FP32 output as a $k = 16$ adder with $F = 25$ and truncation, so $u_v = 17 \cdot 2^{-25} + 2^{-23} = 6.26 \times 10^{-7}$ over $D/16 = 160$ nodes, and $(1 + u_v)^{160} - 1 \approx 1.001 \times 10^{-4}$. An allowance of up to 160 further truncating FP32 additions for split- K partials gives $(1 + u_v)^{160}(1 + 2^{-23})^{160} - 1 = 1.1922 \times 10^{-4}$, which the paper quotes rounded up as 1.193×10^{-4} . The replays of the head and of the stock divergences compute this value in exact rational arithmetic and round it up to the next double [E6, E23].
- *FP8 tensor cores*. The same published model gives FP8 wgmma only 13 fractional bits in 32-term blocks, consistent with the accumulation limits reported for FP8 training (DeepSeek-AI, 2024); an FP8 tensor-core pass needs periodic promotion to FP32 and its own analysis, and none is used here.
- *IEEE FP32 on CUDA cores*. A balanced tree over blocks of 128 products, a sequential sum across the 20 blocks and one scale multiply give depth 28 with $u = 2^{-24}$, so $\gamma = 1.67 \times 10^{-6}$. This model does not describe the stock tensor-core GEMM; the paper uses it only as a sensitivity row.

Since the Hopper model is a published measurement-based model and not a documented contract, kernels built on it were validated against exact arithmetic on adversarial inputs [E17], and the stock kernel’s own error at the served shapes is [pending D2].

A.3 The envelope

Proof of Theorem 4.2. $z_i - \tilde{z}_i = \langle e_i, h \rangle + (\langle \tilde{w}_i, h \rangle - \tilde{z}_i)$. The first term is bounded blockwise by Cauchy–Schwarz or Hölder, which gives each of the four forms of Q_i . The second is the rounding error of evaluating $\langle \tilde{w}_i, h \rangle$ by a tree whose leaves are the exact products $s_{ig}q_{ij}h_j$ and $w_{ij}h_j$; the group-scale multiplications are nodes with one child, which Lemma 4.1 admits after multiplying the leaves below them by s_{ig} , so the underflow term carries the factor $\max(1, \max_g s_{ig})$. Lemma 4.1 and $\sum_j |\tilde{w}_{ij}h_j| \leq \|\tilde{w}_i\|_2 \|h\|_2$ complete the bound. $\square$

The componentwise midpoint–radius form $|z - \tilde{z}| \leq |e|^\top |h|$ is tighter than any of the four relaxations and costs the same to evaluate when $|e|$ is stored per group (Rump, 2010); the norm forms are used because they need one constant per row or block.

A.4 The integer pass

Quantizing the activation as well as the weights removes the floating-point accumulation from the pass altogether, because int8 products accumulated in int32 are exact for this shape (Abdelfattah et al., 2025). The price is an activation-quantization term. In retrieval, EAHR bounds int8 inner products by $\|e_q\| \|v\| + \|\hat{q}\| \|e_v\|$ with a floating-point guard and rechecks ambiguous items in FP32 (Zhang, 2026); the theorem below is the corresponding statement for a head with exact int32 accumulation.

Theorem A.1 (Integer pass). *Let $s_h > 0$ be any activation scale in the output format, q_h integer codes, and $e_h = h - s_h q_h$ the exact residual. Let $I_i = \sum_j q_{ij} q_{h,j}$ be accumulated exactly in 32-bit integers ($D q_{\max}^w q_{\max}^h < 2^{31}$; $2560 \cdot 127^2$ qualifies) and $\tilde{z}_i = \text{fl}(\text{fl}(s_i \text{fl}(I_i)) s_h)$ without overflow or underflow. Then*

$$|z_i - \tilde{z}_i| \leq Q_i + \|s_i q_i\|_2 \|e_h\|_2 + \gamma_k \|s_i q_i\|_2 \|s_h q_h\|_2,$$

with $k = 3$ in general and $k = 2$ when s_h is a power of two.

Proof. $z_i = s_i s_h I_i + s_i \langle q_i, e_h \rangle + \langle e_i, h \rangle$ holds in exact arithmetic for any s_h and any codes, because e_h is defined as the residual. The computed value applies at most three roundings to the product chain (the conversion of I_i and the two multiplications), so $|\tilde{z}_i - s_i s_h I_i| \leq \gamma_3 s_i s_h |I_i|$; when s_h is a power of two the last multiplication is exact and γ_2 suffices. Apply Cauchy–Schwarz to $I_i = \langle q_i, q_h \rangle$ and to $\langle q_i, e_h \rangle$, and bound $\langle e_i, h \rangle$ by Q_i as in Theorem 4.2. $\square$

The bound holds however the scale and codes of h were chosen, so an implementation must bound $\|e_h\|_2$ from above for the codes it actually produced; $\sqrt{D} s_h/2$ is valid only if every code is the nearest integer to h_j/s_h and none is clipped.

A.5 Computed norms and outward rounding

The envelope of Theorem 4.2 is a real number, and the kernel evaluates it in floating point. The two lemmas below make the evaluated interval an enclosure: the first computes norms of h that are never below the exact norms, and the second rounds the endpoints outward so that every comparison between computed endpoints is sound. Offline constants ($\|e_i\|_2, \|\tilde{w}_i\|_2, \|w_i\|_2$) are computed exactly and stored rounded up.

Lemma A.2 (Computed norms). *In FP32 with round to nearest and $D \leq 2^{14}$, let $\hat{\sigma}$ be any computed sum of the rounded squares $\text{fl}(h_j^2)$ with depth $d \leq 2^{14}$, $\hat{r} \geq (1 - 2^{-16})\sqrt{\hat{\sigma}}$ any computed square root, and $N_2 = \text{fl}(\text{fl}(\hat{r}(1 + 2^{-10})) + 2^{-50})$. Then $N_2 \geq \|h\|_2$. Likewise, if $\hat{\sigma}_1$ is any computed sum of the $|h_j|$ with depth $d \leq 2^{13}$, then $N_1 = \text{fl}(\text{fl}(\hat{\sigma}_1(1 + 2^{-10})) + 2^{-50}) \geq \|h\|_1$.*

Proof. A rounded nonnegative value, and a rounded addition of nonnegative values, returns at least $(1 - u)$ times its exact value less μ (which also covers flush-to-zero). For N_2 the squares add one rounding to the d levels of the sum, so $\|h\|_2^2 \leq (1 - u)^{-(d+1)} \hat{\sigma} + 3D\mu$ and $\|h\|_2 \leq (1 - u)^{-(d+1)/2} (1 - 2^{-16})^{-1} \hat{r} + 2^{-54}$ for $D \leq 2^{14}$. The computed $N_2 \geq (1 - u)^2 (1 + 2^{-10}) \hat{r} + 2^{-51}$, and $(1 - u)^2 (1 + 2^{-10}) \geq (1 - u)^{-(d+1)/2} (1 - 2^{-16})^{-1}$ because $(d + 1)u/2 \leq 2^{-11} + 2^{-25}$. For N_1 the terms $|h_j|$ are exact, so $\|h\|_1 \leq (1 - u)^{-d} (\hat{\sigma}_1 + D\mu)$, and $(1 - u)^2 (1 + 2^{-10}) \geq (1 - u)^{-d}$ holds for $d \leq 2^{13}$ (then $du \leq 2^{-11}$) but fails at $d = 2^{14}$, where the margin is about -6×10^{-7} ; there N_1 must be inflated by $1 + 2^{-9}$ instead. $\square$

Lemma A.3 (Outward rounding of the computed interval). *In a format with $p \geq 12$ and round to nearest, let $\tilde{z}$ be representable, let β^* be a real bound with $|z - \tilde{z}| \leq \beta^*$, and let b be computed from representable upper bounds by at most $k \leq 59$ rounded operations $(+, \times, \sqrt{\cdot})$ on nonnegative values along any path of the expression, so that $b \geq (1 - u)^k \beta^* - k\mu$. Set*

$$\hat{b} = \text{fl}(\text{fl}(\text{fl}(b(1 + 2^{6-p})) + \text{fl}(2^{4-p}|\tilde{z}|)) + 2^{e_{\min}+26}), \quad lo = \text{fl}(\tilde{z} - \hat{b}), \quad hi = \text{fl}(\tilde{z} + \hat{b}).$$

Then $lo \leq z \leq hi$.

Proof. Each rounding of a nonnegative value loses at most a factor $(1 - u)$ and μ , so $\hat{b}(1 - u) \geq (1 - u)^4(1 + 64u)b + 16u(1 - u)^3|\tilde{z}| + (1 - u)^2 2^{e_{\min}+26} - 4\mu$. Also $lo \leq \tilde{z} - \hat{b}(1 - u) + u|\tilde{z}| + \mu$. It suffices that $\hat{b}(1 - u) \geq \beta^* + u|\tilde{z}| + \mu$: the $|\tilde{z}|$ terms satisfy $16u(1 - u)^3 \geq u$, the b terms satisfy $(1 - u)^{4+k}(1 + 64u) \geq (1 - 63u)(1 + 64u) \geq 1$ for $u \leq 2^{-12}$, and $2^{26}\mu$ absorbs the absolute terms. The upper endpoint is symmetric. $\square$

For FP32 the constants are $1 + 2^{-18}$, $2^{-20}|\tilde{z}|$ and 2^{-100} . The exact reference contains witnesses that each ingredient is necessary: dropping the accumulation term of the envelope, the outward rounding or the norm inflation produces an interval that misses the exact logit (575 cases for the accumulation term, 58 of 300 checks for the norm inflation, and a constructed witness for the outward rounding) [E17].

A.6 The refinement ladder

Proof of Theorem 4.3. $[f_i(lo_i), f_i(hi_i)]$ encloses $f_i(z_i)$ because f_i is nondecreasing. A maximizer m is never dropped: for the row j that attains τ , $f_m(hi_m) \geq f_m(z_m) \geq f_j(z_j) \geq f_j(lo_j) = \tau$. A dropped row is strictly below the row attaining τ , so it is not a maximizer. If one row remains, it is the maximizer. If every kept interval is a point, each kept value is at least τ and at most τ , so every kept row is a maximizer, and every maximizer is kept. The exact level produces points. The returned index is a function of the maximizer set alone, so it does not depend on the precision or reduction order of any level. $\square$

A.7 How many rows a decision re-scores

The ladder says which rows survive; the proposition below bounds how many do, which is what decides whether re-scoring is cheap.

Proposition A.4 (Candidate counts). *Let the level-0 intervals have widths $\omega_i = hi_i - lo_i \leq \omega$ (with outward rounding, $\omega_i \approx 2\beta_i$) and let C be the rows kept at level 0.*

1. *Greedy:* $C \subseteq \{i : z_i \geq z_{\max} - \omega_i - \omega_a\}$ for any maximizer a , hence $|C| \leq e^{2\omega/T'} / p_{\max}^{(T')}$ for every $T' > 0$, where $p^{(T')} = \text{softmax}(z/T')$.
2. *Race:* if the g_i are i.i.d. standard Gumbel and $p = \text{softmax}(z/T)$, then with $t = 2\omega/T$,

$$\mathbb{E}|C| \leq \sum_i \frac{p_i e^t}{1 + p_i(e^t - 1)} \leq \min(V, e^t).$$

Proof. (1) $hi_i \leq z_i + \omega_i$ and $lo_a \geq z_a - \omega_a$, so a kept row satisfies $z_i + \omega_i \geq \tau \geq z_a - \omega_a$. Each such row has $p_i^{(T')} \geq p_{\max}^{(T')} e^{-2\omega/T'}$, and the probabilities sum to at most one. (2) In score space the widths are ω_i/T , so $C \subseteq \{i : y_i \geq M - t\}$ with $y_i = z_i/T + g_i$ and $M = \max_j y_j$. Row i belongs to that set if and only if $y_i + t \geq \max_{j \neq i} y_j$. The maximum of independent Gumbels with locations z_j/T is Gumbel with location $\log \sum_{j \neq i} e^{z_j/T}$, and two independent Gumbels with locations a and b satisfy $P(G_a \geq G_b) = e^a/(e^a + e^b)$. Hence the probability is $p_i e^t / (p_i e^t + 1 - p_i)$; sum over i . $\square$

At $T = 1$ and envelope half-width β (so $t \approx 4\beta$), the bound on the expected re-scoring set of an exact race is 55 rows for $\beta = 1$, 2,981 for $\beta = 2$ and up to 1.6×10^5 for $\beta = 3$. Exact sampling through the cheap pass is therefore guaranteed to be cheap only if the envelope is about two logits wide or less. Greedy selection has no distribution-free bound, since a flat distribution (small $p_{\max}$) keeps many rows. The envelope in logits is $\beta_i \approx \|e_i\|_2 \|h\|_2$; with the int8 per-row constants of the head (median $\|e_i\|_2 / \|w_i\|_2 = 0.97\%$ and $\|w_i\|_2 = 0.652$) that is about $0.0063 \|h\|_2$, so the norm of the head input decides whether β is a fraction of a logit or several. On the captured plain-decode states that norm has a median of 159 (1st percentile 132, 99th 169), which puts the int8 row envelope at about one logit [E2, E10]. The measured candidate sets are far below the bound because most top-two margins are several logits.

Measured sets beyond Table 3 The same replay covers heads and states the main text summarizes in a sentence [E2, E7]. Int4 alone is useless as a certificate: with 32-column groups a decision needs 20,751 candidate rows on average (p99 230,238, maximum 248,192) and 0.02% of positions are decided by one row, and with 128-column groups the median set is 13,991 rows. A two-plane cascade, an int4 plane for every row and an int4 residual plane only for rows the first cannot exclude, reads 0.32 of the BF16 head’s bytes for one row and 0.41–0.42 over capture steps with 5 to 16 held-out rows, at the price of an irregular second gather. On the plain-decode positions, re-scoring candidates with IEEE FP32 accumulation on CUDA cores leaves no position undecided; with the tensor-core error model applied to the re-scoring as well, 2.1% of positions still overlap and need FP64. Centring the head before quantizing it is instructive: the mean row points along the bulk of the vocabulary (median cosine 0.39) but not along the rows that compete for the argmax (median cosine -0.11 over each position’s top eight), so it lowers the median error norm over all rows by 14% and raises it by 20% on the rows that matter [E5]. The int8 counts on speculative rows below, split by whether the verifier needs the row, and FP8’s count over all verify rows use the elementwise minimum of the rigorous per-row envelopes under the conservative model; it is the only envelope for which the replay records that split [E7, E8]. On DFlash-4B verify rows after the first rejection, which are conditioned on a wrong prefix and have flatter distributions, the int8 per-row copy needs 4.0 candidates on average, and on the draft head’s own rows that are reached 1.28 (1,138 of 4,020 rows; 3.36 on the unreached ones); FP8 needs 28.9 over all verify rows. No envelope was violated and no winner was missed in any of these sets. The gap condition, the only R-stock rule evaluated on the DFlash-4B verify rows, leaves at least one row to the stock kernel in 78% (Hopper model) to 84% (conservative model) of the capture’s verify steps, against 16.0% and 21.6% of plain-decode steps, so a whole-batch fallback would run in most verify passes unless the interval form does much better there, which is untested [E7, E6]. On the MTP-4B capture [E8], the int8 per-row copy behaves on the verify rows a greedy verifier needs (2,901 of 4,010) as on plain decoding, with 1.51 candidates on average (p99 7) and one row at 76.1% of them, and needs 2.44 after the first rejection. Unlike DFlash’s, the MTP draft head needs more candidates than the verifier, 2.44 on its reached rows (p99 15), because its input keeps the target’s scale and the envelope grows with $\|h\|$. The gap condition leaves 4.04% (conservative model) and 2.84% (Hopper model) of the verify rows to the stock kernel, and 6.44% and 4.20% of the draft rows, with at least one such row in 48% and 37.5% of the capture’s verify steps, against 84% and 78% for DFlash-4B. No envelope was violated and no winner was missed. For comparison, a row-wise int8 copy of the Qwen3-8B head agrees with the BF16 argmax on 97.56% of positions (Kocabay et al., 2026), so an exact re-score must catch the rest.

A.8 The directional margin

When the information is about the hidden state rather than the logits, a box of logit bounds is loose; the certificate below uses the direction of each competitor instead.

Proposition A.5 (Directional margin). *Let the logits be $z = Wh$ in real arithmetic and suppose $\|h - \tilde{h}\|_2 \leq \varepsilon_h$ for a known $\tilde{h}$. Then c is the strict argmax of Wh for every such h if and only if*

$$(w_c - w_j)^\top \tilde{h} > \|w_c - w_j\|_2 \varepsilon_h \quad \text{for all } j \neq c.$$

This condition is implied by, and can be much weaker than, the box condition obtained from the row bounds $|z_j - w_j^\top \tilde{h}| \leq \|w_j\|_2 \varepsilon_h$.

Proof. For fixed j , $\min_{\|e\|_2 \leq \varepsilon_h} (w_c - w_j)^\top (\tilde{h} + e) = (w_c - w_j)^\top \tilde{h} - \|w_c - w_j\|_2 \varepsilon_h$ by Cauchy–Schwarz, with equality at $e = -\varepsilon_h (w_c - w_j) / \|w_c - w_j\|_2$. So $z_c > z_j$ throughout the ball exactly when the stated inequality holds. The box condition reads $(w_c - w_j)^\top \tilde{h} > (\|w_c\|_2 + \|w_j\|_2) \varepsilon_h$, and $\|w_c - w_j\|_2 \leq \|w_c\|_2 + \|w_j\|_2$. $\square$

The directional certificate is not free: it needs $\|w_c - w_j\|_2$ for every competitor, which for all j costs as much as a head pass. It is useful after a box screen has reduced the competitors to a few rows, and for the stock contract the gap condition of Theorem 2.3 applies on top of it, as it does for the ladder. It is also a statement in real arithmetic; the floating-point terms of Appendix A.2 turn it into an enclosure of computed values.

A.9 Beyond the head: a certified decoder tail (P1, rejected)

Skipping a computation removes its cost only if nothing downstream needs its result, so every skipped result falls into one of three classes. It is *dead* when it has no remaining consumer. It is *deferred* when a later consumer needs it and a reconstruction path exists; the cost is moved, not removed, and must be charged where it lands. It is *soundly enclosed* when

a certificate bounds it well enough for every consumer that remains. A claimed saving should name its class, because early exits and approximations can borrow time from future tokens.

The head is dead work in this sense: once a token is committed, no future computation reads the logits. In target-only decoding of Qwen3.5-4B the same is true of more. The last decoder layer is a full-attention layer; its attention writes the key-value cache before its MLP runs, and the final MLP, the final norm and the head have no consumer in future state. A certificate that decided the token before that tail could therefore remove about 1.41 GB of the 8.41 GB streamed per step (the head and the last layer’s MLP, Table 16), permanently. The cut does not transfer to speculation without re-analysis: the MTP layer and the DFlash drafter consume target hidden states, so work that is dead in plain decoding is deferred there.

The hypothesis (P1) was that a cheap surrogate of that tail with a sound enclosure of the head input could let the commit rule decide the token before the tail runs. A one-shot surrogate that costs C (surrogate plus certificate) and certifies a fraction p of positions pays only if $C < p T_{\text{tail}}$, and the ceiling for a tail that is a fraction f of the step is $1/(1 - f)$. The directional margin is the natural certificate when the surrogate predicts the head input with a radius. The kill test was deliberately optimistic: record the residual stream at the cut, the reference head input, the logits and the tokens; freeze an int8 surrogate of the tail; take the oracle radius $\|h - \tilde{h}\|_2$; and apply the directional certificate with no overhead. If the oracle certification rate satisfied $p_{\text{oracle}} T_{\text{tail}} \leq C_{\text{surrogate}}$, the design was to be dropped; otherwise real bounds would be compared with the dense tail and with the certified head alone.

The kill test rejects the design [E9]. It used 2,064 positions from 16 held-out prompts (four per domain), with the head input taken from a BF16 forward pass of a reference implementation on the CPU, teacher-forced on the engine’s greedy outputs; that head input differs from the engine’s captured one by a median 1.8% in relative ℓ_2 norm, and its argmax equals the engine’s token at 99.5% of positions. An int8 per-row copy of the final MLP moves the head input by a median 0.94%. With that realized error as an oracle radius and no overhead, the directional certificate decides 85.4% of positions when the head is exact and 71.8% when the head is int8 as well, while the int8 head alone, with the exact MLP, decides 79.3% without any re-scoring. The MLP is 0.142 GB of the tail’s 1.413 GB, so an int8 surrogate can save at most 0.071 GB per token, and it lowers the rate at which the certified head decides on its own. Certifying the head alone is the part worth building.

A.10 What is machine-checked, what is tested, and what is proved on paper

Machine-checked Lean 4.19.0 elaborates two files with no sorry, no admit and no added axiom [E18]. Both work over integers, which stand for scores after clearing a common positive denominator. `formal/CertifiedArgmax.lean` proves that a quantization bound and an accumulation bound add and that outward widening keeps an enclosure valid (`envelope_compose`, `widen`); that disjoint enclosures decide comparisons, that a certified winner is the unique maximizer and that any two certificates from valid enclosures name the same token (`separation`, `certified_argmax`, `certificate_unique`), which is the decision logic of Theorem 4.3; that the filter $hi_i \geq \max_j lo_j$ keeps every maximizer and drops only beaten rows (`maximizer_survives`, `dropped_row_loses`); that adding a fixed noise value or scaling by a positive temperature preserves an enclosure (`shift_encloses`, `scale_encloses`); the acceptance guards of Theorem B.2 with an enclosed numerator and their exclusivity (`accept_guard`, `reject_guard`, `guards_exclusive`); the residual-weight enclosure of Theorem B.4 (`residual_interval`); the ℓ_∞/ℓ_1 Hölder bound behind the Hölder form of transport (Appendix C.1) and the Hölder form of the quantization envelope (`holder`); and the sequential case of Lemma 4.1 (`sequential_sum_bound`). `formal/DecisionGuards.lean` holds nine declarations: scaled-integer acceptance and rejection guards and their stability under refinement, associativity of chronological composition, deterministic replay of records from equal starts (the logical skeleton of Proposition D.2), and guards that stop a stale epoch or prefix from publishing state. A deliberately false variant of `sequential_sum_bound` is rejected, so the check is not vacuous.

Outside the checker are IEEE rounding and the bridge from floating-point values to these integers; the tree, pairwise, blocked and fused forms of the accumulation bound; Cauchy–Schwarz; real exponentials and the transported mass bounds; the Gumbel-max law and every probability statement; the tensor-core adder model; and compiler lowering, race freedom on the GPU and the engine’s state restoration.

Tested against exact arithmetic An exact floating-point reference (`src/precision_reference.py`) emulates every BF16, FP32 and FP64 rounding on rationals and checks each bound and decision of Sections 4 and 4.3 against real arithmetic [E17]. Its 21 test methods run 40,015 generated checks: 5,600 accumulation-bound checks over eight summation models (sequential, pairwise, blocked, split- K , round toward zero, flush to zero and the fused multi-term adder), where the largest observed error was 17–83% of the bound depending on the model; 1,767 approximate-pass rows and 2,767 integer-pass rows enclosed;

420 certified argmax cases, 72 of them exact ties, of which 183 ended at the approximate pass, 191 after FP32 re-scoring, 24 in FP64 and 22 in exact arithmetic; 360 races under a fixed noise field (62 with a forced tie at the top), 200 top- k sets, 300 acceptance decisions (148 with the uniform placed next to the threshold), 300 residual races and 300 verifications in SGLang’s sibling form; 900 emulations of the stock head’s rounding on adversarial near-tie heads with $D \leq 8$; 160 heads with subnormal weights and inputs; and the witnesses cited in the text for R-bf16 (Section 2.2), for BF16 split- K partials and for a noise stream consumed in evaluation order. The heads are synthetic with small V and D , and the fused adder is a model, not hardware.

A second exact reference, with bounded integer heads and base-2 masses represented as rationals (so that no floating-point exponential is trusted), tests the decision and race algorithms [E19]. Its 18 decision and 3 race test methods pass on the instances of Table 6. The race tests use fixed positive rational priorities, so they establish pathwise equivalence of bounded and dense algorithms, not properties of continuous Gumbel noise. The exact witnesses that separate the contracts and those for the recurrent state are in Appendices D.1 and E.5.

Check	Instances	What agrees
Top- k plus tail composition	250	Exact union and associative merge (Proposition C.1)
Transported tile bounds	300 heads, 1,048 tiles	3,787 weight rows enclosed (Theorem 5.1, Hölder form)
Greedy certified screening	250	Same winner under a deterministic total order
Retained-tail tightening	565 tiles	Exact mass enclosed; bounds never looser (Equation (7))
Acceptance guards	1,600	Certified bits agree with the complete mass (Theorem B.2)
Unresolved-interval measure	160	Exact agreement with P_γ of Theorem B.2
Lazy acceptance	500	With and without retained seeds; finite completion
Coupled one-step sampling	200	Same output with exact residual fallback
Rejection-correction identity	300 pairs	Includes zero-probability support entries
Demand frontier	1,092 partial states	5,460 complete decision assignments (Proposition B.8)
Exclusion-aware race summaries	300	Exact maxima after support exclusions
Target priority race	250	Same token as a dense priority race
Sparse-support residual race	250	Bounded and dense races agree (Theorem B.3)
Finite-map execution	400	Scan, walk and composition agree (Proposition F.2)
Gated-delta replay	150 trajectories	868 accepted-prefix replay checks (Proposition D.2)
Lease reuse	300 schedules	Sequential epoch and base-prefix guards

Table 6. Instances checked by the exact rational reference for decisions and races [E19]. It also contains counterexamples for biased depth selection, graph-cost cliffs, batch unions, invalid numerical bounds, support truncation and stale namespaces. No GPU code is exercised.

Proved on paper only Every other statement of the paper, including the floating-point lemmas above, the transport and crossover theorems, the sampling results of Appendix B and the propositions about the recurrent state, is proved in the text and checked by the tests above only where a test is named. Tests are not proofs, and neither is a GPU measurement.

B SAMPLED DECISIONS: FURTHER ANALYSIS

The engine path of Section 4.3 verifies sampled drafts with a fixed noise field, which turns every sampled decision into a race. This appendix proves the theorem behind that choice and develops what the choice avoids: the standard acceptance rule evaluated with enclosures, residual races, sampling before the scores are known, an approximate mode, adaptive verification depth, and a demand frontier. Every statement takes enclosures of the logits as input and does not depend on the mechanism that produced them, so it applies to transport as well as to self-evidence. None of these alternatives is implemented on the GPU; the last subsection is a CPU experiment that explains why.

B.1 Fixed-noise verification and the rest of the random-number contract

Proof of Theorem 4.5. By induction on positions, every accepted token equals the argmax at a prefix equal to $Y_{<t}$, so it is Y_t ; at the first mismatch the prefix is $Y_{<t}$ and the emitted token is Y_t . The drafter changes only how many positions one target pass verifies, so the output is the same for every drafter and depth policy. Given $Y_{<t}$, the field $G_{t,\cdot}$ is independent of $z_t(Y_{<t})$, so $Y_t \mid Y_{<t} \sim \text{softmax}(z_t(Y_{<t})/T)$ by the Gumbel-max identity. $\square$

The theorem treats z_t as a function of the prefix alone, which the engine does not guarantee (Section 2.5 and Appendix D.4); the contract of Section 4.3 is scoped accordingly.

Two further conditions complete the random-number contract of Section 4.3. Equality in distribution, as opposed to seeded equality, needs i.i.d. noise with the correct marginal that is independent of the proposal draw, the acceptance uniform and the scores; a residual race (Appendix B.4) also needs independence from the acceptance event. And a hash with 2^{32} levels makes the law exact only for the idealized continuous field; the stock seeded sampler shares that approximation.

Truncated sampling The Gumbel-max identity has a top- k form that samples k tokens without replacement (Kool et al., 2019). Top- k sampling applies the ladder to membership (drop i when hi_i is below the k -th largest lo) and races over the certified set. With noise keyed by token, as in the full-vocabulary race, this gives equality in distribution with top- k sampling from the reference scores and, once the engine’s tie semantics at the k -th position are matched, pathwise equality with a token-keyed dense race over the same set. It does not give seeded equality with SGLang’s sampler at the pin. That sampler’s seeded top- k and top- p path sorts the probabilities, keys each token’s Gumbel noise by the token’s position in the sorted row rather than by its index, and maps the winner back to a token through the sort’s permutation, so the noise a token receives depends on its rank (SGLang contributors, 2026d). Seeded equality with that path therefore needs, beyond membership, the complete order within the certified set: enclosures of the FP32 probabilities the sampler sorts that are strictly separated for every pair of members whose ranks could otherwise differ, and `torch.sort`’s order where probabilities are equal. The pinned sampler also refuses min- p with a seed. Its full-vocabulary seeded path keys the noise by token index, and that is the case the certified race of Section 4.3 covers. Min- p admits i when $z_i \geq z_{\max} + T \log(\min-p)$, which intervals decide without a partition function. Top- p admission depends on the mass above each token relative to the partition function Z , so it needs partition bounds and is usually cheaper to hand to the ordinary path. Fused samplers take the other route and make the dense path cheap: FlashInfer samples top- k , top- p and min- p without sorting and implements chain speculative sampling (Ye et al., 2025; Xing et al., 2025), and SonicSampler fuses the post-logit sampling and verification kernels over a bounded top-128 pool and describes top- p under that pool as theoretically lossy (Ponnusamy et al., 2026). The closest precedent for sampling from bounded scores, Musmann, Levy and Ermon’s lazy Gumbel sampling, draws noise only for a retrieved top set and, with an approximate top set, widens the cutoff by the approximation error (Musmann et al., 2017; Musmann & Ermon, 2016); the certified race differs in bounding every row with a dense cheap pass and in keying the noise by token, which gives pathwise equality with a dense race keyed the same way, not with the stock sampler’s rank-keyed truncated path.

B.2 Acceptance under fixed-noise coupling

Fixed-noise verification of a sampled draft accepts less often than the standard rule. The proposition says by how much.

Proposition B.1 (Acceptance under fixed-noise coupling). *Suppose that the drafter proposes the token $\arg \max_i (\log q_i + G_i)$ and that the target accepts it exactly when it equals $\arg \max_i (\log p_i + G_i)$. Then*

$$P(\text{accept}) = \sum_i \left(\sum_j \max\left(\frac{p_j}{p_i}, \frac{q_j}{q_i}\right) \right)^{-1},$$

$$\frac{1}{2} \sum_i \min(p_i, q_i) \leq \sum_i \frac{p_i q_i}{p_i + q_i} \leq P(\text{accept}) \leq \sum_i \min(p_i, q_i).$$

Proof. Both choose i if and only if $G_i > \max_{j \neq i} (G_j + m_j)$ with $e^{m_j} = \max(p_j/p_i, q_j/q_i)$, an event of probability $(1 + \sum_{j \neq i} e^{m_j})^{-1}$. The bounds follow from $\max(x, y) \leq x + y$, from $\max(x, y) \geq x$ and $\max(x, y) \geq y$ summed over j , and from $p + q \leq 2 \max(p, q)$. $\square$

Per position, fixed-noise acceptance of a sampled draft is therefore at most the standard rule’s rate $1 - \text{TV}(p, q)$ and at least half of it, and the two agree when $p = q$. For SGLang’s default point-mass drafts the question does not arise: the stock rule accepts a draft d with probability $p(d)$, and so does the keyed race. Whether a cheaper certificate pays for any lost acceptance with sampled drafts is an empirical question, and the acceptance under fixed-noise coupling against standard verification on real drafts is [pending D10].

B.3 The standard acceptance rule with enclosures

At the pin, SGLang’s sampled speculative verification uses point-mass drafts by default: a drafted token is accepted when a uniform coin falls below the target probability mass of the siblings tried so far, and a rejection samples the target restricted to the untried tokens by inverse CDF (code reading). In general, let p be the target law at the reached prefix and q the actual proposal law, conditional on the information used to produce the proposal. A softmax buffer from a different selector

is not q , and a deterministic proposal is a point mass. The standard one-step correction accepts $X \sim q$ with probability $\min(1, p_X/q_X)$ and otherwise samples $(p - q)_+$ normalized; the accepted mass at i is $\min(p_i, q_i)$ and the residual contributes $(p_i - q_i)_+$, which sum to p_i (Leviathan et al., 2023; Chen et al., 2023). SpecTr and block verification extend the rule to several drafts and to whole blocks (Sun et al., 2023; 2025). A certificate may change how this comparison is evaluated, never the law used in it.

For one reached proposal $X = x$ write $w_x = e^{z_x}$ and $Z = \sum_i e^{z_i}$, and draw $U \in [0, 1)$ with the reference convention. For $q_x > 0$, acceptance is equivalent to

$$Uq_xZ < w_x, \quad (3)$$

and the $\min(1, \cdot)$ need not be evaluated because $U < 1$ already handles $p_x \geq q_x$.

Theorem B.2 (Acceptance guards with enclosures). *Let $w_x \in [N^-, N^+]$ and $Z \in [Z^-, Z^+]$ with $N^- > 0$, let $0 < q_x \leq 1$ be the actual proposal mass and $U \in [0, 1)$. Then $Uq_xZ^+ < N^-$ implies acceptance, $Uq_xZ^- \geq N^+$ implies rejection, the two never both hold, and replacing an enclosure by a tighter one cannot reverse a certified decision. For U uniform and independent of the enclosures, neither guard fires with probability*

$$P_\gamma = \min(1, N^+/(q_xZ^-)) - \min(1, N^-/(q_xZ^+)).$$

If $N^- = N^+ = w_x$ and every row outside a re-scored set R has a mass enclosure whose upper and lower ends differ by a factor at most κ , then with $\pi = \sum_{i \notin R} p_i$,

$$P_\gamma \leq \frac{p_x}{q_x} \cdot \frac{(\kappa - 1)\pi}{1 - \pi(1 - 1/\kappa)}.$$

Proof. The implications are transitivity with (3); both firing would give $N^+ \leq Uq_xZ^- \leq Uq_xZ^+ < N^-$. A tighter enclosure stays on the same side of the true values. The unresolved uniforms form $[N^-/(q_xZ^+), N^+/(q_xZ^-)) \cap [0, 1)$. For the bound, $P_\gamma \leq (w_x/q_x)(Z^+ - Z^-)/(Z^+Z^-)$; $Z^+ - Z^- \leq \sum_{i \notin R} e^{z_i}(\kappa - 1)$ because each lower mass is at most e^{z_i} ; and $Z^- \geq Z - \sum_{i \notin R} e^{z_i}(1 - 1/\kappa)$ because each lower mass is at least e^{z_i}/κ . Divide by Z . $\square$

Transported tile masses (Theorem 5.1) and exponentiated self-evidence intervals (Theorem 4.2) both supply $Z^\pm$. With self-evidence $\kappa = e^{2\beta}$ times the outward rounding of the exponential; at $\beta = 1$ ($\kappa \approx 7.4$), keeping P_γ below 10% at $p_x/q_x \approx 1$ requires the unre-scored mass π to be below about 1.5%. After an adaptive refinement chosen using U the probability statement no longer applies, although the guards remain sound pointwise. The measured partition widths on real draft states are in Appendix C.8; for transport they leave P_γ near one.

The guards decide acceptance, and that is where they stop being enough. A rejection still requires a sample from $(p - q)_+$, and knowing that rejection occurred does not supply one. Under a stock-token contract the stock inverse-CDF residual would need a dense head row per verify step, and on constructed heads a cheap acceptance decision followed by a dense residual erased most of the saving (Appendix B.9). This is why the engine path uses fixed-noise verification, which needs neither a partition function nor a residual sampler, and why the guards and the residual races below are analysis, not built.

B.4 Residual and bonus races

When all drafted tokens are accepted, a bonus token needs a target sample; after a rejection the residual needs one. Two facts about the engine and one construction reduce the problem. First, SGLang’s default sampled verification uses point-mass proposals (Appendix B.3); with q a point mass at x , the residual is the target restricted to the untried tokens, a race over $V \setminus \{x\}$ that needs no partition function. Second, under fixed-noise verification there is no residual at all. Third, for a proposal with small support there is an exact bounded race.

Let $Q = \{i : q_i > 0\}$. With unnormalized target weights w_i and partition Z , the residual can be sampled from the weights

$$r_i = (w_i - Zq_i)_+, \quad \frac{r_i}{\sum_j r_j} = \frac{(p_i - q_i)_+}{\sum_j (p_j - q_j)_+}, \quad (4)$$

with $r_i = w_i$ outside Q . Given $Z^- \leq Z \leq Z^+$, positive-part monotonicity gives $(w_i - Z^+q_i)_+ \leq r_i \leq (w_i - Z^-q_i)_+$ inside Q .

Theorem B.3 (Bounded residual race). *Assume valid enclosures of the target weights and of Z , the actual proposal law, a nonzero residual, and a priority field $\pi_i = e^{G_i} > 0$ that is fixed and independent of the proposal draw, the acceptance uniform and the rejection event. A refinement procedure that returns only an interval-certified ordered maximizer of $r_i\pi_i$, or*

completes the dense race, returns the same token as the dense race. With i.i.d. standard Gumbel G_i it samples the residual law (4) exactly.

Proof. The enclosures of r_i multiplied by $\pi_i > 0$ enclose the priorities. A certified maximizer dominates every possible competitor, including the actual one, and a completed evaluation yields the dense ordered maximizer. The distributional statement is Gumbel-max applied to the nonzero residual weights. $\square$

Theorem B.4 (Residual races with self-evidence). *Replace the enclosures above by $r_i \in [(N_i^- - Z^+ q_i)_+, (N_i^+ - Z^- q_i)_+]$ for $q_i > 0$ and $r_i \in [N_i^-, N_i^+]$ for $q_i = 0$. The conclusions of Theorem B.3 hold. For a point-mass proposal at x , $r_x = 0$ as soon as $Z^- \geq N_x^+$, and the residual is a race over $V \setminus \{x\}$ that needs no partition bound. SGLang’s target-only rule (accept sibling j if $UZ < \sum_{l \leq j} w_{x_l}$; otherwise sample the target restricted to the untried tokens) is certified by the guards with a cumulative numerator and an exclusion race.*

Proof. Positive-part monotonicity gives the enclosures; the rest is the proof of Theorem B.3. For the point mass, $w_x - Zq_x = w_x - Z \leq 0$. $\square$

The engine samples that residual by inverse CDF, so a race matches it only in distribution. For a support chosen after the draft pass, retaining the top $K + 1$ perturbed entries per tile tightens the maximum outside the support after up to K support tokens are excluded; it is not needed for validity, since $r_i \leq w_i$ makes the plain tile maximum an upper bound. A bonus sample is simpler still: a certified race over the target scores with the bonus position’s field, with no partition function.

B.5 Sampling before the scores are known

A race needs the winner’s interval to separate from every other. A different construction can emit a token with some probability before any candidate is separated, and finish the rest of the law after more computation. Let $\mathcal{P}(I)$ be the set of target laws compatible with the information I held so far.

Proposition B.5 (Common mass). *Consider procedures that, before acquiring more information, emit token i with probability r_i (the same for every $p \in \mathcal{P}(I)$) and otherwise continue until p is known. Such a procedure outputs exactly p for every $p \in \mathcal{P}(I)$ if and only if $r_i \leq \inf_{p \in \mathcal{P}(I)} p_i$ for every i , and then it may finish by sampling the residual $(p - r)/(1 - \alpha)$ with $\alpha = \sum_i r_i$. The largest immediate probability is*

$$\alpha^*(I) = \sum_i \inf_{p \in \mathcal{P}(I)} p_i.$$

Proof. If the procedure is exact, the final probability of i is at least the probability r_i of emitting it immediately, for every compatible p , so $r_i \leq \inf_p p_i$. Conversely, if $r_i \leq \inf_p p_i$ then $p - r \geq 0$ and sums to $1 - \alpha$ for every compatible p , so the residual is a law, and the output law is $r + (1 - \alpha) \cdot (p - r)/(1 - \alpha) = p$. $\square$

For two candidate laws $\mathcal{P} = \{p, q\}$, $\alpha^* = \sum_i \min(p_i, q_i) = 1 - \text{TV}(p, q)$, the overlap of a maximal coupling, which is exactly the acceptance probability of speculative sampling (Leviathan et al., 2023; Chen et al., 2023). The proposition says that the same idea applies to any set of laws the evidence leaves open, and that nothing can do better without more information.

A two-token example shows that exact sampling can finish where greedy selection cannot. Suppose the evidence says only that the probability of token 1 lies in $[0.49, 0.51]$. Then $\alpha^* = 0.49 + 0.49 = 0.98$. Draw U uniform on $[0, 1]$: emit token 1 if $U < 0.49$, token 0 if $U \geq 0.51$, and otherwise compute p and emit token 1 exactly when $U < p$, reusing the same U . The output has law p , and 98% of draws never need the scores, whereas the greedy decision is undecided because the interval contains $1/2$. For two tokens this procedure is inverse-CDF sampling with bounds on the CDF. In general it gives equality in distribution, not seeded equality with a dense race.

Proposition B.6 (Bounded logit error). *Let $p = \text{softmax}(z/T)$ and $q = \text{softmax}(\tilde{z}/T)$, and suppose $\max_i (z_i - \tilde{z}_i) - \min_i (z_i - \tilde{z}_i) \leq \Delta$. Then $p_i \geq e^{-\Delta/T} q_i$ for every i . Consequently, with $\alpha = e^{-\Delta/T}$:*

1. *sampling q with probability α , and otherwise computing p and sampling $(p - \alpha q)/(1 - \alpha)$, samples p exactly;*
2. *always sampling q gives $\text{TV}(p, q) \leq 1 - \alpha$.*

Proof. Write $d_i = z_i - \tilde{z}_i$ with $m \leq d_i \leq M$ and $M - m \leq \Delta$. Then $p_i/q_i = e^{d_i/T} / \sum_j q_j e^{d_j/T} \geq e^{m/T} / e^{M/T} \geq e^{-\Delta/T}$. Part 1 is Proposition B.5 with $r = \alpha q$. For part 2, $\text{TV}(p, q) = \sum_i (q_i - p_i)_+ \leq \sum_i (1 - \alpha) q_i = 1 - \alpha$. $\square$

With self-evidence the range is at most twice the largest envelope half-width, so $\alpha \geq e^{-2 \max_i \beta_i / T}$. The trade-off against the race differs: this sampler needs the cheap pass’s full softmax and, with probability $1 - \alpha$, the whole exact head, whereas the race re-scores a candidate set whose expected size grows like $e^{4\beta/T}$ (Proposition A.4) and returns the seeded token.

B.6 An approximate mode, and why it cannot be certified harmless

Proposition B.7 (Approximate mode: KL and sequence total variation). *Suppose that $\min_c \|z - \tilde{z} - c\mathbf{1}\|_\infty \leq \varepsilon$ at temperature one; at temperature T , replace ε by ε/T . Then $\text{KL}(p \parallel q) \leq \varepsilon^2/2$ and $\text{KL}(q \parallel p) \leq \varepsilon^2/2$, so $\text{TV}(p, q) \leq \varepsilon/2$. If an approximate decoder samples q_t in place of p_t at every position and the bound holds with ε_t at every reachable prefix, the laws P and Q of the first N tokens satisfy $\text{KL}(P \parallel Q) \leq \frac{1}{2} \sum_t \varepsilon_t^2$ and*

$$\text{TV}(P, Q) \leq \frac{1}{2} \left(\sum_{t=1}^N \varepsilon_t^2 \right)^{1/2}.$$

Proof. Let $d = z - \tilde{z} - c\mathbf{1}$ with $\|d\|_\infty \leq \varepsilon$; the shift c changes neither law. Then p is q tilted by d : $p_i = q_i e^{d_i} / \mathbb{E}_q[e^d]$. Let $\psi(\lambda) = \log \mathbb{E}_q[e^{\lambda d}]$, so $\psi(0) = 0$ and $\text{KL}(p \parallel q) = \mathbb{E}_p[d] - \psi(1) = \psi'(1) - \psi(1) = \int_0^1 \lambda \psi''(\lambda) d\lambda$. Here $\psi''(\lambda)$ is the variance of d under the tilted law $q^{(\lambda)}$, at most $(2\varepsilon)^2/4 = \varepsilon^2$ because d takes values in an interval of length 2ε . Hence $\text{KL}(p \parallel q) \leq \varepsilon^2/2$; the reverse divergence follows by exchanging p and q . Pinsker’s inequality gives $\text{TV} \leq \sqrt{\text{KL}/2} \leq \varepsilon/2$. For sequences, the chain rule gives $\text{KL}(P \parallel Q) = \sum_t \mathbb{E}_P[\text{KL}(p_t \parallel q_t)] \leq \sum_t \varepsilon_t^2/2$, and Pinsker’s inequality again gives the total-variation bound. $\square$

To guarantee a sequence total variation of at most 0.01 over $N = 1,024$ tokens with a uniform bound, the proposition needs a logit error, up to a common shift, of at most $\varepsilon = 6.25 \times 10^{-4}$, far tighter than any int8 envelope of Section 4. An approximate head is therefore a quality trade-off to be measured (Appendix E.7); these bounds cannot certify it as harmless over long outputs.

B.7 When adaptive verification changes the sampling law

Avoiding work whose result is already determined differs from choosing which sampled proposals count. DSpark states that lossless speculation requires admission decisions that do not depend on future candidate tokens, and D-cut gives a counterexample and a repair (Cheng et al., 2026; Liu et al., 2026b); the two-token illustration below is theirs in substance. Let $p = q = (1/2, 1/2)$, draw $X \sim q$, discard it and draw afresh from p if $X = 0$, and admit it to the ordinary verifier if $X = 1$. Since $p = q$ an admitted proposal is always accepted, and the output law is

$$\Pr(Y = 0) = \frac{1}{4}, \quad \Pr(Y = 1) = \frac{3}{4}. \quad (5)$$

The acceptance formula was unchanged; admission selected proposals by their realized value. A mandatory progress floor on the first token does not help, since the same construction applies at the second position. Three settings avoid the issue: greedy decoding; target-only verification with point-mass proposals, where there is no proposal law to bias; and fixed-noise verification, under which every depth policy yields the same output (Theorem 4.5). The demand frontier below changes only the amount and order of arithmetic used to decide a fixed protocol, so it is exact pointwise.

B.8 A demand frontier over a fixed protocol

For a chain of L proposals each head row is accepted, rejected or unresolved. The first certified rejection at position r proves that no row after r can affect the cycle’s output, while earlier unresolved rows still matter because one of them may be the true first rejection.

Proposition B.8 (Safe suppression after a certified rejection). *If every issued certificate equals the decision of a fixed reference verifier on its row, the first true rejection is no later than the first certified rejection. Suppressing head work strictly after the latter does not change the reference output, provided completion and state commit use the actual first rejection.*

Proof. At the certified rejection the reference decision is rejection; an earlier unresolved row may also reject, and no later row can be the first rejection. All accepted outputs, the replacement token and the state prefix are determined by rows up to the first rejection. $\square$

The statement is for chains. SGLang’s trees share one coin across siblings and choose the next coin by the accepted node, so a tree frontier must follow the tree’s paths. The target backbone has already run over every admitted position, so the frontier saves head work, not transformer layers.

B.9 A cheap decision is not a cheap verification

A CPU experiment makes the point of Appendix B.3 concrete. It constructs two families of 128×8 integer heads with 16-row tiles and 36 paired fixtures each: one clusters rows around separated centres, the control uses unstructured rows. For each fixture $h^t = h^d + d v$ with drift scale $d \in \{0, 1, 2, 4\}$; proposals are sampled from the anchor head’s law, and all probability arithmetic is exact rational. The experiment counts distinct target rows projected, which is work, not time [E20].

At zero drift the transported summaries describe the target exactly and the acceptance decision needs only its candidate row, an identity check rather than a forecast. At nonzero drift the unstructured head defeats greedy screening. More subtly, large drift makes rejection easy to certify, which lowers the rows needed for the decision while destroying acceptance itself, and completing the residual then erases the saving (Figure 5). At drift four the clustered family decides with 11.4% of the rows but needs 96.5% once the dense residual is charged. The experiment does not test transport on trained heads (Section 5.3 does); it refutes the inference that a cheap acceptance decision means a cheap verification. Residual races answer it and remain unmeasured; fixed-noise verification avoids the residual altogether.

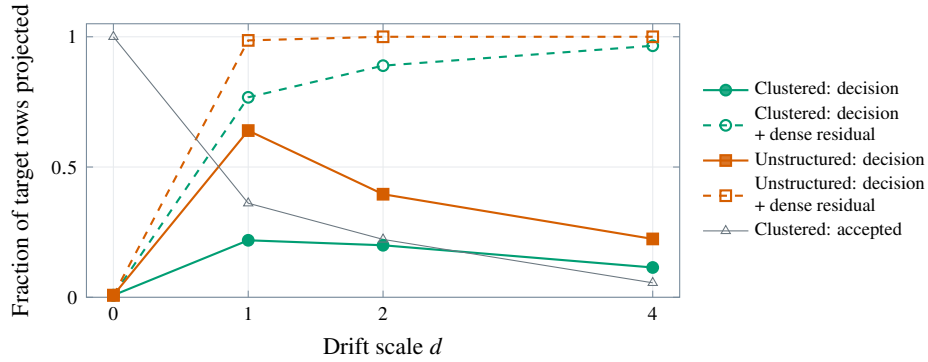


Figure 5. Work counts on constructed heads, not timings [E20]. Solid lines count the rows needed to decide acceptance; dashed lines add a dense residual completion after each rejection. The thin line is the fraction of proposals accepted in the clustered family. Evidence production, bounds, bonus sampling and the backbone are excluded.

C TRANSPORT: SUPPLEMENTARY ANALYSIS AND RESULTS

Section 5 states the transport bound, the condition under which it can be tighter than an int8 envelope, and its measured failure on both drafters. This appendix gives the geometry a transport bound can use, the proofs behind the crossover, the full weight-only constants, how the draft and target pairs were captured and checked, and the measurements the main text summarizes: the drift by block position, outcome, domain and context, certification by bound family, sampled acceptance, batch unions, and the drift-scaling extrapolation. Throughout, W is the tied head, h^d and h^t are the draft and target head inputs behind the same verified draft token, $\Delta = h^t - h^d$, and a tile c has centre μ_c and radius r_c as in Theorem 5.1.

C.1 What the bound can use

Theorem 5.1 uses a Euclidean tile radius $r_c \geq \max_{i \in c} \|w_i - \mu_c\|_2$ and charges $\epsilon_c = r_c \|\Delta\|_2$. The other natural pairing is Hölder’s, an ℓ_∞ radius $r_c^\infty \geq \max_{i \in c} \|w_i - \mu_c\|_\infty$ with $\epsilon_c = r_c^\infty \|\Delta\|_1$; its proof is the same with Hölder’s inequality in place of Cauchy–Schwarz. Either way a good draft does not imply a small ϵ_c . The head can ignore directions in which the states differ, and probabilities ignore a common shift of all logits, yet a hidden-space norm charges both. With $W = ((1, 0), (0, 0))$

and $\Delta = (0, L)$ both logits are unchanged for every L , while $r_c^\infty \|\Delta\|_1$ grows with L ; coordinate radii remove that particular failure. The bounds form a hierarchy: a scalar radius; coordinate radii, with $\epsilon_c = \sum_j r_{cj} |\Delta_j|$; grouped norms

$$\epsilon_c = \sum_g r_{cg} \|\Delta_g\|_2, \quad r_{cg} \geq \max_{i \in c} \|(w_i - \mu_c)_g\|_2; \quad (6)$$

and a low-rank split $w_i = \mu_c + Qu_i + e_i$ with the projected part evaluated exactly and a norm bound on e_i . Richer geometry costs metadata and bound arithmetic, and a learned centre or subspace does not certify the residual radius, which must cover the stored weights. In the engine the draft maxima and masses come from BF16-rounded draft logits, so every transported bound must also be widened by the draft kernel's error before it encloses a value the engine computes; the terms are those of Appendix A.2, and the replay omits them, which favours transport.

Greedy and top- k admission Suppose at least k distinct target rows have been evaluated and τ is the k -th best evaluated score. A tile may be skipped only when its transported upper bound $U_c = M_c^d + a_c + \epsilon_c$ is below τ ; then at least k evaluated items outrank every skipped one. The strict inequality preserves unresolved ties. This establishes exact selection, not an exact unrestricted softmax, because an omitted logit still contributes to the normalizer even when it cannot win.

Three small falsifiers Each shows a shortcut that the bound cannot take. *Agreement at an anchor proves nothing away from it*: let $h^d = (1, 0)$ and $h^t = (1, 1)$; rows with the same first coordinate have identical anchor logits, while their second coordinates make the target logits arbitrarily different, so an anchor maximum or mass alone cannot certify the target, and a geometric bound, more projection or exact completion is necessary. *One anchor mass cannot follow a temperature change*: the unnormalized masses $(1, 3)$ and $(2, 2)$ both sum to four, while their squares sum to ten and eight, so the mass at a new temperature cannot be reconstructed from the old sum. *Transformed logits*: if a transform adds a known per-token correction b_i with tile bounds $b_c^- \leq b_i \leq b_c^+$, then $z_i^d + a_c - \epsilon_c + b_c^- \leq z_i^t + b_i \leq z_i^d + a_c + \epsilon_c + b_c^+$ and the mass bounds multiply by $e^{b_c^\pm}$, but a mask that deletes unknown members cannot preserve the unmasked lower mass, and a safe lower bound may collapse to zero.

C.2 A summary that supports both selection and mass

For a set of labelled logits, keep its top k items under descending score and then ascending token index, together with the mass of every discarded item. Two such summaries merge by taking the top k of their retained union and adding the newly discarded mass to the two old tails.

Proposition C.1 (Top- k plus tail is a compositional summary). *For disjoint finite domains the merge produces the same top- k items and tail mass as summarizing the union, and it is associative and commutative in exact arithmetic under the common total order.*

Proof. An item omitted from a local top- k has at least k locally superior items and cannot enter the union's top- k , so every possible winner is retained. The original tails and the newly discarded items partition the final tail, so summing them gives its mass without subtracting nearly equal totals. Every parenthesization and order represents the same union. $\square$

This is a sufficient statistic built from familiar reductions, and floating-point tail masses are not associative. Its value is an interface: one epilogue can serve candidate generation, bounds and completion. Retaining a small set R_c per tile also tightens the mass bounds: recompute its target logits exactly and transport only the rest,

$$\sum_{i \in R_c} e^{z_i^t} + e^{a_c - \epsilon_c} Z_{c \setminus R_c}^d \leq Z_c^t \leq \sum_{i \in R_c} e^{z_i^t} + e^{a_c + \epsilon_c} Z_{c \setminus R_c}^d, \quad (7)$$

which is never looser than transporting the whole tile, at the cost of $|R_c|$ extra row projections.

C.3 Proofs for the crossover

The crossover compares two envelopes that are each optimal for the information they use, so no sharper inequality can change its conclusion.

Proposition C.2 (No better bound from the same information). *For fixed h and radii $\epsilon_b \geq 0$ over a partition of the coordinates into blocks b , $\sup\{|\langle e, h \rangle| : \|e_b\|_2 \leq \epsilon_b \forall b\} = \sum_b \epsilon_b \|h_b\|_2$ and $\sup\{|\langle e, h \rangle| : \|e_b\|_\infty \leq \epsilon_b \forall b\} = \sum_b \epsilon_b \|h_b\|_1$. Likewise $\sup\{\langle w - \mu, \Delta \rangle : \|w - \mu\|_\infty \leq r\} = r \|\Delta\|_1$.*

Proof. The inequalities are Cauchy–Schwarz and Hölder. Equality holds at $e_b = \epsilon_b h_b / \|h_b\|_2$, at $e_b = \epsilon_b \text{sign}(h_b)$ and at $w = \mu + r \text{sign}(\Delta)$. $\square$

For errors not aligned with h the realized $|\langle e_i, h \rangle|$ is typically about $\|e_i\|_2 \|h\|_2 / \sqrt{D}$; closing that gap needs more arithmetic, not a better inequality.

Proposition C.3 (Exclusion conditions). *After the winner a is scored exactly, let $m_i = z_a - z_i$. With ϵ_c either transported half-width, row-level transport excludes i if and only if $m_i > \epsilon_c - \langle w_i - \mu_c, \Delta \rangle$; tile-level transport excludes tile c if and only if $z_a > M_c^d + \langle \mu_c, \Delta \rangle + \epsilon_c$; self-evidence (in real arithmetic, with envelope β_i) excludes i if and only if $m_i > \beta_i - \langle e_i, h \rangle$. The slack terms lie in $[0, 2\epsilon_c]$ and $[0, 2\beta_i]$.*

Proof. Substitute $z_i^d + \langle \mu_c, \Delta \rangle = z_i - \langle w_i - \mu_c, \Delta \rangle$ and $\tilde{z}_i = z_i - \langle e_i, h \rangle$ into the upper bounds. $\square$

Proof of Theorem 5.2. For rows k and l of the tile, $\|w_k - w_l\|_2 \leq \|w_k - \mu_c\|_2 + \|w_l - \mu_c\|_2 \leq 2r_c$, so $r_c \geq \frac{1}{2} \text{diam}_2(c)$ for every centre. The equivalence is the definition of δ_i^* rearranged: $r_c \|\Delta\|_2 < \|e_i\|_2 \|h'\|_2$ exactly when $\|\Delta\|_2 / \|h'\|_2 < \|e_i\|_2 / r_c$. $\square$

The Hölder pairing has a crossover of the same kind, with a sufficient condition that depends on the tile only through one property of its rows.

Proposition C.4 (Crossover in the Hölder family). *With b -bit round-to-nearest per-row codes, $s_i = \|w_i\|_\infty / (2^{b-1} - 1)$ and no clipping, $|\langle e_i, h \rangle| \leq \frac{s_i}{2} \|h\|_1$, and $\frac{s_i}{2} \|h\|_1 \leq r_c^\infty \|\Delta\|_1$ whenever $\|\Delta\|_1 / \|h\|_1 \geq \|w_i\|_\infty / ((2^{b-1} - 1) \text{diam}_\infty(c))$. If some row of c has the opposite sign, or zero, in the coordinate where $|w_{ij}|$ peaks, then $\text{diam}_\infty(c) \geq \|w_i\|_\infty$ and the condition is implied by $\|\Delta\|_1 / \|h\|_1 \geq 1/(2^{b-1} - 1)$: $1/127$ for int8 and $1/7$ for int4.*

Proof. Round-to-nearest codes without clipping give $|e_{ij}| \leq s_i/2$, and Hölder’s inequality gives the first bound. As in the Euclidean case $r_c^\infty \geq \frac{1}{2} \text{diam}_\infty(c)$, which gives the second. If row k has the opposite sign, or zero, where $|w_{ij}|$ peaks, then $|w_{ij} - w_{kj}| \geq |w_{ij}|$, so $\text{diam}_\infty(c) \geq \|w_i\|_\infty$. $\square$

With the tile radius measured rather than bounded by the diameter, the corresponding threshold on $\|\Delta\|_1 / \|h\|_1$ has a median of 0.30% against int8 per row (Table 7); the condition $\text{diam}_\infty(c) \geq \|w_i\|_\infty$ holds for 99.9% of rows with 64-row tiles [E10].

Proposition C.5 (Transport against the static screen). *If μ_c lies in the convex hull of the tile’s rows, then $U_c^T - U_c^{\text{static}} \geq r_c(\|\Delta\|_2 - \|h\|_2)$, where $U_c^T = M_c^d + \langle \mu_c, \Delta \rangle + r_c \|\Delta\|_2$ and $U_c^{\text{static}} = \langle \mu_c, h \rangle + r_c \|h\|_2$ are the upper bounds of transport and of a static screen at the target state with the same geometry. For the Hölder pairing, $U_c^T - U_c^{\text{static}} \geq r_c^\infty (\|\Delta\|_1 - \|h\|_1)$ with both bounds in r_c^∞ and ℓ_1 norms. Transport is therefore no tighter than the static screen once the drift is as large as h in the norm the bound uses.*

Proof. $U_c^T - U_c^{\text{static}} = (M_c^d - \langle \mu_c, h^d \rangle) + r_c(\|\Delta\|_2 - \|h\|_2)$, and $M_c^d = \max_k \langle w_k, h^d \rangle \geq \langle \mu_c, h^d \rangle$ for a convex combination μ_c . The Hölder case is the same computation. $\square$

On both drafters the drift is close to the size of h in either norm: $\|\Delta\|_2 / \|h'\|_2$ has a median of 0.92 for DFlash-4B and 0.95 for MTP-4B, and $\|\Delta\|_1 / \|h'\|_1$ of 0.93 and 0.99 [E11, E13]. The proposition therefore leaves transport at most a few percent of the static screen’s radius term to gain, and Table 5 shows that it gains nothing measurable.

C.4 Weight-only constants

Table 7 gives the constants of the tied head from which the thresholds of Section 5.2 are derived. They use no hidden states, so they decide nothing about certification rates on their own. Two further facts follow from them. The static tile screen is weak on this head, because the 64-row tile radius exceeds the row norm (median $r_c / \|w_i\|_2 = 1.14$), so $\langle \mu_c, h \rangle + r_c \|h\|_2$ is wider than the plain row bound $\|w_i\|_2 \|h\|_2$, and tiles in token order are barely tighter than random ones; a public experiment on a Qwen3.8 head with the same vocabulary also found that centroid-radius bounds prune almost nothing (qwen38-challenge_senpai contributors, 2026). Bounding an inner product under a bounded displacement of the query, which is what transport does, is known from sequential screening rules for the lasso (Wang et al., 2015) and from cached retrieval with drifting queries (Frieder et al., 2024). And a scalar ℓ_∞ radius paired with $\|\Delta\|_1$ is about four times looser than the Euclidean pairing for a spread-out Δ (derived from the same constants, $r_c^\infty \approx 1.30 \|w_i\|_\infty$ and $\|\Delta\|_1 \approx 0.8 \sqrt{D} \|\Delta\|_2$).

Quantity (tied head, $248,320 \times 2,560$)	p10	p50	p90
Row norm $\ w_i\ _2$	0.574	0.652	0.746
int8 per-row $\ e_i\ _2/\ w_i\ _2$	0.81%	0.97%	1.25%
int8, 128-column groups	0.64%	0.68%	0.75%
int4 per-row	14.8%	17.5%	22.6%
64-row tile $r_c/\ w_i\ _2$, token-index order	0.99	1.14	1.31
random tiles (control)	1.03	1.19	1.37
δ^* against int8 per-row	0.67%	0.85%	1.15%
δ^* against int8, 128-column groups	0.51%	0.61%	0.72%
δ^* against int4 per-row	12.2%	15.5%	20.8%
Hölder threshold on $\ \Delta\ _1/\ h\ _1$ against int8 per-row (Proposition C.4)	0.23%	0.30%	0.40%

Table 7. Weight-only constants of the Qwen3.5-4B head, computed in float64 on the CPU; quantiles over all 248,320 rows. $\delta_i^* = \|e_i\|_2/r_c(i)$ is the drift ratio below which transport’s Euclidean envelope for row i is narrower than the Euclidean envelope of the named low-precision head, with 64-row tiles of contiguous token indices and μ_c the tile mean; the last row is the analogous threshold for the ℓ_∞/ℓ_1 pairing, $s_i\|h\|_1/2$ against $r_c^\infty\|\Delta\|_1$. Envelope widths are compared, not certification rates or runtime. Derived [E10].

C.5 Captured pairs and their alignment

Speculative pairs follow the verify layout. A verify step for one request holds $[x_0, d_1, \dots, d_S]$ with x_0 the last committed token; draft token d_k came from the draft head input h_k^d and is decided by the target head input at verify position $k - 1$. Each pair is labelled accepted, rejected or unreached (after the first rejection). The target input at the last verify position, which produces the bonus token, has no draft anchor at all, so a transport-only verifier still streams the whole head for that row. The captures used the patch and the server configuration of Section 3.1 (no CUDA graphs and no overlap scheduler, because the hooks run in Python between forwards), the same 320 prompts split by prompt into analysis and held-out halves, greedy decoding and 384 new tokens; the DFlash drafter ran at block size 16 with five concurrent requests, and the MTP layer with four draft steps, top- k 1 and at most eight running requests [E15, E13].

The DFlash-4B capture was checked as the plain-decode capture was [E15]. It holds 24,020 verify blocks and 360,300 draft slots. The FP64 argmax of the captured draft head input equals the engine’s draft token at 98.2% of slots, and that of the target head input the engine’s target token at 98.9%; every disagreement lies within one BF16 spacing of the FP64 winner, and the exact logits rounded to BF16 with the first-index rule (R-bf16) reproduce the engine at 99.996% (draft) and 99.998% (target). The engine’s accepted lengths match the labels derived from the captured tokens on all 24,020 blocks. Draft agreement falls from 99.3% at block position 1 to 97.0% at position 15, as later draft distributions become flatter and near ties more common. Only 39% of prompts produce output identical to plain decoding; over all prompts, counting identical outputs at their full 384 tokens, the median first diverging output token is 181. This is not a pairing error, since every per-slot check uses the engine’s own decisions. The verify pass runs different kernels, at different batch shapes, from plain decoding, and 75 of the 77 divergences of DFlash outputs from plain decoding measured on a second panel are at near ties (Appendix F.4); their trace to a kernel is [pending D25].

The MTP-4B capture passes the same checks [E16]. It holds 30,328 verify blocks and 121,312 draft rows. The FP64 argmax of the captured draft head input equals the engine’s draft token at 99.08% of rows, and that of the target head input the engine’s target token at 99.33%; all 1,116 draft and 816 target disagreements lie within one BF16 spacing, R-bf16 reproduces the engine at 99.997% (draft) and 99.998% (target), and the engine’s accepted lengths match the derived labels on all 30,328 blocks. Draft agreement falls from 99.31% at step 1 to 98.90% at step 4, and a reached step is accepted at 84.5%, 80.0%, 79.5% and 80.6% at steps 1 to 4. The engine’s own top two target logits were equal in BF16 at 1.3% of rows. 42% of prompts produce output identical to plain decoding, and the median first diverging output token, counted as for DFlash-4B, is 193.5.

The analyses use subsets of the held-out pairs: the DFlash drift on 40,000 pairs (9,296 accepted, 2,434 rejected and 28,270 unreached), DFlash transport on 4,020 pairs from 110 prompts, the MTP drift on 40,000 pairs from 160 prompts (24,835 accepted, 5,698 rejected and 9,467 unreached), and MTP transport on all 16,016 held-out pairs from 160 prompts (9,969 accepted, 2,298 rejected and 3,749 unreached) [E11, E12, E14, E13].

C.6 The drift in detail

Table 8 breaks the DFlash drift ratio down. It barely depends on anything: accepted drafts are no closer than rejected ones, the median rises from 0.88 at block position 1 to 0.92 from position 6 on, and domain and context length move it by a few hundredths. For every pair, transport’s envelope is narrower than any of the four low-precision envelopes (int8 and int4, per row and in 128-column groups) on 0.08% of rows, and those rows are the three tiles of identical unused-token rows, whose radius is zero [E11].

Group	Pairs	p10	Median	p90	Group	Pairs	p10	Median	p90
All	40,000	0.875	0.917	0.964	Block position 1	2,690	0.852	0.882	0.938
Accepted	9,296	0.865	0.909	0.958	Block position 2	2,634	0.865	0.900	0.957
Rejected	2,434	0.856	0.891	0.952	Block position 4	2,625	0.875	0.915	0.963
Unreached	28,270	0.884	0.920	0.966	Block position 8	2,729	0.880	0.921	0.966
Chat	13,374	0.876	0.915	0.956	Block position 12	2,661	0.886	0.922	0.966
Code	4,880	0.880	0.923	0.961	Block position 15	2,716	0.889	0.924	0.964
Mathematics	6,240	0.875	0.925	0.965	Context 0–255	20,109	0.874	0.915	0.959
Multilingual	15,506	0.872	0.914	0.970	Context 256–511	17,715	0.875	0.918	0.965
					Context 512–1,023	1,909	0.881	0.934	0.981
					Context $\geq 1,024$	267	0.877	0.930	0.966

Table 8. Drift ratio $\rho = \|h^t - h^d\|_2 / \|h^t\|_2$ of DFlash-4B pairs by outcome, domain, block position and context length (tokens before the verified position), on 40,000 held-out pairs. The smallest ratio over all pairs is 0.823. Measured [E11].

The main text explains the size of the drift by the draft input’s fixed norm and its angle to the target’s. Three further measurements show that the drift is not an artefact of the Euclidean metric [E11, E12]. In the head’s own metric the drift stays of order one: $\|W\Delta\|_2 / \|Wh^t\|_2$ has a median of 0.87 before and 0.85 after removing each position’s mean logit, and the median over pairs of the largest logit drift $\max_i |\langle w_i, \Delta \rangle|$ is 13.6 logits (root mean square 2.0). Restricted to W ’s leading 64 right singular directions, the relative drift is 0.89, and those directions carry 25% of Δ ’s norm, against 16% for an isotropic vector. In the ℓ_1 norm of the Hölder family the ratio is 0.93.

Table 9 breaks down the drift of the MTP layer. It is wider than DFlash’s and reaches lower, with a median of 0.954 and a smallest ratio of 0.385 against 0.917 and 0.823, because the MTP head input is not held at a fixed norm (medians 176 for the draft and 160 for the target) [E13]. It also depends more on the draft: the median rises from 0.86 at step 1 to 1.00 at step 4, rejected drafts lie lower than accepted ones (0.89 against 0.99), and code and mathematics (both 1.02) lie above chat and multilingual prompts (0.92). The other metrics agree [E14]. In the head’s metric the median is 0.83, and 0.88 after removing each position’s mean logit; in W ’s leading 64 right singular directions it is 0.86, and those directions carry 23% of Δ ’s norm; the median ℓ_1 ratio is 0.99; and the median largest logit drift is 10.7 [E13]. None of this changes the verdict. Transport’s envelope is narrower than int8 per row only on the 0.08% of rows in the tiles of unused tokens, the realized per-tile transport error is a median 81 times int8’s, and certified ℓ_2 transport and the static screen each skip 0.08% of rows. Even oracle radii skip 99.97% (median; p10 98.6%), where int8 per-row self-evidence decides with about one row [E14].

C.7 Certification by bound family

Table 5 in the main text gives, per tiling, the best certified family, which is the elementwise minimum of all certified radii. Table 10 gives every family on the tiling where transport did best, k -means tiles of 64 rows. The greedy skip hardly depends on the family: every certified family skips between 0.52% and 0.64% of the vocabulary on average, within 0.014 percentage points of the static screen with the same family. The family does matter for the partition interval of sampled acceptance, which the scalar ℓ_∞ radius leaves 820–901 nats wide and the grouped, low-rank and combined families narrow about sixfold, to 134–169 nats. No certified family brings it below 130 nats, and even oracle radii, at 12–13 nats, leave the acceptance test undecided most of the time (Appendix C.8). On these tiles, row-level transport, which bounds each row by its own deviation from the tile centre instead of the tile’s largest, adds at most 0.012 percentage points to the tile-level skip.

C.8 Sampled acceptance and batch unions

For sampled acceptance the relevant quantity is the probability $P_?$ that the guards of Theorem B.2 leave the test undecided. Transported masses leave it near one whichever way it is taken [E12, E13]. For DFlash-4B with the best certified family, its mean is 0.92–0.96 on k -means 64-row tiles, whether it is taken at the draft token or averaged over proposals $x \sim q$, at

The Work a Verifier Needs

Group	Pairs	Smallest	p10	Median	p90
All	40,000	0.385	0.767	0.954	1.174
Accepted	24,835	0.385	0.760	0.986	1.201
Rejected	5,698	0.415	0.728	0.887	1.085
Unreached	9,467	0.597	0.808	0.935	1.109
Chat	11,711	0.410	0.757	0.919	1.119
Code	7,720	0.470	0.804	1.016	1.216
Mathematics	8,929	0.385	0.796	1.025	1.211
Multilingual	11,640	0.418	0.747	0.917	1.131
Draft step 1	10,069	0.385	0.667	0.864	1.092
Draft step 2	10,035	0.520	0.783	0.947	1.157
Draft step 3	9,871	0.611	0.823	0.982	1.190
Draft step 4	10,025	0.607	0.845	1.002	1.214
Context 0–255	19,604	0.385	0.767	0.949	1.165
Context 256–511	18,379	0.410	0.766	0.957	1.182
Context 512–1,023	1,733	0.445	0.781	0.972	1.175
Context $\geq 1,024$	284	0.516	0.782	0.967	1.169

Table 9. Drift ratio ρ of MTP-4B pairs by outcome, domain, draft step and context length, on 40,000 held-out pairs from 160 prompts. Measured [E14].

Bound family	DFlash-4B (4,020 pairs)				MTP-4B (16,016 pairs)			
	Skipped		$\log(Z^+/Z^-)$		Skipped		$\log(Z^+/Z^-)$	
	transport	static	transport	static	transport	static	transport	static
Scalar ℓ_∞ radius	0.52%	0.52%	820	890	0.53%	0.53%	901	930
Coordinate radii	0.59%	0.58%	287	312	0.64%	0.64%	315	324
Grouped ℓ_2 , groups of 32	0.53%	0.53%	158	173	0.53%	0.53%	169	176
Grouped ℓ_2 , groups of 128	0.53%	0.53%	144	158	0.53%	0.53%	152	159
Low rank, basis from W (64)	0.53%	0.53%	150	165	0.54%	0.53%	151	160
Low rank, basis from Δ (64)	0.54%	0.53%	135	146	0.55%	0.54%	147	157
Low rank, basis from Δ (256)	0.55%	0.54%	151	163	0.56%	0.55%	166	174
Minimum of all certified radii	0.59%	0.58%	134	146	0.64%	0.64%	146	156
Oracle radii (not certified)	83.7%	89.7%	13	17	96.0%	99.4%	12	23

Table 10. Transport by bound family on k -means tiles of 64 rows: mean share of the vocabulary skipped for greedy verification (tile level), and the median width of the partition interval in nats at $T = 1$, for transport and for the static screen at the target state with the same geometry. Oracle radii are the realized per-tile deviations, a ceiling that no certified geometry reaches. Measured [E12, E13].

$T = 1$ or $T = 0.7$, and 0.95–1.00 on every other tiling; for MTP-4B it is at least 0.997 on every tiling. Computing the exact logits of the 64 largest draft rows of each tile and transporting only the rest (Equation (7)) narrows the median partition interval to 110–125 nats on DFlash-4B and lowers P_γ at the draft token to 0.58 on average, with a median still of one (0.81 on MTP-4B). Even oracle radii leave the mean P_γ over proposals at 0.68–0.78 for DFlash-4B and 0.79–0.92 for MTP-4B across tilings, because their partition interval is still 12–18 nats wide at the median.

A skipped tile saves weight traffic only if no row of the batch needs it. If each row of a batch of B independently needed a tile with probability f , the batch would need it with probability

$$1 - (1 - f)^B, \quad (8)$$

about 87.3% at $f = 1/16$ and $B = 32$, so a per-row skip of 93.75% can coexist with an almost full weight stream. That is a model of independent masks; the measured unions follow. Over the capture’s real verify batches, the union of tiles left unresolved covers 99.48–100% of the vocabulary for every certified family and tiling on DFlash-4B (batches of 9 to 128 paired rows) and 99.4–100% on MTP-4B (1 to 32 rows). Oracle radii fare little better once rows are grouped: with k -means 256-row tiles, random groups of 4, 16 and 64 DFlash-4B pairs need 63%, 97% and 99.9% of the vocabulary, and groups of 16 and 64 MTP-4B pairs need 62% and 92%. The union alone therefore defeats a perfect geometry at batch sizes the verifier serves.

C.9 The drift-scaling extrapolation

How far would a drafter have to move before transport became useful? The replay answers with a model, not a drafter: it replaces h^d by $h^t - s\Delta$, which scales ρ , the logit drift and every radius term by s , and re-evaluates the best certified family on k -means 256-row tiles. Table 11 gives the result. On DFlash-4B, greedy transport skips half the vocabulary on average only at a tenth of the measured drift, and the sampled acceptance test decides a third of proposals only at a hundredth. The MTP layer needs a somewhat smaller reduction for greedy transport (80% skipped on average at $s = 0.1$, against 49%) and the same for sampled acceptance.

Scale s	DFlash-4B				MTP-4B			
	skip, mean	skip, median	$\log(Z^+/Z^-)$	$\mathbb{E}_q P_\gamma$	skip, mean	skip, median	$\log(Z^+/Z^-)$	$\mathbb{E}_q P_\gamma$
1 (measured)	0.10%	0.09%	138	0.98	0.11%	0.11%	152	1.00
0.3	1.6%	0.3%	49	1.00	1.9%	0.4%	58	1.00
0.1	49%	49%	23	1.00	80%	96%	25	1.00
0.03	77%	96%	7.0	0.96	96%	99.8%	7.5	0.97
0.01	84%	99%	2.3	0.66	98%	99.9%	2.5	0.70

Table 11. Drift-scaling extrapolation, a model and not a measured drafter: the draft input is replaced by $h^t - s\Delta$ and the best certified family on k -means 256-row tiles is re-evaluated. Shares skipped are for greedy verification; the partition width is a median in nats and $\mathbb{E}_q P_\gamma$ a mean, both at $T = 1$. Derived from measured pairs [E12, E13].

The extrapolation says nothing about whether any drafter can reach such a drift. A drafter trained with an auxiliary loss on $\|h^t - h^d\|_2^2 / \|h^t\|_2^2$ is the obvious test and is untried (Appendix F.4).

C.10 From skipped work to time

A skipped tile saves time only if the GPU avoids the work. For each mechanism the replay counts the weight bytes read (refined tiles including their union over the real batch, gathered candidate rows, the low-precision pass), the metadata and evidence bytes, the sequential stages and the fallbacks, and converts them to predicted time with the kernel’s measured primitive costs. The result would be a model prediction, not a timing, and it is [pending D9].

D EXACTNESS CONTRACTS AND THE STOCK ENGINE

Section 2 names the stock head kernel at the same batch shape as the reference for the certified head. This appendix places that choice among the contracts a decoder can promise, lists what each consumer of the head needs, and gives the evidence behind the same-shape rule: where stock configurations that ought to agree already disagree, and how the engine keeps its recurrent state under speculation.

D.1 Five contracts and the witnesses that separate them

An exactness claim needs a reference and a sense of equality. The reference is an executable specification, not an idealized network: the weights, the kernels, the decoding policy, the random-number protocol and the execution shape, together with whether it is stock plain decoding, stock speculative verification or a recorded execution. Mathematical equivalence over the reals does not imply that two GPU reductions choose the same token, because a different batch shape or reduction order can move logits that are close (NVIDIA, 2026b; PyTorch contributors, 2026); a certificate must therefore enclose the value the reference computes, casts included. Table 12 lists the contracts a change can claim against such a reference and the results of this paper that rest on each. They are ordered roughly from strongest to weakest, but none implies the next in general.

Contract	What must be equal	Results that claim it	Status
Identical internal state	Named logical tensors (recurrent state, attention cache) at named boundaries, bitwise; not layouts or scratch buffers	Replay of recorded GDN updates (Proposition D.2); rounding-preserving replay (P4, Appendix E.5); SGLang’s speculative state path (H5, Appendix D.5)	Proved; P4 and H5 tests [pending D17, D12]
Identical greedy outputs	Decisions, ties and stopping, with a valid state from which to continue	Certified head under R-stock (Theorem 2.3, Proposition 2.4)	Proved under an error model; measured on replayed rows
Identical seeded samples	The reference’s mapping from seed to output; with an inverse-CDF reference, certify with CDF bounds against the same uniform and fall back without redrawing	Certified race with the stock seeded field; fixed-noise verification against the seeded sampler on the verify pass’s logits (Section 4.3)	Proved; measurement [pending D8]
Exact sampling distribution	A named law: the ideal model’s, the categorical law of the reference’s scores, or the finite random-number implementation’s; speculative correction targets the verifier’s own p	Races (Theorem 4.4); fixed-noise verification in law; residual races (Appendix B.4)	Proved
Empirical quality equivalence	Task scores and distances within declared margins, with confidence intervals	The lossy stack under its quality budget (Appendix E.7)	Budget declared; measurement [pending D21]

Table 12. Exactness contracts and the results that claim them. “Measured on replayed rows” means captured engine states replayed offline, not a served run.

Small exact witnesses separate the contracts [E21]. Reassociating an FP32 sum flips a greedy winner: $1 + 2^{-24} + 2^{-24}$ summed left to right returns 1 and ties a competing logit of 1, so the first-index rule picks the competitor, while the regrouped sum returns $1 + 2^{-23}$ and wins, as it does in real arithmetic. Distinct FP32 values, 1 and $1 + 2^{-10}$, fall in one BF16 rounding cell, so a BF16-output head ties them and the index rule decides. Logits (1, 0) and (2, 0) have the same greedy decision but different laws: $(\frac{2}{3}, \frac{1}{3})$ against $(\frac{4}{5}, \frac{1}{5})$ in the base-2 mass model, and 0.731 against 0.881 on the first token at temperature one. Two inverse-CDF samplers that list the tokens of $p = (\frac{1}{2}, \frac{1}{2})$ in different orders have the same law but map the uniform $\frac{1}{4}$ to different tokens. Evaluating the gated-delta recurrence $S_t = S_{t-1}A_t + B_t$ in the split form $S_t = S_0P_t + W_t$, which is equal in exact arithmetic, changed 1,481 of 1,600 FP32 outputs over 50 steps at width 32, and 2 of them after rounding to BF16. A state difference the current query cannot see becomes visible after one recurrent step (Appendix E.5), so equal outputs now do not imply equal state. Two greedy decoders that each answer two of four problems correctly have equal accuracy, yet their answers can differ on every problem, so empirical quality equivalence implies none of the stronger contracts. Equal token sequences, finally, say nothing about latency or about whether the cache that produced them is owned correctly. A last witness, in the exact floating-point reference, separates R-bf16 from R-stock: with $w_0 = (16, 0, 0)$, $w_1 = (16, \frac{1}{16}, 2^{-30})$ and $h = (1, 1, 1)$, the real logits are 16 and $16.0625 + 2^{-30}$, so R-bf16 rounds them to 16 and 16.125 and picks token 1, while a sequential FP32 accumulation loses the 2^{-30} , rounds 16.0625 to even and returns 16 for both rows, so the stock argmax picks token 0; the gap condition refuses to certify this row [E17]. The floating-point witnesses use IEEE binary32 with round to nearest and every reduction written as an explicit sequential loop; a kernel that contracts, reorders or accumulates differently can round differently. They show that the contracts are distinct in general and say nothing about how often the differences occur in the engine.

D.2 The commit rule in general

Every certificate in this paper instantiates one rule. Partial computation yields information I about the final logits: an approximate pass with error bounds, a transported summary, or an approximate hidden state with a radius. From I we keep a *sound set* $Z(I) \subseteq \mathbb{R}^V$ that contains the reference implementation’s logits. A decision rule $D(z, R)$ maps logits and the randomness fixed by the contract (a noise field, a uniform draw) to a token or an accept bit.

Definition D.1 (Commit rule). Commit the decision d when $\{D(z, R) : z \in Z(I)\} = \{d\}$. Otherwise acquire more information, which must shrink or replace $Z(I)$ by another sound set, and test again.

If the reference logits z^* lie in $Z(I)$, a committed d equals $D(z^*, R)$; that is the whole soundness argument, and it holds whatever produced $Z(I)$. For a box $Z(I) = \prod_j [l_j, u_j]$ the greedy rule commits c when $l_c > \max_{j \neq c} u_j$, and a Gumbel race with a fixed field G commits c when $l_c/T + G_c > \max_{j \neq c} (u_j/T + G_j)$, which needs no partition function as long as the field stays fixed through every refinement (Maddison et al., 2014). When the information is about the hidden state rather than the logits, a box is loose and the directional certificate of Proposition A.5 is the right one. The rule names the reference; it is the gap condition of Theorem 2.3 that turns a sound set for the real logits into one for the stock kernel’s decision.

D.3 What each consumer of the head needs

Table 13 lists the information each consumer of the head requires. A fast path is admitted by these contracts, not by a request being labelled “decode”. Grammar masks, repetition penalties and temperature transforms are part of the scores; they cannot be applied after candidates have been discarded.

Consumer	Required information	Certificate in this paper
Greedy token	Winning index under the reference kernel and tie rule	Ladder plus stock contract (Theorems 4.3 and 2.3)
Exact top- k	Winning indices in order, and their scores; no partition function	Ladder on membership and on the order within the set (Appendix B.1)
Seeded or fixed-noise sample, full vocabulary	Argmax of perturbed scores under the token-keyed field	Certified race (Theorem 4.4)
Seeded top- k or top- p sample (stock)	Each candidate’s rank in the sorted probabilities, which keys its noise, and the sort’s order on ties; for top- p , the mass above each token	Membership and complete order within the set, then the race (Appendix B.1; analysis)
Acceptance bit	$\exp(z_X)$ against $Uq_X Z$; an interval can suffice	Guards (Appendix B.3; analysis). The engine path uses fixed-noise verification instead
Residual sample	The $(p - q)_+$ law, including omitted mass	Residual races (Appendix B.4; analysis)
Bonus sample	A sample from the target law under the random-number contract	Certified race
Top- p sample	Mass above each token relative to Z	Partition bounds, or the ordinary path
Arbitrary log-probabilities	Requested logits and the full partition function	None; the ordinary path

Table 13. Consumers of the head and what each needs. The last rows are where a narrow fast path usually loses.

D.4 Where equivalent stock configurations disagree

Plain decoding at different batch sizes, plain decoding against MTP speculation, and speculation at different batch sizes can emit different greedy tokens. Both greedy paths reduce to one rule, `torch.argmax` (first maximal index) over the BF16 head output widened exactly to FP32, so two runs that feed identical logits to it cannot choose different tokens. Every divergence therefore means that the computed logits differed, and the question is where. “A tie” is an observation about where the divergence became visible, not an explanation of why it happened. Batch-invariant kernels make greedy outputs reproducible across batch compositions (He & Thinking Machines Lab, 2025), and floating-point nondeterminism in inference has been characterized and partly mitigated (Yuan et al., 2025); what follows locates the kernels responsible in this engine. In brief: neither the tie rule nor the head GEMM caused a divergence [E23]; plain decoding and MTP speculation part inside layer 0’s recurrence, whose decode and verify kernels produce different bits from identical inputs and state [E26]; plain decoding at two concurrencies parts at batch-dependent kernels [E23]; with identical pools, MTP diverges from plain decoding at 3.5 to 4.0 per 1,000 compared tokens, always at an exact tie or within two BF16 spacings [E28]; and with the prefix cache on, a request’s output can depend on which earlier request computed its shared prefix and on what the cache holds when it is

prefilled [E29, E26].

Differential tests A fixed set of 320 prompts from GSM8K, HumanEval, MT-Bench, AlpacaEval and CNN/DailyMail at pinned revisions, tokenized once so that every configuration receives identical token IDs (105 of them in the model’s thinking mode), is decoded greedily for up to 256 tokens with the top five log-probabilities at every position. The configurations differ only in settings that should leave greedy output unchanged: plain decoding against MTP speculation (one, three and five draft steps with top- k 1, and a small tree with top- k 2), the prefix cache and the overlap scheduler on or off, `--enable-deterministic-inference` and `--enable-fp32-lm-head`, each with one request in flight and with a client concurrency of 32. In the first matrix every server was capped at 16 running requests, so “concurrency 32” meant decode batches of up to 16 with the rest queued; the pinned matrix below caps every server at 8 with identical pools. Two runs of a prompt are compared position by position; the first position where the tokens differ is a divergence, the positions before it are the exposure, and the rate per 1,000 tokens is divergences over exposure. Every run uses the engine at the pin with CUDA graphs, the overlap scheduler and the FlashInfer attention backend [E24].

Locating the first difference A diagnostic tap (`engine/sglang/patches/state/0001-state-tap.patch`) hashes the exact bits of every module output of the target, one row per token, and two points inside the GDN backend (after the causal convolution and after the recurrent kernel), into static device buffers that the CUDA graphs capture and replay; for selected requests it also saves the head input and the logits fed to the argmax. For each divergence both configurations are replayed on the prompt and the committed rows (for MTP, only verify rows on the accepted path) are walked in order to the first position and, in execution order, the first module whose output bits differ. Before that point every hashed module output agrees. The caches (attention KV, the GDN convolution window and the recurrent state) are not module outputs, so a difference written into a cache without a differing output would first show in the module that reads it. A later version of the tap closes that gap: it also hashes the caches each forward reads, the keys and values of every attention layer at every cached position and the convolution window and recurrent state of every GDN layer [E26]. The comparison skips the caches of a prefill with no cached prefix, which reads no earlier state: its recurrent slot can still hold an earlier request’s state when the tap hashes it, because the engine clears the slot inside the forward. This version has been run at batch 1 for plain decoding against MTP, for the history pairs, for the prefix cache on against off and for one prompt’s same-server repeats (below); for the concurrency pair the check is [pending D11]. The tool was checked on itself: a one-ulp change injected into layer 9’s MLP down-projection output is named at that module, at the first prompt token, in 4 of 4 prompts, and one injected into layer 13’s GDN recurrence output is named at that operation in 4 of 4 when the search starts at the first decode position; searched from the prompt, the second is attributed to the enclosing GDN attention module, because in prefill the attention backends run eagerly between captured graph segments [E25]. With the tap version behind Figure 6, tapped runs match untapped ones in tokens and log-probabilities for 162 of 167 prompts [E23], and with the cache-hashing version for 35 of 40, the other five being the same five prompts [E26]. The tapped and untapped servers allocated different pools, 159,322 (159,324 for the cache-hashing version) against 97,672 attention-cache tokens and 199 against 122 recurrent-state slots, so the five exceptions are not attributed to the tap itself [E27]; they are discussed below, with the history dependence through the prefix cache. Outside the pinned matrix below, every comparison in this section across two servers had different pools, while the concurrency comparison, the same-server repeats and the deterministic-mode checks compare passes on one server.

Findings that do not depend on an error model Two pairs have been analysed [E23]. First, neither the tie rule nor the head GEMM caused a single divergence: in every case the head input itself already differed. Second, the first differing module output is specific to each pair (Figure 6). For plain decoding against MTP speculation with three draft steps, both at batch 1, it is layer 0’s GDN recurrence output at the first speculative cycle, in all 167 prompts analysed, while that layer’s causal-convolution output, the recurrence’s immediate input, is bitwise equal. At the pin, decode and target verify call different Triton kernels (`fused_recurrent_gated_delta_rule_packed_decode` and `fused_sigmoid_gating_delta_rule_update`), which fuse the gating and the state update differently. Rerun with the cache-hashing tap on 40 of these prompts, every cache entering every forward is identical up to that output, and the first cache to differ is layer 0’s recurrent state, which the differing kernel writes [E26]; the two kernels therefore produce different bits from identical inputs and identical state. The two tapped servers of the 167-prompt analysis had different pools (159,322 against 166,522 attention-cache tokens and 199 against 97 recurrent-state slots; 159,324 against 166,481 tokens and 199 against 97 slots for the cache-hashing rerun), and MTP’s draft layer attends over its own cached prefix, so a different copy of a shared prefix could change the proposals, the accepted lengths and the later verify forwards without any target attention output differing first. The location of the first difference bounds the pools only for that difference, which is at output index 1, the first row of the first verify forward, whose layer-0 recurrence output depends on its token and on the state left by the prefill, not on the proposals or on cached attention

values (reasoning from the row-wise structure of the kernels); whether the pools contributed to the divergences that follow is [pending D11]. For plain decoding at concurrency 1 against 32 (batches of up to 16; 40 tapped prompts), it is a GDN gated RMSNorm in decode in 29 prompts, a full-attention layer’s output in decode in 6, and the MLP down-projection of a prefill batched with other requests in 5; never the GDN recurrence and never a decode GEMM. These two tapped servers also had different pools (159,322 against 89,652 attention-cache tokens and 199 against 111 recurrent-state slots). A different copy of a shared prefix’s attention values would first show at a full-attention output, so for the 6 prompts that first differ there the pools and the prefix cache’s history are possible causes besides the attention kernel’s dependence on the batch; for the other 34, every attention output before the first difference agrees. The gated norm chooses how many rows each program reduces from the batch’s row count (one row per program for a batch of 1, two for a batch of 16), which changes the reduction tile and so the FP32 sum order of a row; the same function returns a constant in batch-invariant mode. FlashInfer’s decode plan and cuBLAS’s GEMM choice also depend on the batch. For this pair these are first differing module *outputs* only; that both runs enter the differing kernel with equal caches is [pending D11].

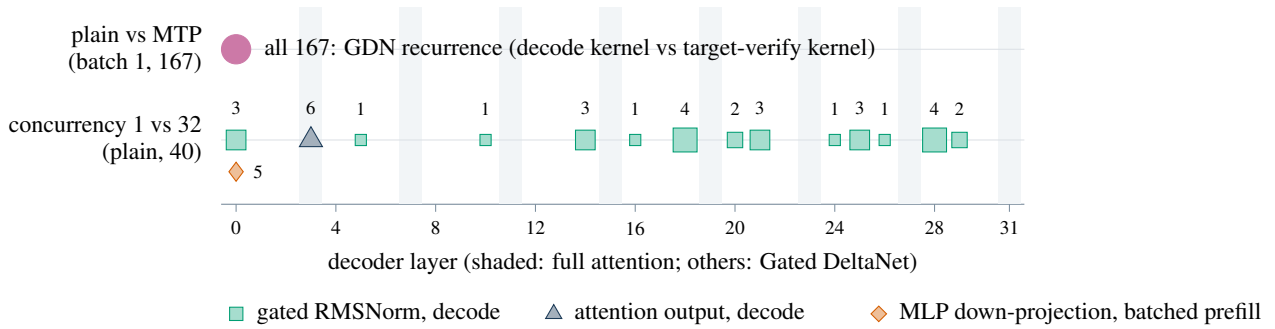


Figure 6. Where equivalent stock configurations first produce different module outputs, by decoder layer. Plain decoding and MTP speculation part at layer 0’s GDN recurrence in every prompt, because decode and target verify use different recurrent kernels. Plain decoding at concurrency 1 and 32 (batches of up to 16) part at batch-dependent kernels spread over the depth: the gated RMSNorm of GDN layers (marker size and label give the count), layer 3’s attention, or layer 0’s MLP in a batched prefill. For plain decoding against MTP, a rerun of 40 prompts that also hashes the caches finds every cache entering the differing recurrence identical; for the concurrency pair these are first differing module outputs only, and whether the caches entering them agree is pending. Each pair’s two tapped servers had different pools. Measured with a bit-exact tap [E23, E26].

How the difference reaches the token At the position where the runs first choose differently, with competing tokens a and b , the analysis compares in each run the order of the two tokens’ FP32 accumulator values in the head GEMM, before BF16 rounding. Where a run’s BF16 logits differ, rounding is monotone and gives that order. Where they are tied, the float64 dot product of that run’s head input with the two rows stands in for the accumulator, but the two can differ by up to $\gamma(A_a + A_b)$, with A_t the sum of absolute products of token t ’s dot product. A *rounding flip* keeps the accumulator order in both runs, but in one run BF16 rounding merges the two into an exact tie that the index rule resolves against it; an *order flip* reverses the accumulator order between the runs; an *ambiguous* case has a tied run whose float64 gap lies inside the accumulation bound, so its accumulator order cannot be read from the data. The split therefore depends on the error model assumed for the stock accumulator (Table 14). Where both the BF16 order and a float64 gap outside the bound are available, they never disagree under either model. Under the conservative model most divergences are ambiguous, 93 of 129 and 22 of 40: the stock kernel’s possible accumulation error is as large as the gaps that decide these tokens, which is the same fact that makes the gap condition of Theorem 2.3 fail at near ties. Under the Hopper model most plain-versus-MTP divergences are rounding flips (89 of 129), and the concurrency-1-versus-32 divergences divide between order flips (17) and rounding flips (14). Whatever the model, a difference changes the chosen token only where the two leading logits are close: in the concurrency comparison, 151 of its 167 divergences are at an exact BF16 tie, 14 within one BF16 step, and none has a margin above 0.375 nats [E24].

How often In the first matrix, at a cap of 16 running requests, plain decoding at concurrency 1 against 32, two passes on one server with identical pools, diverges in 167 of 320 prompts within 256 tokens, 3.42 divergences per 1,000 compared tokens, with a largest margin at the divergence of 0.375 nats. Repeating either configuration on the same server reproduced every token of all 320 prompts, and every log-probability except those of one 128-token prompt, which differ from its prefill onward in both repeats. A fresh server at batch 1 reproduced the first pass bitwise on all 320 prompts, that one included, so

Pair	Diverged	Caused by		Error model	Class at the divergence		
		tie rule	head GEMM		rounding	order	ambiguous
Plain vs MTP, batch 1	129/167	0	0	conservative model	29	7	93
				Hopper model	89	16	24
Plain, concurrency 1 vs 32	40/40	0	0	conservative model	3	15	22
				Hopper model	14	17	9

Table 14. How the first token divergence arises, under the two named models of the stock kernel’s FP32 accumulation error: the conservative model ($\gamma = 6.11 \times 10^{-4}$), which covers any reduction order, and the Hopper model ($\gamma = 1.193 \times 10^{-4}$), Khattak and Mikaitis’s blocked wgmma model with a split- K allowance (Khattak & Mikaitis, 2026). The tie-rule and head-GEMM columns count divergences caused by the argmax’s tie rule on bitwise-equal logits and by the head GEMM on bitwise-equal inputs; they do not depend on the model. The plain-versus-MTP prompts are the 167 that diverged in the concurrency comparison, generated to two tokens past their known divergence; the 40 concurrency prompts are the 16 whose divergence was not an exact tie and 24 of the 151 that were, drawn at random. Measured [E23].

its difference on the same server comes from what the prefix cache already held, and the cache-hashing tap places it in the recurrent prefill (below). A fresh server at concurrency 32 reproduced 296 of the 320 outputs; the other 24 diverge at exact ties, 0.35 per 1,000 compared tokens [E24]. The two sessions differed in request arrival, and with it batch composition, and also in their pools: 97,672 against 133,885 attention-cache tokens and 122 against 167 recurrent-state slots [E27]. Plain decoding at batch 1 has an exact BF16 tie between its top two logits at 11.3 of every 1,000 positions and a nonzero gap of at most 0.125 at another 22.2, and no run committed a token that was not its own top-1 [E24]. SGLang’s deterministic mode, which replaces the default matrix multiplication with a batch-invariant one (DeepGEMM’s deterministic BF16 GEMM at the pin’s defaults on this GPU, a persistent Triton kernel otherwise) (The SGLang Team, 2025) and, with the FlashInfer backend, also switches sampling to PyTorch and disables the prefix cache, made plain decoding batch-invariant in an 8-prompt, 128-token check (concurrency 1 against 8: identical tokens and bitwise-identical log-probabilities) but not MTP speculation (2 of 8 prompts diverged). Passing the deterministic KV split to FlashInfer’s target-verify plan, as an engine patch (engine/sglang/patches/state/0002-verify-kv-split-deterministic.patch, behind an environment switch that is off by default and was on for this run), left 28 of 96 MTP prompts diverging between concurrency 1 and 32, 1.55 per 1,000 compared tokens, all at exact ties [E24]. A plausible reason, not tested, is that the draft is not batch-invariant, so accepted lengths, and with them a position’s offset inside its verify block, differ between batch sizes.

With pinned pools A second matrix pins the pools: every server is capped at 8 running requests with 49,152 attention-cache tokens and 40 recurrent-state slots, each was checked to have allocated exactly those, and every pair compares identical pools [E28, E27]. These runs replace the first matrix’s comparisons for every pair they cover. A fresh-server repeat reproduces all 320 outputs at concurrency 1 and 316 at concurrency 32, whose 4 divergences are at exact ties (0.06 per 1,000). The drop from 24 comes with both identical pools and the smaller cap, which also narrows the batch compositions, so it is not attributed to the pools alone. Plain decoding at concurrency 1 against 32 on one server diverges in 41 of 320 prompts, 0.63 per 1,000 (39 at exact ties, 2 within one BF16 spacing); with decode batches of at most 8 instead of 16, this rate is not comparable with the 3.42 above. A same-server repeat no longer reproduces every token: 17 of 320 prompts diverge at concurrency 1 (0.25 per 1,000) and 4 at concurrency 32, all at exact ties or within one spacing, where the first matrix’s larger pool gave none. No prefill in any pass reused a cached prefix, and one pass computes 126,408 tokens of prompt and output, more than the 49,152-token pool, so the prefix cache evicts during the first pass. The working explanation, not checked at the level of the cache, is the history dependence below: the second pass’s requests read different earlier copies of shared prefixes, or run a recurrent prefill that depends on what the prefix cache holds, as one prompt’s same-server repeats do. With the prefix cache on, a same-server repeat is therefore not an equality reference. The pool regime alone leaves batch-1 tokens unchanged: the pinned and first-matrix plain runs at concurrency 1 agree on all 320 outputs, and one prompt’s log-probabilities differ, from output index 2 and by at most 0.24 nats. With identical pools every MTP configuration, with one, three and five draft steps at top- k 1 and the tree with three steps at top- k 2, diverges from plain decoding at 3.5 to 4.0 per 1,000 compared tokens (164 to 181 of 320 prompts) at both concurrencies, about as often as it diverges from itself between concurrency 1 and 32 (3.1 to 3.5 per 1,000). In every pinned pair each divergence is at an exact tie in one run or has both margins within one BF16 spacing, or within 0.25 nats, two spacings, for 11 events in 5 prompts; none falls outside these classes, no run committed a token that was not its own top-1, and log-probability drift without a token change is at most 0.62 nats [E28].

Open questions The remaining pinned pairs (the prefix cache and the overlap scheduler off, deterministic mode and the FP32 head, each for plain decoding and MTP with three steps; the log-probabilities-off control; retraction at concurrency 32; a fresh-server repeat of MTP; and the FlashInfer and ReplaySSM decode paths of the recurrent state) and the cache-level check of the concurrency pair are [pending D11].

History dependence through the prefix cache Five prompts differ between the tapped and untapped runs, whose servers had different pools [E25, E27]. In all five the tapped run first differs from the untapped one in log-probabilities at output index 2, and three of them change tokens, at near ties: at the first changed token the top-two gap is 0 or 0.125 in both runs (tap_check.json). Untapped runs record no module outputs, so the module-level signature comes from two tapped sessions with different tap versions, which served the same 167 prompts in the same order at batch 1 and also differ for four of the five, in plain decoding and with MTP at three steps (tap_signature.json). Their pools differ as well, 93,742 against 159,322 attention-cache tokens and 118 against 199 recurrent-state slots for plain decoding, and 84,003 against 166,522 tokens and 55 against 97 slots for MTP; the attention cache never filled in either plain session. In all eight comparisons the first differing module output is layer 3’s attention, at output index 2 for plain decoding and at a verify step at output index 3 or 5 for MTP, while its QKV projection and every earlier module output that both tap versions hash agree, which at the same batch shape points to different cached KV. The comparison leaves out the GDN convolution, recurrence and attention outputs, the gated norm and the logits, which the two versions do not hash comparably, so one of those could differ first.

At the pin, after a request’s prefill, SGLang inserts its prefix into the prefix cache, matches it again and repoints the request’s KV slots to the tree’s copies, freeing its own. For tokens the tree already held, such as a chat-template prefix shared with an earlier request, the request then attends over the earlier request’s copy of that KV, valid values for the same tokens at the same positions but computed in a prefill of another shape and so not necessarily bitwise equal; under the overlap scheduler decode step 1 is already in flight with the request’s own KV, so the switch shows from output index 2 (code reading at the pin). No prefill in the tapped sessions or in the test below reused a cached prefix, and by the code this hybrid cache reuses a prefix only at a stored recurrent state, which these short shared prefixes did not have, so the repoint is the only way one request’s KV reaches another.

A deterministic test on stock SGLang serves each of 12 prompts on an empty cache and again right after the earlier prompt that shares the longest prefix with it [E29]; the 12 include all five tap-changed prompts. With the prefix cache on, plain decoding and MTP with three steps keep every token but change log-probabilities for 2 of the 12 prompts, mt_bench-0056 and humaneval-0008, from output index 2 (and 5 for one MTP prompt); for plain decoding with the prefix cache off all 12 are identical in tokens and log-probabilities. So with the prefix cache on, a request’s output at a fixed configuration and batch shape can depend on which earlier request computed its shared prefix; that is shown for these two prompts. The cache-hashing tap confirms the repoint for both at the level of the cache [E26]. Each prompt was served alone and right after its predecessor, on two fresh servers whose pools differ by at most 30 attention-cache tokens. In both, the first differing module output is layer 3’s attention at output index 2; the caches entering decode step 1 are identical, and the keys and values of all 8 attention layers entering step 2 differ at cached positions 3 to 5 for mt_bench-0056, whose shared prefix is 6 tokens, and 3 to 21 for humaneval-0008, with 22 shared tokens, while positions 0 to 2 are bitwise equal in both copies. The other four tap-changed prompts do not change in this test, under plain decoding or MTP. For them attention over a different copy of the cached KV is still the likely mechanism, because of the signature at the same batch shape, but what decides which copy a decode step reads in the two tapped sessions is unknown: the sessions differ in tap version, which changes the host time per forward, in how many tokens the earlier requests generated, and in their pools. One untested candidate is ordering: the repoint is a write issued from the host while the next decode step is already queued, so the step from which the switch shows could depend on timing. The values involved are valid by the code path, no token changed in the history test, and where the tap changed tokens it changed only near-tie decisions; we do not consider it state corruption. The cache-level check of these four prompts and the prefix-cache-off control for MTP are [pending D11].

History dependence through the recurrent prefill The cache-hashing tap also found a second route, separate from the repoint [E26]. humaneval-0044, the 128-token prompt whose log-probabilities differed between same-server repeats in the first matrix, was served five times on one server. With the prefix cache flushed between requests, all 4 repeats are bitwise identical to the first. Without a flush, the second request again prefills all 128 tokens, and its module outputs differ from prompt position 1 on, starting at layer 0’s GDN recurrence, with different log-probabilities but the same tokens; the third to fifth reuse a 64-token cached prefix, prefill the other 64, differ from position 64 on and change tokens. That prompt’s recurrent prefill therefore depends on what the prefix cache already holds, and the difference shows from the first output token. A related comparison, plain decoding at batch 1 with the prefix cache on and off on 40 prompts, on two servers with

different pools (159,324 against 447,027 attention-cache tokens and 199 against 16 recurrent-state slots) [E27], is identical throughout for the 34 prompts of at most 63 tokens. The 6 prompts longer than one 64-token chunk of the recurrent prefill differ from prompt position 1, at layer 0’s GDN recurrence, before any cache is read, and their log-probabilities differ from output index 0; two of them change a token, each at an exact BF16 tie in one run. Reading the code, the likely cause is that with the prefix cache on, the prefill kernel also tracks recurrent states for checkpoints; that has not been verified. This is not a question of which copy of the KV a request reads, but it means that a prefix-cache-off control can differ from the prefix-cache-on run from the first output token, as all 6 such prompts tested did.

Consequences A certified head is compared with the stock head at the same batch shape and in the same serving configuration (Section 2.5). And the reference for a speculative arm is stock speculative verification at the same shape, not plain decoding: greedy speculative verification is exact in real arithmetic, but its verify pass runs different kernels from plain decoding, and on every prompt analysed with MTP their outputs already differ at layer 0’s recurrence in the first speculative cycle. The DFlash capture is consistent with the same effect at the level of whole outputs: only 39% of its prompts produced output identical to plain decoding (Appendix C.5), though its divergences have not been traced to a kernel [pending D25]. The noise floor applies to fixed-noise sampling in the same way (Section 4.3).

D.5 Recurrent state under speculation

Qwen’s hybrid architecture makes state correctness part of verification. Reusable state includes attention KV, recurrent tensors, convolution history, positions and backend metadata. With state orientation $S \in \mathbb{R}^{d_v \times d_k}$, the gated-delta update can be written

$$S_t = \alpha_t S_{t-1} + u_t k_t^\top, \quad u_t = \beta_t (v_t - \alpha_t S_{t-1} k_t), \quad (9)$$

which is the Gated DeltaNet rule $S_t = S_{t-1} \alpha_t (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$ (Yang et al., 2025; 2024) with the update vector made explicit. The update depends on the preceding state.

Proposition D.2 (Replay of valid derived records). *If (α_t, u_t, k_t) were recorded along a causal trajectory from S_0 , applying the first a records to that same S_0 reproduces S_a in exact arithmetic:*

$$S_a = \left(\prod_{j=1}^a \alpha_j \right) S_0 + \sum_{t=1}^a \left(\prod_{j=t+1}^a \alpha_j \right) u_t k_t^\top.$$

Proof. Substitute the expansion at $a - 1$ into (9); the new gate multiplies every preceding term and the new outer product adds the last summand. Induction completes the proof. $\square$

The qualification about S_0 matters: replaying a derived u_t from a different state gives the wrong result even when the token IDs agree. The exact rational reference contains a one-dimensional witness, a record made at state zero and replayed from state two, which gives three where the genuine update gives one [E19]. Replaying raw inputs with the true transition is a different operation. Regrouping the recurrence also does not preserve floating-point results (Appendix D.1); SGLang’s replay fold constrains its operation order and warp count so that it is a bitwise clone of the verify kernel’s state update (SGLang contributors, 2026a).

A cycle may emit a correction or bonus token before that token has been consumed into every cache, so the runtime must distinguish the *emitted* prefix from the *materialized* state prefix; treating both as one accepted length invites off-by-one state errors. Publication of speculative state should check that the request’s slot and epoch are still current, and memory must not be reused while any stream or captured graph can still write it. An epoch check does not by itself prevent a stale kernel from writing reused memory; ownership and delayed reuse are separate obligations.

What SGLang does, and how it is tested (H5) During verification SGLang computes the recurrent state and the convolution window after every draft position into per-position buffers and leaves the live state untouched; after acceptance a fused scatter copies the buffers of the last accepted position (for a tree, the last accepted node) into the request’s slot (code reading at the pin). The emitted prefix includes the bonus token while the materialized state does not; the bonus token is the first input of the next cycle. Prefix reuse checkpoints the recurrent state every 256 tokens of sequence length into a ping-pong pair of slots, including checkpoints that fall inside a verify cycle, and a freshly allocated slot is cleared by the next forward on the same stream, after any in-flight verification by its previous owner. The MTP draft layer is a full-attention

layer, and under greedy verification every committed token is the target’s argmax, so an error in draft-side state could lower acceptance but not change the output.

Each point where a speculative cycle touches this state is tested against a reference that isolates it. Truncation by `max_new_tokens` and stop tokens at every index inside a verify cycle must reproduce the untruncated run’s tokens on the same server. Checkpoints taken during speculative decoding, including in the cycle that finished the request, are restored for a continuation and compared with a cold prefill of the same prefix. Requests aborted mid-stream with a recurrent pool of exactly four slots force the next request onto the freed slot while the aborted request’s last verification may still be in flight. Chunked prefill at 200 and 256 tokens is compared with an unchunked prefill. And the divergences of every MTP run from plain decoding are bucketed by the previous cycle’s commit length, which fixes the draft position that cycle rejected. The first two tests are complete [E29], for MTP with three and five draft steps, top- k 1, the prefix cache, overlap scheduler and CUDA graphs on, one request in flight and the first 40 prompts. Both tests compare output token IDs with the reference prefix; log-probabilities and state are not compared. Truncation leaves every truncated run token-identical to the untruncated run’s prefix: 157 of 157 with three steps, keeping 1 to 4 tokens of the final cycle, and 240 of 240 with five steps, keeping 1 to 6. A stop token at every index of a verify cycle also leaves the output token-identical to the untruncated prefix, in 160 of 160 and 240 of 240 cases. In 120 and 200 of them the stop falls inside the draft block, so drafts after it were accepted and folded into the recurrent state before the stop was detected, and none of it changed an output token. Extending each stopped conversation with a new user turn, served warm from the restored checkpoint and cold after a flush, gives token-identical continuations in 139 of 160 and 210 of 240 cases; the rest diverge at exact ties (19 and 30) or within one BF16 step (2, both with three steps), consistent with the warm path’s different prefill computation and with the history dependence above.

The same two tests for the tree, with three steps and top- k 2, and for plain decoding also compare the top five log-probabilities [E29]. Truncation leaves 159 of 159 tree runs and 39 of 39 plain runs identical in tokens and log-probabilities to the untruncated prefix, and a stop token at every index of a cycle leaves 160 of 160 and 40 of 40 identical up to the stop, 120 of the tree cases with the stop inside the draft block. Warm against cold continuation gives 142 of 160 identical for the tree (17 exact ties, 1 within one spacing) and 33 of 40 for plain decoding (7 exact ties); plain decoding has no speculative state, so such flips occur without it, consistent with the warm path’s prefill and the prefix cache’s history.

The bucketing by rejection position uses the pinned matrix at concurrency 1 against plain decoding [E28]. At fragile positions, where plain decoding’s top-two gap is at most 0.25 nats, the divergences per fragile position after each commit length of the previous cycle, full acceptance included, lie between 0.061 and 0.098 for MTP with one, three and five steps and the tree, and no length stands out. The rates are compatible with one common rate, with chi-square $p = 0.75, 0.72, 0.54$ and 0.38 ; that is a failure to reject, not a proof of equal rates, and the test treats positions as independent although they cluster by prompt and a prompt’s positions stop at its first divergence. No prompt-clustered analysis or bound on a per-length excess rate has been done. No divergence in any of the four MTP configurations falls at a non-fragile position, where a state error would show most plainly. The first verify cycle after the prefill follows no earlier cycle and is counted apart. Its rate is higher for five steps and the tree, 7 of 40 and 6 of 33 fragile positions against 0.077 and 0.081 per fragile position later, and a Fisher exact test against all later cycles gives $p = 0.033$ and 0.049 (0.71 and 0.75 for one and three steps). That test is exploratory: it was chosen after looking at the data, it is not corrected for the four configurations and several cycle buckets examined, and the groups differ in more than state, since the first cycle follows the prefill chunk and every prompt contributes it, while later cycles count only prompts that have not yet diverged. It is a lead, not a finding. A test of it is declared before any of its data exist (experiments/state_safety/README.md, “Declared follow-up”): on 960 fresh prompts disjoint from the 320, with frozen token IDs, it compares the first cycle with later ones for five steps and the tree against plain decoding at concurrency 1, and uses the same configurations at concurrency 1 against 32, which share the early exit of high-hazard prompts but do not compare verification with plain decoding, as a selection control. A first-cycle excess specific to speculation counts as supported only if the one-sided 95% bootstrap lower bounds of both the pooled primary log odds ratio and its excess over the control’s are above 0; otherwise the result is reported as inconclusive, not as evidence of no effect. Runs that fail the declared provenance, completeness or configuration checks are void, as is the analysis if every prompt is excluded, and a void result reports no statistic [pending D13]. An effect that also differs between concurrency 1 and 32 would cancel in the control, and a supported result could be a benign difference in numerical path rather than a state error. A check of the recurrent state handed from the prefill to the first verify forward depends on its outcome. For three steps at batch 1 the cache-hashing tap already finds the caches entering the first verify forward, recurrent state included, identical to those entering plain decoding’s first decode step, on 40 prompts (Appendix D.4); five steps and the tree were not checked. Checkpoint reuse, aborts with slot reuse, chunked prefill and run-to-run repeats are [pending D12].

D.6 Engine code paths read at the pin

The statements about the stock engine in this paper come from reading SGLang at the pinned commit (Appendix G.5), a frozen research pin rather than the latest revision. Table 15 lists the code paths that decide them and where a certified head would attach; the integration itself is not part of this paper’s evidence.

Path under python/sglang/	Observation
srt/layers/logits_processor.py	<code>_compute_lm_head</code> is the single hook through which plain decoding, the MTP drafter and the verifier compute logits: <code>torch.matmul</code> with BF16 output by default, <code>torch.mm(..., out_dtype=float32)</code> under <code>--enable-fp32-lm-head</code> . A certified head would replace this call for admitted consumers only; log-probability requests and unsupported transforms keep the ordinary path.
srt/models/qwen3_5_text.py	The Qwen3.5 head is tied to <code>embed_tokens</code> , which is not a <code>ParallelLMHead</code> , so SGLang’s quantized-head paths do not apply; a low-precision copy must be separate, and the BF16 head stays resident for re-scoring and fallback.
srt/models/qwen3_5_mtp.py	<code>set_embed_and_head</code> skips the head when embeddings are tied, so a reduced draft head from <code>--speculative-token-map</code> does not reach the MTP drafter while indices are still remapped (Appendix E.6).
srt/layers/sampler.py	Unseeded sampling is an exponential race; seeded sampling over the full vocabulary is a Gumbel race keyed by (seed, position, token) in FP64 (<code>multinomial_with_seed</code>), and the certified race uses the same field. The seeded top- k and top- p path keys the same hash by position in the sorted probabilities instead, and the sampler refuses min- p with a seed (SGLang contributors, 2026d).
srt/models/dflash.py	DFlash projects draft hidden states through the target head for its candidates; this is where transport’s draft-side summaries would be produced.
kernels/ops/attention/fla/gdn_replayssm_spec_fold.py	The replay fold is written as a bitwise clone of the verify kernel and constrains operation order and warp count; its <code>accept_lens</code> include the bonus token in that backend’s indexing (SGLang contributors, 2026a).

Table 15. Code paths of the stock engine behind the paper’s statements about it, read at the pin; observations, not measurements.

E WHERE THE TIME GOES, AND THE SERVING STACK

A component that occupies a fraction f of a step can speed the step up by at most $1/(1 - f)$, so the head’s share of the time bounds what any head certificate can deliver, and the other shares say where the rest of the stack leaves room. This appendix measures where the time of a decode step and of a speculative cycle goes, derives what each lever can deliver under a stated execution model, reports the first lever measurements and the proposals for the recurrent state (P4, P5), and defines the serving protocol against which every served result is to be measured.

The yardstick for every lever is useful progress per unit time. Let a cycle commit R output tokens in elapsed time $T > 0$. Under a stationary regenerative model with finite expectations the long-run rate is

$$\lambda = \frac{\mathbb{E}R}{\mathbb{E}T}, \quad (10)$$

the ratio of accumulated reward to accumulated time, not $\mathbb{E}[R/T]$, and not a queueing-latency model. A change improves the rate exactly when $\mathbb{E}R_1/\mathbb{E}R_0 > \mathbb{E}T_1/\mathbb{E}T_0$. Higher acceptance does not compensate for arbitrary overhead, and lower acceptance can be worthwhile when the saved work exceeds the lost progress. The denominator includes drafting, verification, the head, refinement, completion, state commit, non-overlapped dispatch and control. A fixed-width target pass evaluates all admitted positions even when the first proposal is rejected, and overlapped kernels require a critical-path measurement rather than a sum of kernel durations. If positive intervals enclosed the expected progress and time of a candidate and a baseline, $R_1^-/T_1^+ > R_0^+/T_0^-$ (with $T_0^- > 0$) would certify $\lambda_1 > \lambda_0$; benchmark confidence intervals are not deterministic enclosures, so the inequality says what must be estimated rather than certifying a speedup.

E.1 Bytes per step

Decode on this GPU streams bytes, so the byte counts of Table 16 give a first picture before any trace. A plain decode step streams every decoder weight once plus the whole head, while the embedding lookup reads one row per request, so the head is 15.1% of the weight bytes of a plain step. An MTP draft step runs one small layer followed by the same head, so there the

head is 84.1% of the weight bytes. The recurrent state is per request: the configuration stores it in FP32 (`mamba_ssm_dtype`), so each request holds 50.3 MB of GDN state, and a decode kernel that reads and writes it once per step moves 100.7 MB per request per step, which equals the weight bytes of a step at a batch of 84 requests.

Component	Bytes	Derivation
Tied head (and input embedding)	1.271 GB	$248,320 \times 2,560 \times 2$ (BF16)
Decoder weights other than the embedding	7.140 GB	sum of tensor sizes in the checkpoint
Weights streamed per plain decode step	8.411 GB	decoder weights plus the head; head share 15.1%
MTP layer	0.241 GB	sum of tensor sizes; a draft step streams it and the head (head share 84.1%)
Final decoder layer’s MLP	0.142 GB	$3 \times 2,560 \times 9,216 \times 2$
Vision tower (loaded, unused)	0.667 GB	sum of tensor sizes
GDN recurrent state per request	50.3 MB	$24 \times 32 \times 128 \times 128 \times 4$ (FP32)
Attention KV per context token	32 KiB	$8 \times 4 \times 256 \times 2 \times 2$ (BF16 K and V)

Table 16. Byte counts for Qwen3.5-4B at revision 851bf6e806ef, derived from the checkpoint’s safetensors headers and `config.json` (Qwen, 2026b); calculations, not measurements.

E.2 Measured attribution

Method The attribution uses the production configuration (CUDA graphs for prefill and decode, the overlap scheduler, FlashInfer attention, greedy decoding; Appendix G.5) with plain decoding at batch sizes 1, 8, 32 and 128 and MTP speculation (three steps, top- k 1, four draft tokens, otherwise default flags, not tuned) at 1, 8 and 32. For each batch size B the client holds B streaming requests with chat-templated essay prompts of 26–36 tokens and a budget large enough that none finishes, and a two-second window is profiled once decode has settled, so the batch is exactly B and no prefill falls inside the window; mean contexts in the plain windows were 987, 862, 688 and 418 tokens. Nsight Systems (NVIDIA, 2026c) traces each window with node-level CUDA-graph tracing. Each step is reassembled from the correlation identifiers of its graph replays: a plain step runs from one target-graph replay to the next, and a speculative cycle from one draft-graph replay to the next. Kernels are grouped by name, GEMMs are labelled by their neighbours in the replay, and the labels were checked structurally against the per-step counts the model implies (32 MLP gate/up and down, 48 GDN input and 24 output projections, 8 attention projections of each kind and one head per replay). Where kernels on two streams overlap, each instant is split equally among them, so the categories and the idle time sum exactly to the step [E30]. Every configuration was also measured with no profiler attached, three windows each, which gives the throughput numbers: plain decoding produced 289, 2,048, 6,298 and 14,388 output tokens per second at $B = 1, 8, 32$ and 128 (3.46, 3.91, 5.08 and 8.90 ms per step), with a standard deviation of 0.1–0.4% of the mean across windows. The collected plain steps are within 2.3% of the uncollected ones, so their kernel shares are reported directly. The bandwidth reference is measured on this GPU: the best of nine launch configurations of a read kernel sustains 3.83 TB/s over 4 GiB (the nine span 3.59–3.83 TB/s), the same configuration reads exactly the head’s 1.27 GB at 3.79 TB/s, and the attribution uses the latter as the peak [E32].

Share of a plain decode step	$B = 1$	$B = 8$	$B = 32$	$B = 128$
Step under the profiler (μ s)	3,541	3,982	5,128	8,886
Head chain (head GEMM, FP32 copy of its logits, argmax)	10.3%	9.4%	7.8%	6.1%
Weight GEMMs (GDN, attention and MLP projections)	68.0%	64.5%	54.0%	34.5%
GDN recurrent kernel	3.5%	6.8%	18.0%	41.6%
Other GDN kernels (fused convolution, gated norm, state tracking)	2.6%	2.7%	3.3%	3.8%
Attention kernels, QK norm and RoPE, gate, KV store	4.8%	5.5%	7.2%	8.5%
Norms, activation, small kernels and copies	6.5%	7.1%	6.7%	3.8%
GPU idle, inside and outside graph replays	4.3%	4.0%	3.0%	1.7%

Table 17. Where a plain decode step goes, from Nsight Systems windows of 227–572 steps at the stated batch sizes, CUDA graphs and the overlap scheduler on, greedy decoding. Measured [E30].

Plain decoding Table 17 gives the attribution. The head chain takes 10.3% of a step at $B = 1$ and 6.1% at $B = 128$ [E31], below the head’s 15.1% share of the weight bytes, for two reasons visible in the table. The head GEMM is the most efficient kernel in the step: it streams the 1.27 GB weight at 3.3–3.6 TB/s, while the backbone’s weight GEMMs reach 1.6–3.2 TB/s, the split- K projections from 4,096 to 2,560 features being the slowest; if every weight GEMM ran at the head GEMM’s

rate the step would be 469 μ s ($B = 1$) to 923 μ s ($B = 128$) shorter. And the step contains work that does not scale with the weights: the recurrent state, attention, 65 RMSNorm launches and idle time. The recurrent kernel’s time is linear in the batch, 28.9 μ s per request per step, which implies 3.47–3.48 TB/s at $B = 32$ and 128: it streams the state once each way at 92% of the read peak. In bytes the state equals the weights at $B = 84$; in time the crossover comes later, near $B = 110$ –120, because the weight GEMMs run below the peak while the state kernel runs near it, and at $B = 128$ the recurrent kernel alone (3.70 ms of wall time) already takes longer than all weight GEMMs together, head included (3.47 ms) [E30, E33]. The host is not on the critical path: each graph launch is issued 1.7 ms ($B = 1$) to 5.9 ms ($B = 128$) before the GPU starts it, and the GPU is busy 95.7–98.3% of the step. The margin is finite, though. At $B = 1$ the scheduler thread spends only 0.73 ms of a 3.54 ms step blocked on the GPU, so a GPU-side speedup much above 1.25 $\times$ at batch one would make the host the bottleneck.

The stock head in isolation Replaying the exact calls SGLang makes with the real tied weight under CUDA graphs, the head GEMM takes 354.5 μ s at $M = 1$, 362.0 μ s at $M = 32$ and 569.2 μ s at $M = 256$; up to $M = 32$ it runs at 93–95% of the measured read bandwidth, and reading the BF16 weight once at that bandwidth takes 335 μ s. cuBLAS selects an `nvjet_sm90_tst` kernel whose tile depends on M . At $M \geq 64$ the FP32 copy and the argmax, each re-reading an $M \times 248,320$ tensor, add 22–50% on top of the GEMM (the whole chain takes 558.6 μ s at $M = 128$ and 850.6 μ s at $M = 256$). Cold L2 changes the GEMM by under 1 μ s up to $M = 128$ (5 μ s at $M = 256$), since the weight is 21 times the 60 MB L2 [E31]. This chain is the comparator a certified head must beat, and a kernel that reads the same BF16 bytes has at most a few percent to gain at small M .

MTP speculation, three draft steps	$B = 1$	$B = 8$	$B = 32$
Cycle under the profiler (μ s)	8,407	9,833	13,886
Cycle without a profiler (ms, mean of 3 windows)	6.70	8.27	12.49
Output tokens/s without a profiler (plain decoding at the same B)	384 (289)	2,573 (2,048)	7,087 (6,298)
Accepted length, tokens per cycle (windows without a profiler)	2.57	2.66	2.77
Target head chain (verify GEMM over 4B rows, FP32 copy, argmax)	4.4%	4.1%	3.9%
Draft head chains (three draft GEMMs, FP32 copies, top-1, extend argmax)	12.9%	11.3%	8.5%
MTP layer, excluding its head	4.6%	4.4%	3.7%
Target weight GEMMs	29.6%	28.4%	22.2%
GDN verify kernel (writes four FP32 states per request)	1.4%	5.5%	18.3%
GDN state commit (scatter of the accepted state)	0.5%	3.2%	8.9%
All other kernels (remainder)	9.3%	10.4%	11.0%
GPU idle, inside and outside graph replays	37.3%	32.7%	23.5%

Table 18. Where a speculative cycle goes with the native MTP layer (untuned: three steps, top- k 1, four draft tokens). Shares are of the cycle under the profiler; the idle share is inflated by the profiler, which lengthens this host-bound cycle by 25%, 19% and 11% at $B = 1$, 8 and 32 (first two rows). Measured [E30, E31].

MTP speculation A cycle replays a draft graph (the MTP layer twice, each time followed by the full head, the FP32 copy and a split-vocabulary top-1), a target-verify graph over four tokens per request, and a draft-extend graph (the MTP layer over the verified tokens and the head once more), with verification and the recurrent-state commit running eagerly between them. Every cycle therefore streams the 1.27 GB head four times. Table 18 gives the attribution. The head takes 17.2%, 15.3% and 12.4% of a cycle at $B = 1$, 8 and 32, three quarters of it in the draft heads; because much of the cycle is idle, its share of the time the GPU is busy is larger, 27.5% at $B = 1$ [E31]. The idle time has one main cause [E34]. Outside graph replays the GPU waits about 3 ms per profiled cycle at every batch size, and the draft and verify graphs are launched only 44–46 μ s and 380–411 μ s before the GPU reaches them. NVTX ranges on the speculative worker’s host functions and samples of the scheduler put 1.2–1.5 ms of exposed host time per cycle in the attention planning for the verify pass (FlashInfer’s prefill planner, reached through the verify preparation and the graph runner’s batch loading, with blocking device-to-host copies), of which only 14% overlaps GPU work at $B = 1$, and another 0.5 ms in the draft pass’s planning; the host work of the draft-extend pass is 97% hidden behind the verify graph. The profiler inflates this gap, chiefly because each graph launch costs about 350 μ s under node-level tracing; subtracting the traced GPU-busy time from the cycle measured without a profiler leaves about 1.4, 1.7 and 1.9 ms per cycle idle (21%, 20% and 15%; derived from two measurements). Without a profiler, MTP produced 1.33 $\times$, 1.26 $\times$ and 1.13 $\times$ the plain throughput at $B = 1$, 8 and 32 in these fixed-batch windows, which contain no prefill.

What the attribution decides Other work finds the head a large share of drafting: FR-Spec profiles EAGLE-2 drafting on Llama-3-8B, with a 128k vocabulary, and finds the head projection at 49% of draft time and the softmax over its output at a further 13% (Zhao et al., 2025), a trimmed-vocabulary study finds the head at 64% of an EAGLE-3-style drafter’s FLOPs (Ben Shoham, 2026), and vocabulary sizes have grown with compute budgets (Tao et al., 2024). On this model and GPU the head is a smaller share of the time than of the bytes (H1): removing the head chain entirely is worth at most $1.11\times$ for plain decoding at $B = 1$, $1.07\times$ at $B = 128$, and $1.21\times$ for the MTP cycle at $B = 1$, before any cost of the certificate. The other layers of the stack leave more headroom (H6), and Table 19 ranks them. These ceilings are Amdahl bounds for removing a component under otherwise unchanged execution, not predicted gains.

Cost	Configuration	f	Ceiling
FP32 recurrent state read and written every step	plain, $B = 128$ ($B = 32$)	41.6% (18.0%)	$1.71\times$ ($1.22\times$)
GPU idle in the cycle, 35.5 of its 37.3 points in host-bound gaps outside the graph replays (attention planning)	MTP, $B = 1$	37.3% ($\approx 21\%$)	$1.59\times$ ($\approx 1.27\times$)
GDN verify kernel and state commit, of which only the per-position FP32 state writes and the commit copy are removable	MTP, $B = 32$	27.2%	$1.37\times$ (whole)
Weight GEMMs below the head GEMM’s bandwidth (excess time only)	plain, $B = 32$ ($B = 1$)	15.5% (13.2%)	$1.18\times$ ($1.15\times$)
Three full-vocabulary draft heads per cycle	MTP, $B = 1$ ($B = 32$)	12.9% (8.5%)	$1.15\times$ ($1.09\times$)
The target head chain	plain, $B = 1$ ($B = 128$)	10.3% (6.1%)	$1.11\times$ ($1.07\times$)
65 RMSNorm launches per step	plain, $B = 8$	4.8%	$1.05\times$
Gaps between graph nodes	plain, $B = 1$	3.0%	$1.03\times$
Eager sampling and bookkeeping after each replay	plain, $B = 1$	2.6%	$1.03\times$

Table 19. Measured costs and their Amdahl ceilings $1/(1 - f)$, with f the share of the step or cycle in the stated configuration. For the idle time the parenthesized values are the share derived without the profiler and its ceiling. Measured [E30, E34].

E.3 Ceilings under a stated execution model

A decode step streams the linear weights once whatever the batch size, $W = 8.41$ GB, and everything else is per request: 100.7 MB of FP32 recurrent state read and written, and 32 KiB of attention KV per context token (Table 16). With F the floating-point operations per token (two per weight), s the per-request bytes per step, BW the measured HBM read bandwidth and P an achievable tensor-core rate, and with each layer’s weight GEMMs and its per-request kernels running in turn, as they do at the pin, the step time at batch B is bounded below by

$$t(B) \geq \max\left(\frac{W}{BW}, \frac{BF}{P}\right) + \frac{Bs}{BW}, \quad (11)$$

since in that order the weight stream and the GEMM arithmetic can overlap each other while the per-request traffic adds to them. An engine that overlapped the memory-bound per-request kernels with the batch GEMMs would be bounded only by

$$t(B) \geq \max\left(\frac{W + Bs}{BW}, \frac{BF}{P}\right), \quad (12)$$

whose large-batch limit is $\min(BW/s, P/F)$. Equation (11) models the engine as it runs; Equation (12) is the overlapped roofline at the same inputs, for an engine that splits the batch to overlap the two kinds of kernel. Neither is a bound for every engine on this machine. The bandwidth input is the 3.79 TB/s measured over the head’s footprint [E32], and P is assumed to be 70% of the BF16 and 60% of the FP8 datasheet peaks, assumptions stated so that the ceilings can be recomputed. Two consequences follow. Under Equation (11), as B grows the throughput ceiling approaches $1/(F/P + s/BW)$, set by the per-request state and the arithmetic rather than by the weights, so halving the state’s bytes or writing it less often raises it while a faster head does not; and even with every byte removed, the arithmetic term caps an FP8 engine, so no single lever gives an order of magnitude at high concurrency. At batch one the weights bound the step, and the draft head and the target’s weight bytes are the terms to cut.

Table 20 and Figure 7 evaluate the floor for stacks of levers [E36]; they are derived, not measured. At batch one the floor is 2.25 ms per token (445 tokens/s), of which the weights alone account for 2.22 ms; plain decoding measured 280 tokens/s at concurrency one in a single served run [E37], 63% of that rate, and 289 tokens/s in the profile’s fixed-batch windows [E30]. At large batch the plain engine’s ceiling under Equation (11) is 23.7k tokens/s, because each request moves 114 MB per step

and needs 8.4 GFLOP. Halving the state bytes, writing the state less often and moving weights and cache to FP8 raise the ceiling to between 34k and 79k tokens/s. The overlapped roofline is higher, 32.1k tokens/s for the plain engine and 141.2k for the full lossy stack at $B = 2,048$ (the plain engine’s limit as B grows is 33.3k), but under neither model does any stack reach ten times the plain engine’s ceiling: 8.4 GFLOP per token caps an FP8 engine near 140k tokens/s at the assumed rate even with every byte removed (about 235k at the datasheet FP8 peak, of which the assumed rate is 60%; derived from the same constants).

Lever stack	MB per request per step	Tokens/s at $B = 2,048$	
		layer-serial	overlapped
Plain, FP32 state	114.0	23.7k	32.1k
16-bit state	63.6	34.6k	55.9k
Replayed state writes, FP32	66.8	33.6k	53.5k
Replayed state writes, 16-bit state	40.0	44.0k	82.3k
Replayed state writes, int8 state	26.7	52.1k	82.3k
Replayed, 16-bit state, FP8 weights and KV	34.6	61.7k	102.6k
Replayed, int8 state, FP8 weights, KV and head	21.2	78.9k	141.2k
MTP (accepted length 3), stock verify, FP32	—	19.5k	—
MTP (accepted length 3), replayed verify, FP32	—	34.3k	—

Table 20. Derived large-batch throughput ceilings under the layer-serial model of Equation (11), which assumes that per-request kernels do not overlap the weight GEMMs, and under the overlapped roofline of Equation (12), at a 334-token context, with the measured read bandwidth of 3.79 TB/s and the tensor-core rate assumed at 70% (BF16) or 60% (FP8) of the datasheet peak. The MTP rows are for the layer-serial model only. Not measurements [E36].

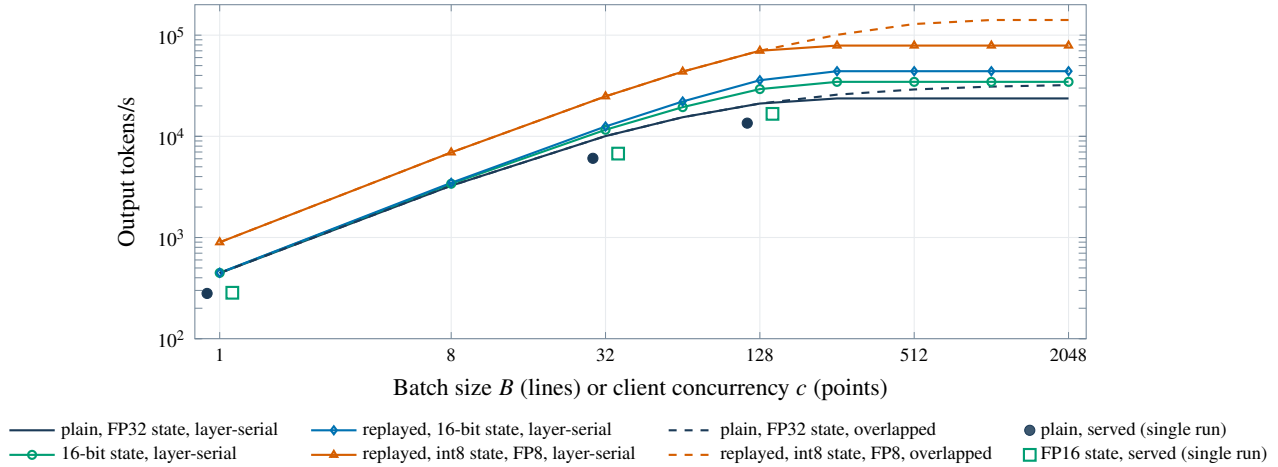


Figure 7. Derived throughput ceilings per lever stack against batch size under the layer-serial model of Equation (11) and, for two stacks, the overlapped roofline of Equation (12), at the measured bandwidth and assumed tensor-core rate (lines [E36]), and served output throughput of single benchmark runs at client concurrency 1, 32 and 128 (points [E37]; the two series are offset slightly left and right of their concurrency so that both stay visible). The gap between a line and a point at the same concurrency is overhead above the layer-serial floor; at high concurrency it includes the server’s streaming path (Appendix E.8).

Repeated engine-only and served measurements up to $B = 1,024$ for each state and weight format are [pending D14].

E.4 Speculation and the recurrent state

The hybrid architecture changes the economics of speculation. A verify pass over D draft tokens must be able to roll back to any accepted prefix, and SGLang’s Triton verify kernel does so by writing one full FP32 state per draft position into an intermediate buffer and copying the accepted one back at commit. Per request and cycle that is one state read, D state writes and a read and write at commit: 352 MB for $D = 4$, three and a half plain steps (derived from the code path and checked against the trace). The two kinds of traffic scale differently with the batch. Speculation saves weight bytes: one verify pass streams the weights once for several committed tokens, and that stream is shared by every request in the batch, so its cost

per output token falls as $1/B$. The snapshot writes are per request and per draft position, so their cost per output token does not fall with the batch at all. At batch one the weight saving dominates; at high batch, where the state dominates, speculation’s bandwidth advantage shrinks and can turn into a loss. This cost belongs to a recurrent hybrid; an attention-only model rolls back by discarding cache entries and writes no snapshots. Rollback of recurrent state is an active problem: the Mamba-in-Llama work advances one cached state lazily and recomputes after a rejection (Wang et al., 2024); STree, TreeWY, Bole, GDN Tree-Scan and SpecLA verify trees or chains on SSM and GDN hybrids (Wu et al., 2025; Ghantasala, 2026; Wang et al., 2026a; Ma, 2026; Wang et al., 2026b), and TreeWY states that its outputs are not bit-identical to the baseline; ReplaySSM caches recent inputs instead of writing the state at every step, equivalent to ordinary decoding up to floating-point error (Liou & Dao, 2026).

The measurements agree with the mechanism. At $B = 32$ the verify kernel runs for 2.54 ms and the commit for 1.23 ms per cycle, 27% of the cycle, at an implied 3.2 and 2.6 TB/s [E30]. The byte model, at a context of 700 tokens and an accepted length of 2.8, has MTP moving 44% fewer bytes than plain decoding per output token at $B = 1$, 26% fewer at $B = 32$, 6% fewer at $B = 128$ and 2% more at $B = 256$ [E33]; the crossover moves with the context, because attention KV is read once per verify pass for several committed tokens, so longer contexts favour speculation. In the profile’s fixed-batch windows MTP’s throughput advantage over plain decoding fell from $1.33\times$ at $B = 1$ to $1.13\times$ at $B = 32$ (Appendix E.2). Served single runs on the tuning split show the same turn: MTP with three steps and snapshot verification led plain decoding by $1.64\times$ at concurrency 1 and trailed it from concurrency 32, and replayed verification, which removes the snapshots, raised its throughput at concurrency 128 by 24% (Table 22). At that point the snapshots alone would be written at about 0.59 TB/s if each draft token’s 50.3 MB snapshot is written once per cycle, against 1.35 TB/s of state traffic for plain decoding at the same concurrency under the same assumption (derived from the measured throughput and accepted length) [E66]. On the confirmation split, over three sessions with matched plain baselines, every speculative family leads plain decoding through concurrency 32 and trails it from 48, with accepted length flat across concurrency (3.24 to 3.28 tokens per cycle for MTP, 4.66 to 4.81 for DFlash at block 8), so the loss is not a fall in acceptance. At concurrency 128 MTP with replayed verification delivers 0.86 of plain decoding’s throughput, and with SGLang’s snapshot verification 0.68 (one session) [E67].

Bytes do not explain the whole cycle at high concurrency [E47]. At concurrency 128 the tuning points imply an MTP cycle of 43.5 ms with snapshot verification and 35.0 ms with replayed verification (128 requests times 3.26 tokens per cycle, over the measured throughput), against 9.5 ms per plain step. The implied cycle is derived from client throughput, so it includes the prefill passes of arriving requests and the benchmark’s ramp and drain as well as the decode cycle. Derived from bytes and FLOPs at 3.8 TB/s, 650 TFLOP/s and a context of about 400 tokens, its parts add up to about 21.7 and 13.2 ms: the verify GEMMs over 512 tokens take 6.6 ms, the three draft steps about 1.7 ms, verify attention and small kernels about 1.5 ms, and the GDN state traffic 11.9 ms with snapshots (352 MB per request) or 3.4 ms with replay (about 100 MB). Replay saved about 8.5 ms per cycle, which the 252 MB per request of snapshot and commit traffic it removes accounts for, but the implied cycle is 2.0 to 2.7 times the sum of its derived parts: most of it is time the byte model does not see, in the decode cycle itself or in the prefill, ramp and drain, which this arithmetic cannot separate.

SGLang’s replay path for speculation stores each draft token’s inputs and folds the accepted prefix into the state at commit, reading and writing the state once per cycle, which would make speculation cheaper in state bytes per token than plain decoding; trees do not get this benefit yet, because the replay path handles only chains and a tree keeps one full state per node (Ghantasala, 2026; Ma, 2026). The block drafter is more exposed than the MTP layer: a 16-token DFlash block writes about 805 MB of states per request per cycle (Appendix F.1).

Verifying a draft block in parallel over its positions (P7b), instead of through the sequential recurrence, would make wide blocks cheap only with a fast path that is both faster and close enough to the recurrent outputs to be certified. SGLang’s chunked GDN kernel is neither at one request [E45]. On one layer, with synthetic activations and the checkpoint’s gates, its outputs over a block of 2 to 16 tokens differ from the packed decode’s in 52–55% of BF16 output words, and it takes 815–990 μ s per layer at block widths 4 to 256, against 75–378 μ s for the recurrent verify kernel with its per-position snapshots. A purpose-built block-parallel kernel is untested.

State precision is the cheapest lever and the most dangerous. DAMP reports reasoning accuracy of 85.5 with FP32 recurrent state, 84.6 with FP16, 79.7 with BF16 and below 30 with FP8 or int8 on another GDN model (Zhang et al., 2026b), and LeapQuant recovers near-FP32 accuracy with 8-bit state by re-quantizing once per 16-token window (Pan et al., 2026). SGLang accepts FP16 state (`--mamba-ssm-dtype float16`). MTP with and without replayed verification at concurrency 128 to 1,024, and the logit-probe KL, divergence rate and GSM8K accuracy for FP32, FP16, BF16 and FP8 state with and without replay, are [pending D15].

E.5 How much recurrent state an exact computation needs

The dense FP32 state is sufficient for exact decoding; nothing yet shows that it is necessary. Two questions should be kept apart: whether less information could be retained, and whether the state has to be materialized as often as it is.

For the first, fix one GDN head of one layer and treat the layer’s inputs as given. Along any permitted history the state evolves as $S_t = S_{t-1}A_t + B_t$ with $A_t = \alpha_t(I - \beta_t k_t k_t^\top)$ and $B_t = \beta_t v_t k_t^\top$, which depend on the inputs and not on the state, and the head reads $o_t = S_t q_t$. A state difference ΔS at time t therefore propagates linearly, to $\Delta S A_{t+1} \cdots A_{t+j}$, and is observed as $\Delta S A_{t+1} \cdots A_{t+j} q_{t+j}$.

Proposition E.1 (What an exact state must retain). *Let $\mathcal{S}$ be the set of states reachable over permitted histories, $\mathcal{D} = \text{span}\{S - S' : S, S' \in \mathcal{S}\}$, and $\mathcal{N}$ the set of differences ΔS with $\Delta S A_{t+1} \cdots A_{t+j} q_{t+j} = 0$ for every permitted continuation and every $j \geq 0$. Two reachable states produce the same outputs under every permitted continuation if and only if their difference lies in $\mathcal{N}$. If $\mathcal{S}$ contains a relatively open subset of its affine hull, then a linear map that separates every pair of reachable states with different outputs has rank at least $\dim \mathcal{D} - \dim(\mathcal{D} \cap \mathcal{N})$ on $\mathcal{D}$.*

Proof. Because B_t does not depend on the state, the output difference under a continuation is the linear expression above, and it vanishes for every continuation exactly when $\Delta S \in \mathcal{N}$. For the bound, take $d \in \mathcal{D}$ with $\phi(d) = 0$. By the hypothesis there are a reachable S and an $\varepsilon > 0$ with $S + \varepsilon d$ reachable; ϕ gives them the same image, so they must have the same outputs, and $\varepsilon d \in \mathcal{N}$. Hence $\ker \phi \cap \mathcal{D} \subseteq \mathcal{N} \cap \mathcal{D}$, and the rank of ϕ on $\mathcal{D}$ is at least $\dim \mathcal{D} - \dim(\mathcal{D} \cap \mathcal{N})$. $\square$

The hypothesis matters: if only one state is reachable, $\dim \mathcal{D} = 0$ and a representation with no dimensions is exact. Treating every input sequence as permitted is conservative, since in the full model a layer’s outputs influence later tokens; it is the notion that makes a reduction safe for any request. Exact witnesses show that neither $\mathcal{D}$ nor $\mathcal{N}$ is favourable without structure in the keys [E22]. Writes with unit coordinate keys and gates inside $(0, 1)$ reach any matrix, so the reachable set can fill the whole state space and $\mathcal{D}$ with it. A direction the current query cannot see is not unobservable: with $\alpha = \beta = \frac{1}{2}$, $k = (\frac{3}{5}, \frac{4}{5})$, $q = e_1$ and $\Delta S = u e_2^\top$ for any value vector u , the observed difference $\Delta S q$ is zero now, but one step later it is $\Delta S A q = -\frac{3}{25}u$. Each transition is a contraction, not an erasure: $\det A_t = \alpha_t^d (1 - \beta_t \|k_t\|^2) > 0$ when $\beta_t \|k_t\|^2 < 1$, so no step forgets a direction exactly. Transitions with non-orthogonal keys do not commute, so there is no common basis in which they are diagonal. And because a gated RMSNorm precedes the output projection, the null space of the projection is not unobservable: two outputs that $W_o = (1, 0)$ cannot tell apart, $(1, 1)$ and $(1, 7)$, project to 1 and $\frac{1}{5}$ after normalization.

One structural reduction is exact. If every key lies in the column space of a fixed orthonormal $U \in \mathbb{R}^{d_k \times r}$ and $S_0 = Z_0 U^\top$, then $S_t = Z_t U^\top$ with

$$Z_t = \alpha_t Z_{t-1} + \beta_t (v_t - \alpha_t Z_{t-1} \kappa_t) \kappa_t^\top, \quad \kappa_t = U^\top k_t,$$

the same recurrence in dimension r , since $U^\top A_t (I - U U^\top) = 0$ and the write $\beta_t v_t k_t^\top$ stays in the form exactly when $k_t \in \text{col}(U)$. The keys are produced by a short convolution, a SiLU and a normalization, and SiLU raises rank (SiLU(log 2 [[1, 2], [2, 4]]) has rank two), so a low-dimensional input does not imply low-dimensional keys; whether real keys are confined to a subspace is a measurement, to be certified by exact or modular rank rather than an SVD threshold [pending D16]. With U fixed per layer and head and shared across requests, the per-request state is $d_v r$ values instead of $d_v d_k$, a saving for every $r < d_k$; a per-request low-rank factorization $S = AB^\top$ instead stores $d_v r + r d_k$ values, which beats the dense state only below $r = 64$ at $d_v = d_k = 128$ and halves it only at $r \leq 32$ (derived). Floating point adds a further obstruction: an exact rational reduction does not preserve bitwise results. The split form of the recurrence changed 1,481 of 1,600 FP32 outputs (Appendix D.1), deferring a decay reassociates FP32 roundings (one ulp apart for $\alpha_1 = \alpha_2 = 0.9$, $x = 1.3$), and rounding an exactly rank-one BF16 outer product to BF16 gives a 32×32 matrix of exact rank 28 [E22].

Errors that do enter the state stay bounded as long as each transition is non-expansive: $(I - \beta k k^\top)$ has spectral norm at most 1 exactly when $0 \leq \beta \|k\|^2 \leq 2$ (its eigenvalues are 1 and $1 - \beta \|k\|^2$), so a perturbation η_t injected at each step obeys $\|E_t\| \leq |\alpha_t| \|E_{t-1}\| + \|\eta_t\|$; with normalized keys and $\beta \in (0, 1)$ the condition always holds [E21].

The second question has a better answer. A decoder that keeps a dense anchor and the recent actual FP32 operands of each step, and replays them in the reference kernel’s order, reproduces every state word bit for bit by induction, while reading the dense state once per step and writing it once every m steps. That reduces information movement, not information, and it is what replay-based rollback already does (Liou & Dao, 2026; SGLang contributors, 2026a). The paragraphs below treat it as a lever with a pre-registered test (P4), and the opposite idea, sharing exact computation across requests (P5).

Writing the recurrent state less often (P4, hypothesis) Rounding-preserving live replay is exact and cuts dense-state traffic per step from $2D$ (read and write) to $D + D/m$ when the state is flushed every m steps, 0.625 of the original at $m = 4$, for a few percent more storage (derived). If the recurrent kernel is a fraction f of a decode step, a kernel twice as fast gives at most $1/(1 - f/2)$, removing it entirely $1/(1 - f)$, and the $m = 4$ traffic cut alone $1/(1 - 0.375f)$. The measured share at $B = 128$ and a 418-token mean context is $f = 0.416$ (Table 17), which gives 1.26 $\times$, 1.71 $\times$ and 1.18 $\times$ (derived from a measured share). The pre-registered test has two parts. A kernel check compares every state word and GDN output of the replayed and dense paths, one layer at a time, and one differing word rejects bit-exactness (below). The served timing decodes 512 tokens greedily at batch 128 from 2,048-token prompts with FP32 state and no speculation, with the threshold at 1.10 $\times$; through the server only the output probe below checks exactness. The served timing’s design, metric and decision rule were declared before its first run, and every later change before the run it governs [E41]. Dense decoding and replay at $m = 4$ run as four server pairs in the order A B B A A B B A with identical memory pools. The primary metric is the server’s decode rate at exactly 128 running requests: the harmonic mean of the scheduler’s generation-throughput windows (one per 40 decode passes) that show 128 running requests on that line and the previous one, with no prefill batch between them, inside the AIPerf profiling phase, which excludes warm-up, ramp and drain; fewer than 8 such windows in any arm voids the run. It measures decode only, which is what the lever changes, and client throughput, which includes the 2,048-token prefills, is reported beside it. The four log ratios give a 95% t interval, and the claim is rejected if its upper end is below 1.10, supported if its lower end reaches 1.10, and inconclusive otherwise; a supported result is a served decode throughput at 128 running requests, not an end-to-end speedup. A server output probe compares greedy tokens and top-20 log-probabilities at concurrency 1 with dense decoding, with a second dense server as the noise control: a difference refutes end-to-end exactness unless the control also differs from the first dense server, in which case the probe is undecided; a pass does not establish exactness; and a refuted probe leaves P4’s exact claim unsupported whatever the throughput interval. A run that fails its completeness, workload, pool, environment or provenance checks is void and its numbers are not reported. One check was corrected after the first run and before any verdict was computed: the sent prompts are now matched by the hashes of their texts, because the benchmark’s request ids differ from the declared ones for this workload, a check no weaker than the declared one. Three attempts have been void, because in every arm at most 127 of the 128 requests were admitted. The amendments declared after the first two, a pinned attention cache of 655,360 tokens, 132 pinned recurrent-state slots and an admission preflight that must show 128 running requests, did not change that: in the third the attention cache was at 40% use and 127 of 132 recurrent-state slots were taken. Read from the engine source, the admission check counts a request whose last prefill chunk runs in the pass twice: the request-table row it keeps from its first chunk lowers the admission limit, and it also counts against that limit as one of the pass’s requests. A wave therefore stops one short of the running limit whenever its last pass carries a chunk tail, and the 128th request runs only after the wave finishes. Every one of the 56 admission waves in the three attempts’ server logs had such a tail and stopped at 127, as predicted [E41]. Those waves test the prediction only at that one limit and one chunk; the discriminating test is the amended admission preflight. Before any further run the running limit was raised to 129, with the client still at 128 concurrent requests, so that at most 128 requests are ever in the server and the prediction is a peak of 128 running in both arms. That preflight is queued. No served result exists. SGLang’s own replay path qualifies only if it passes the bit-exactness check. The replay kernel is committed as an engine patch with a one-layer test that compares every output and state word against the stock decode kernel (tests/test_gdn_exact_replay.py [E39]).

P4 at kernel level The replay is bit-exact at kernel level [E40]. Run side by side with SGLang’s packed decode for 48 steps at batch 8, on synthetic activations with the checkpoint’s gates, with rows flushing at staggered phases and the reconstructed state written out and compared at every step, it differs in none of 201,326,592 state words and none of 1,572,864 output words, at $m = 4$ and 16 on the first GDN layer and at $m = 4$ on layer 20. SGLang’s own replay path differs on the same inputs in 465,102 and 543,225 of the 1,572,864 output words (30% and 35%, layers 0 and 20), so it does not qualify. Timed on one layer under CUDA-graph replay with a cold L2, the kernel is 1.215 $\times$ faster than the packed decode at batch 128 and 1.227 $\times$ at batch 256 with $m = 4$, not the 1.6 $\times$ the bytes allow: at batch 128 it takes 89, 112 and 129 μ s on steps that replay zero, one and two earlier updates and 201 μ s on the step that replays three and writes the anchor, against 161.5 μ s for the packed decode; with $m = 8$ and 16 it is no faster than the packed decode. With the recurrent kernel at 41.6% of a batch-128 step, a kernel 1.22 $\times$ faster predicts about 1.08 $\times$ for the decode step, and less with the pre-registered 2,048-token prompts, where attention takes a larger share: below the 1.10 $\times$ the test requires (derived). The check covers one layer at a time on synthetic activations; the server output probe and the served paired timing are [pending D17].

Sharing computation across requests (P5, rejected) Factoring hidden or recurrent state across unrelated requests does not survive analysis. An exact factorization with a dense residual costs the full projection, so it saves only if $r(d+B)+z < Bd$ with a zero or sparse residual; SiLU and gating raise rank; full-rank pairwise state differences force private factors at least as

large as the dense state; one gated-delta step takes four identical states to batch rank four [E22]; online factorization costs of order B^2 times the update; and reassociation breaks equality with the reference. The one exact candidate is structural: the first GDN layer’s input projections (12,352 outputs) depend only on the current token, so they could be read from a precomputed per-token table (5.71 GiB for the whole vocabulary) with convolution and recurrence kept private. Its derived ceiling is about 0.75% of the dense projection work per token, and at batch 128 even a 99% table hit rate would leave about 72% of batches with a miss. The pre-registered criterion rejects the table unless the stage’s measured share of a decode step allows a 1% end-to-end gain. It does not: layer 0’s input projections take 0.60%, 0.57%, 0.51% and 0.33% of a plain step at $B = 1, 8, 32$ and 128, a ceiling of $1.006\times$ [E35], so P5 is rejected by its criterion as well as in analysis.

E.6 Levers by axis

Table 21 lists the levers, each built as a separate server flag or environment variable so that any subset composes. A lever is *exact* when it leaves the target’s arithmetic unchanged, so every emitted token is the target’s decision at its own batch shape under the stock-kernel contract, with its measured noise floor; drafting levers are exact because the target verifies every token. A lever that changes the target’s arithmetic is classed by its greedy outputs against a matched stock reference (Appendix E.8): *exact up to rounding* if every first divergence is at an exact tie, within one BF16 step or a near tie within two steps, the classes of the stock engine’s own batch-shape divergences (Appendix D.4), and *lossy* otherwise, in which case it carries a measured quality cost. SGLang’s replay path for plain decoding is exact up to rounding by that comparison [E68], although it is not bit-exact: its decode kernel reconstructs the state on tensor cores from BF16-cast operands and persists that rounding at every flush (code reading at the pin), and on synthetic activations its outputs differ from the packed decode’s in 30 to 35% of words (Appendix E.5). The bit-exact alternative is the rounding-preserving replay (P4).

The first single-run measurements on the benchmark harness (plain arm with the prefix cache on, capacity 128) separate the levers by where they act [E37]. FP16 recurrent state raised served throughput at concurrency 128 from 13.5k to 16.7k tokens/s (1.24 $\times$) and changed nothing at concurrency one, which is what a per-request byte term predicts; its quality cost is [pending D15]. These runs streamed one token per chunk, and at that concurrency the streaming path rather than the GPU can limit a plain arm (Appendix E.8), so the ratio may understate the lever; a paired rerun is [pending D14]. FP8 KV did nothing at these short contexts (0.99 $\times$). FP8 W8A8 weights could not use their CUTLASS GEMM: the one in the installed aarch64 sg1-kernel aborts at run time because that binary was not compiled for sm_90a, as its logged error states (Hopper’s asynchronous warp-group MMA instructions exist only for that target (NVIDIA, 2026d; 2025)), and the Triton route showed no consistent gain (0.96–1.06 $\times$ across concurrencies), a negative result until a different GEMM route is wired. A first engine-only pass of decode steps at batch 512 took 26.9 ms with FP32 state, 23.0 ms with BF16 and 22.5 ms with FP16 [E38]; it is noisy at small batch sizes and is to be repeated. All of these are single runs without repeats.

Two levers need engine changes. Draft-vocabulary methods restrict the draft head to a subset of rows (Zhao et al., 2025; Goel et al., 2025; Williams et al., 2026; Zhang et al., 2025; Chen et al., 2026b), and FastMTP compresses only the draft vocabulary while keeping full-vocabulary verification (Cai et al., 2025); they change only the drafter, so the target’s decision is unchanged under target-only or point-mass verification. SGLang’s hot-vocabulary draft head (--speculative-token-map, after FR-Spec) does the wrong thing on Qwen3.5 as read at the pin: the MTP layer’s head is the tied embedding, and set_embed_and_head ignores the truncated head in that case, so the drafter scores all 248,320 rows while the worker remaps indices as if it had scored the hot subset; the patch unties a reduced head for the drafter only. A hot map ranked by token frequency on the tuning split’s greedy outputs covers 97.5% of the 171,968 output tokens of a plain sweep on the confirmation split with 22,936 rows (every token seen in the tuning outputs), 96.3% with 16,384 and 89.1% with 4,096 [E46]. The second change adds a relaxed greedy rule for chains, accepting a draft token whose target logit is within g of the maximum; SGLang’s existing relaxed thresholds act only when sampling, and the rule is lossy by construction, so its quality curve is the measurement (Urbán et al., 2026). Both changes are committed as engine patches with tests that run where SGLang and a GPU are available (tests/test_moonshot_levers.py [E39]); their measured effect and the relaxed rule’s quality curve are [pending D20].

The host gap of the speculative cycle at low concurrency (Appendix E.2) has a configuration-level lever [E42]. With Triton attention for the target and the draft, neither attention backend needs host-side sequence lengths, so the overlap scheduler stops copying them to the host and FlashInfer’s planning leaves the cycle. MTP with three steps then gains 1.18 $\times$ in per-request rate at concurrency 1 and 1.14 $\times$ in throughput at concurrency 4, with unchanged accepted length; Triton attention for the draft alone gains 1.08 $\times$ and 1.06 $\times$ (single runs). The draft-only change is exact, because it changes only which tokens are proposed. Triton attention for the target changes the target’s arithmetic and is exact up to rounding [E68]. SGLang’s overlapped planning stream fails at the first verify on this hybrid model, whose GDN backend does not implement

a call the stream needs.

Removing the blocking reads from attention planning With FlashInfer attention the host gap has an exact lever as well [E44]. Without a profiler, stock SGLang leaves the GPU idle an estimated 1.44 to 1.79 ms of each cycle of the tuned MTP arm at $B = 1$ to 128, 21% of the cycle at $B = 1$ and 9% at $B = 128$, and 1.36 to 1.46 ms of a DFlash cycle at block 8, 21% at $B = 1$ and 9% at $B = 32$ (derived: the untraced stock cycle less the GPU busy time per cycle in a host trace of the patched engine from the same session, assuming the same GPU work in both). In host traces the largest site is FlashInfer’s planning of the verify pass for MTP, which reads a mask size and three index arrays back to the host on every verify, and the drafter’s attention planning for DFlash. Three engine patches (engine/sclang/patches/hostgap/0001–0003) compute those values on the host from lengths the scheduler already holds and leave the plan unchanged. On the GPU the plans and a replayed attention graph’s output bits matched FlashInfer’s own on random batches at every tested size; in the serving engine a validation mode compared at least 8,192 plans of each kind with the stock path and found no difference; and greedy outputs were token-identical to the stock engine in the same configuration (for MTP, the tuned arm with replayed verification), with equal verify-step counts, for all 192 requests of each of MTP and DFlash (64 prompts at concurrency 1, 8 and 32, top- k 1 chains, prefix cache off, running limit and recurrent-state slots pinned; the DFlash attention cache, limited by memory, varied by at most 155 tokens between launches and never bound), as were two stock runs of each. Trees, sampling and the prefix cache are not covered. In held decode batches without a profiler the patches shorten the MTP cycle by 0.19 ms (2.7%) at $B = 1$ to 0.98 ms (4.8%) at $B = 128$, which is 13 to 16% of the estimated idle at $B = 1$ to 8, 27 to 35% at 32 to 64 and 55% at 128; each arm’s stock and patched servers ran one after the other, not interleaved, and the MTP changes are at least four times the larger window standard deviation. The DFlash cycle shrinks by 0.7 to 1.7%, which these sequential runs cannot separate from variation between launches at $B = 1$ and 8. So at low batch most of the idle is host work rather than waiting on a read: in the patched host traces the host still needs 1.70 to 1.83 ms from the previous verify’s completion to the next draft’s launch, while 0.54 to 0.90 ms of GPU work remains queued. Two further patches, cheaper host arithmetic and the split-KV flag FlashInfer sets for NVFP4 caches, have not been validated; they, the interleaved serving sweeps and stock traces retaken with the same window driver are [pending D23].

Backbone GEMMs and RMSNorm The weight GEMMs’ time above the head GEMM’s bandwidth and the 65 RMSNorm launches are the fourth and seventh rows of Table 19. Isolated microbenchmarks under CUDA graphs locate the slow GEMMs and test faster kernels that read the same BF16 weights [E43]. Each timed call multiplies every layer’s own checkpoint weight by its own input, so the weights stream from HBM as in the model; times are medians of 30 timings of 10 graph replays each, in one exclusive session, include any split- K reduction and the gap to the next kernel, and are compared only within these files. The three projections with 2,560 outputs, the GDN and attention output projections and the MLP down-projection, run as split- K kernels followed by a reduction kernel up to $M = 32$ (128 for the down-projection) and reach 49% (output projections) and 68% (down-projection) of the measured 3.79 TB/s read bandwidth [E32] at $M = 1$, against 80–87% for the large projections; disabling cuBLAS’s reduced-precision reduction changes neither their time nor any output bit. At $M = 1$ SGLang’s own Hopper GEMV is the fastest kernel for every projection: it takes 0.77 of cuBLAS’s time on the output projections (8.57 against 11.20 μ s per call for the GDN one), 0.91 on the down-projection, 0.95–0.97 on the other large projections and 0.51 on the 64-row GDN projection, which cuBLAS runs on a single block. At the pin it is unreachable: `--bf16-gemm-backend gemv` loads it, but the unquantized linear layers call the dispatcher only for another backend (code reading; patch engine/sclang/patches/backbone/0001 lets them reach it). At $M = 2$ –16 a Triton skinny GEMM pays only with programmatic dependent launch, at 0.88–0.92 of cuBLAS’s time on the output projections and 0.96–0.99 on the GDN input and MLP projections, with no gain on the attention QKV projection; in this benchmark each call overlaps its successor, which in the model is a different kernel. From $M = 64$ cuBLAS, or a cuBLASLt algorithm within 2–5% of it, is best, except on the down-projection, where PyTorch’s cuBLASLt choice takes 0.92 of the time. None of these routes leaves the target’s arithmetic unchanged in general: the GEMV and the Triton kernel are bitwise equal to cuBLAS on 99.6 to 100% of output elements, and the cuBLASLt choice differs from it on the down-projection from $M = 16$, while all arms lie within the same distance of a float64 product. If every projection ran at its best isolated time, a plain step’s backbone GEMMs would take 197 μ s less at $M = 1$, at most 5.7% of the 3,460 μ s step, and 92 to 120 μ s less at $M = 2$ –16 (derived from the microbenchmarks, not a served result).

SGLang’s packed GDN input projection, one GEMM over the concatenated $12,352 \times 2,560$ weight instead of a 12,288-row and a 64-row GEMM, gives every output element bitwise equal to the separate pair’s at each tested M , the powers of two from 1 to 1,024, with the real weights of all 24 GDN layers and random BF16 inputs, under cuBLAS 13.1 on this GPU [E43]. That is a statement about one isolated GEMM, not about served tokens. Against the stock pair, which runs the

64-row GEMM on a side stream, it is as fast up to $M = 16$, 3% slower at 32 and 5 to 13% faster from 64. The stock fused add-RMSNorm takes $1.47 \mu\text{s}$ per launch at $M = 1$ and 2.0 to $2.2 \mu\text{s}$ at $M = 4$ –32 inside a graph of 64 norms. Folding the norm and the SiLU into the next GEMM’s prologue reproduces the stock norm bit for bit up to $M = 32$ (one element in 10^5 is one BF16 step away at $M = 64$ and 128) and the stock SiLU bit for bit, but as built it makes each layer slower in MLP and GDN layer skeletons, by 12 to $90 \mu\text{s}$ up to $M = 32$, with or without programmatic dependent launch, and by 82 to $543 \mu\text{s}$ at $M = 64$ and 128, far more than the two launches of about 1.5 to $2 \mu\text{s}$ that it removes. In the same skeletons the Triton kernel with programmatic dependent launch saves 1.2 to $2.3 \mu\text{s}$ per MLP layer and 1.4 to $1.8 \mu\text{s}$ per GDN layer up to $M = 16$, except the GDN layer at $M = 4$, and loses at 32. The engine patches that route each projection by a table are committed and off by default; the exactness class of each switch, from greedy outputs against stock plain decoding compared with the noise floor, and paired serving runs against the tuned arms are [pending D22].

What did not work FP8 W8A8 weights, whose CUTLASS GEMM in the installed aarch64 sgl-kernel aborts at run time and whose Triton fallback showed no consistent gain in single runs; FP8 KV cache, which saves nothing at short contexts; BF16 recurrent state, which moves the same bytes as FP16 and is worse on quality in published measurements on another GDN model (Zhang et al., 2026b); folding the RMSNorm and the SiLU into the next GEMM’s prologue, 12 – $90 \mu\text{s}$ per layer slower up to $M = 32$ and 82 – $543 \mu\text{s}$ at 64 and 128 as built, in isolated layer skeletons [E43]; and 2:4 sparsity, which the engine does not support at the pin (its compressed-tensors loader raises an error for 2:4 schemes; code reading, `python/sglang/srt/layers/quantization/compressed_tensors/compressed_tensors.py`). An MTP server with BF16 state at 512 concurrent requests also failed to capture its verify graph, on a FlashInfer workspace overflow (Appendix E.8). Two proposals were rejected by their pre-registered tests: sharing computation across requests (P5, above) and a certified decoder tail (P1, Appendix A.9). The repeated runs, with the mechanism of any lever that fails or regresses in them, are [pending D14].

E.7 Interactions and the two stacks

Speedups from separate levers do not multiply, and quality losses need not add. Three interactions are predictable from the mechanism. A lossy target lowers acceptance, because the drafter was trained against the BF16 target; the published int4 recipe fine-tunes its drafter against the int4 target for exactly this reason (Kim et al., 2026a). A cheaper verifier makes drafting relatively more expensive, which raises the value of a cheap draft head. And every lever that shrinks per-request bytes also raises capacity, so its measured gain depends on whether the server was allowed to use that capacity. Each pair is therefore measured in the baseline, A , B , $A+B$ pattern rather than by multiplying isolated speedups.

The *exact stack* admits only levers that keep the target’s decisions up to rounding-level near ties: capacity and scheduling changes, drafting levers, the levers whose greedy outputs are exact up to rounding (replayed verification among them), and the certified head. The *lossy stack* admits everything, under a quality budget declared before measuring: at most 1.0 point of GSM8K accuracy below the reference on the full test split with thinking on (paired, with an exact McNemar test), teacher-forced top-1 agreement of at least 98% and mean top-20 KL divergence of at most 0.01 nats on a fixed probe set of 48 prompts and 256 tokens, each reported next to the reference’s own run-to-run noise. The combination’s quality is measured on the combined configuration, never summed from its parts. Proposition B.7 explains why the budget is empirical: a per-token logit error small enough to certify a long output as close in total variation is far below what any low-precision lever delivers. The interaction matrix (speed at concurrency 1 and at peak, acceptance and quality for each measured pair) and both frontiers, from one server launch per arm, are [pending D21].

E.8 The serving protocol

Every served speedup is measured against a server, not a kernel. This subsection defines that measurement: what runs, on which prompts, how the latency–throughput frontier is computed, how the baselines are made strong before anything is compared with them, and which quality check a change must pass.

Setup and workload All runs use the machine and engine of Section 3, serving the BF16 target at its pinned revision with FlashInfer attention (Ye et al., 2025); the vision encoder, which text requests never reach, uses Triton attention. The client is AIPerf (NVIDIA, 2026a). Speculative acceptance depends on what the model is writing, so random-token prompts would measure the wrong thing. The benchmark uses a frozen set of 1,857 public prompts in three domains: chat (the first turns of the 80 MT-Bench questions (Zheng et al., 2023) and English root prompts from OASST1 (Köpf et al., 2023)), code (the 164 HumanEval problems (Chen et al., 2021) and MBPP test tasks with their assertions (Austin et al., 2021)) and mathematics

The Work a Verifier Needs

Axis	Lever	Class	Status
Progress per verify pass	MTP and DFlash drafting, deeper chains, trees where the GDN path allows, n-gram drafting	exact	Chosen in single runs on the tuning split: MTP with three steps, DFlash blocks 16, 8 and 4 by concurrency (Table 22); trees and n-gram drafting not searched; served comparison [E67]
Bytes per verify pass	Relaxed greedy acceptance within g logits	lossy	[pending D20]
	Certified int8 head (Section 4) Replayed verification of the recurrent state	exact exact up to rounding*	[pending D7] MTP, three steps: 0.86× plain decoding at $c = 128$ on the confirmation split, against 0.68× with snapshot verification (one session) [E67]; outputs rounding-level against stock MTP [E68]; no detectable GSM8K loss [E69]; beyond $c = 128$ [pending D15]
Bytes per draft step	FP8 W8A8 weights, FP8 KV; int4 target with a matched drafter (Kim et al., 2026a)	lossy	FP8 weights: no consistent gain; FP8 KV: no gain (single runs); int4 [pending D14]
	Hot-vocabulary or low-precision draft head	exact	[pending D20]
Concurrency and state	One state slot per request (lifts the 133-request cap)	exact	[pending D31]
	FP16 state; replayed decoding; rounding-preserving replay (P4)	lossy, exact up to rounding, exact	FP16: 1.24× at $c = 128$ (single run, quality pending); replayed decoding: 1.08× at $c = 128$ on the confirmation split in one session [E67], outputs rounding-level [E68], more sessions [pending D32]; P4: kernel 1.22× at $B = 128$, derived about 1.08× per decode step [E40]; served test [pending D17]
Backbone GEMMs and norms	Kernel routing per projection and M (Hopper GEMV, Triton with programmatic dependent launch, cuBLASLt); packed GDN input projection; norm and SiLU folded into the GEMM	pending	Isolated microbenchmarks, best kernel per call up to $M = 16$: 0.77–0.92 of cuBLAS’s time on the output projections, 0.91–1.00 on the other large ones; packed projection bitwise equal per GEMM; folding 12–90 μ s per layer slower up to $M = 32$, more above [E43]; served [pending D22]
System overheads	Host gaps and synchronizations, fusion, attention backend per shape	exact; Triton target attention exact up to rounding	Measured shares in Table 19; Triton attention: MTP 1.18× at $c = 1$, draft only 1.08× (single runs [E42]); blocking planning reads removed with token-identical outputs, MTP cycle 2.7–4.8% shorter in held batches [E44]; served [pending D23]

Table 21. Levers by axis. A change of the target’s attention backend changes the target’s arithmetic, so it is classed by its greedy outputs like any other lever. *The replay fold is written as a bitwise clone of the verify kernel’s state update, but the verify outputs come from a chunked transform on TF32 tensor cores, whose rounding differs from the recurrent kernel’s by more than a reassociation; that mechanism suggested lossy, but every first divergence of the buffered MTP arm from stock MTP is rounding-level [E68].

(GSM8K training questions with a request to box the final answer (Cobbe et al., 2021); the GSM8K test split is reserved for the quality check, so no workload prompt is a quality problem). Each source is pinned to a Hugging Face revision and the builder reproduces the files byte for byte. Each domain is cut into disjoint warm-up (43 prompts), tuning (192) and confirmation (384) sets, and every split interleaves the domains so that any prefix of it is balanced, which matters because low-concurrency points use only a prefix. Configurations are chosen on the tuning split and reported on the confirmation split. Templated prompts are short: the confirmation split has a mean of 87.9 prompt tokens (median 78, 99th percentile 283), with chat shorter (mean 47) than code (130) and mathematics (86) [E62].

Requests use the model’s chat template with thinking enabled (the template’s default) and no reasoning parser, so the thinking text is streamed and counted as output. Decoding is greedy. The throughput panel fixes the output at 512 tokens with `ignore_eos`, and every point checks that each response has exactly that length. A fixed length is a faithful sample of the model’s behaviour only while the model has not chosen to stop, so a natural-stopping run on 576 tuning prompts measured where greedy outputs end. Only 3.8% stopped before 512 tokens and 0.7% had entered a detected repetition loop by then (a 256-token window with fewer than 30% distinct 8-grams), so almost all of the fixed-length panel is text the model would have produced anyway. The same run shows why greedy decoding is not used for the quality check: 142 of the 576 outputs (25%; 36% in mathematics, 32% in chat, 6% in code) ran to the 16,384-token limit, a degeneration Qwen’s model card warns about for greedy thinking (Qwen, 2026a). The run used the previous workload version, whose mathematics prompts came from GSM8K test, and other processes used up to 24 CPU cores during it, so it is a length measurement, not a timing [E63].

Metrics For request i at a sweep point let n_i be the number of generated tokens the server reports in the `usage.completion_tokens` field, s_i the time the client sends it, f_i the arrival of its first non-empty streamed chunk and e_i the interval from s_i to its last non-empty chunk. Over the N measured requests the frontier plots

$$x = \frac{1}{N} \sum_{i=1}^N \frac{n_i}{e_i}, \quad y = \frac{\sum_i n_i}{\max_i (s_i + e_i) - \min_i s_i}, \quad (13)$$

so x is the mean per-request output rate *including* the time to first token, and y is output tokens per second on the one GPU. Two companions are reported and never substituted for them. The decode rate $x_{\text{dec}} = \frac{1}{N} \sum_i (n_i - c_i) / (e_i - (f_i - s_i))$, where c_i is the exact token count of the first chunk from per-chunk usage (a speculative step streams several tokens per chunk), excludes queueing and prefill; one token per chunk is never assumed. The steady-state throughput counts only the tokens streamed between the first and the last send, when the client holds its full concurrency, and so excludes the drain at the end of a point. Each point also records time-to-first-token, inter-token and end-to-end latency percentiles, failures, the per-request speculative statistics the server attaches to each stream (verify steps, accepted drafts, and accepted length as output tokens per verify step with the bonus token included), and deltas of the server’s counters, among them the fraction of decode passes that replayed a CUDA graph. The scheduler’s logged generation rate while the batch is nearly full is recorded as a diagnostic of what the GPU sustains; clients do not see it, and it is never a frontier metric.

Protocol and baselines A server arm is a model revision plus a declarative set of launch flags. The harness starts it once, verifies it, and sweeps client concurrency against it in closed loop; the confirmation sweep covers $c \in \{1, 2, 4, 8, 16, 32, 48, 64, 96, 128\}$, each arm over the concurrencies where it competes (bench/campaigns/confirm.sh). Before trusting a launch it checks that decode CUDA graphs were captured for every batch size up to the server’s capacity (the target decode graph, or the target-verify, draft-decode and draft-extend graphs under speculation), that prefill graphs were captured, that the overlap scheduler is running (the scheduler reports it, no fallback is logged, and a stack sample of the scheduler process shows its overlap loop), and that the recurrent-state cache did not reduce the capacity.

Capacity needs care on this model. The attention cache is not the constraint: 32 KiB per token, and the default memory split leaves room for about a million tokens. The recurrent state is. With the prefix cache and the overlap scheduler on, SGLang reserves five FP32 state slots per running request (three for prefix-cache retention and two ping-pong buffers), while decode touches one; the 667 slots the default split gives the state pool therefore cap the server at 133 running requests (code reading, confirmed against the startup log) [E30]. Speculative verification adds one intermediate state per draft token, and SGLang resets speculative servers to 48 running requests, and logs it, unless the capacity is given; the capacity limits per arm at default settings are [pending D31]. Feasibility probes show what launches: plain decoding serves 256 concurrent requests with the prefix cache and 1,024 without it, MTP speculation serves 256 without the prefix cache, and an MTP server with BF16 state at 512 fails at verify-graph capture on a FlashInfer workspace overflow. These probes establish what runs, not performance [E64]. Every arm is therefore launched at a capacity of 128 running requests, the top of the client sweep (64

for one DFlash arm, below), with the state cache sized for it and the remaining static memory given to the attention cache and the verify states.

Every arm streams every fourth token (`stream-interval = 4` in `bench/arms.toml`). The server still sends the first token at once and the last at finish, so the times that Equation (13) uses do not depend on the interval, and the choice takes load off the single-threaded streaming path (below). Each point sends $\max(64, 8c)$ measured requests, the first prompts of the confirmation split in order, after one wave of warm-up requests at the same concurrency, and the prefix cache is flushed before every point so that no measured prompt is served from cache. The confirmation sweep runs in three sessions. Each session launches every tuned arm afresh in the same exclusive hold as its matched plain baseline (the FlashInfer plain arm for FlashInfer arms, the Triton one for Triton arms), in reversed order in alternate sessions, so that drift does not align with an arm. Within a session each arm runs once, over its concurrencies in ascending order; the harness alternates the order of concurrencies only between repeats within one launch, which the confirmation sweep does not use. Speedups are ratios within a session; a pair with an invalid point on either side is dropped and counted, never replaced by another run, and points at which the attention cache limited the batch are flagged from the scheduler log. Timed runs hold the GPU exclusively, and because the scheduler, tokenizer manager and client are single-threaded CPU loops, each point also waits until other processes use fewer than two cores and records the foreign load it saw.

The speedup denominator is the strongest existing configuration, not a default. Arm A is ordinary autoregressive decoding. Arm B is native MTP speculation, whose draft steps reuse the target’s tied head. Arm C is the public DFlash drafter (Appendix F). The search ran on the tuning split [E66]: 23 configurations, each launched once and measured at client concurrency 1, 8, 32 and 128 (64 instead of 128 for the two block-16 configurations) with $\max(32, 4c)$ requests per point and 512 output tokens. It covered MTP chain depth (two to five steps with the prefix cache, three, five and seven without it), the prefix cache, SGLang’s replayed GDN verification and replayed decoding, its batch-adaptive depth, Triton against FlashInfer attention, DFlash blocks of 4, 8 and 16 tokens and FA4 attention for the DFlash drafter. A one-step chain is not in the search, because it failed a launch check that required a draft-decode graph such a chain does not capture; replayed verification was refused at launch for DFlash and with FlashInfer’s GDN decode kernel. Draft trees (top- k 2 and 4, which run on this hybrid model, with accepted lengths of 3.45 and 3.56 at concurrency 1 in the feasibility probes [E64]), the verify attention mode, the prefill chunk size and graph backend were not searched. With one launch per configuration, Table 22 is selection evidence, not a result.

Three regularities decide the arms. First, speculation pays at low concurrency and loses at high. MTP with three steps delivers 1.64 times plain decoding’s y at concurrency 1 (461 against 282 tokens/s); with snapshot verification it falls behind from concurrency 32 (5,839 against 6,087), with replayed verification and no prefix cache it still leads there (6,925) and trails at 128 (13,011 against 13,844), and at concurrency 128 every speculative configuration trails plain decoding. Second, replayed verification, which removes the per-draft-token state snapshots (Appendix E.4), recovers 24% at concurrency 128 for three steps (11,941 against 9,593 tokens/s) at a 3% cost at concurrency 1, and depth 3 beat depths 4 and 5 at concurrency 128 in every pair the attention cache did not limit, while trailing them by at most 4% at concurrency 1 and 7% at 8; in the low-concurrency arm’s own configuration (replayed verification, Triton), depth 4 led depth 3 by 3.2% at concurrency 1 and 0.4% at 8 and trailed it by 2% at 32, so one depth serves both MTP arms. Third, Triton attention helps every speculative family at low concurrency and hurts at high concurrency, and plain decoding is indifferent to it. DFlash at block 16 under Triton has the highest throughput of any configuration at concurrency 1 and 8, and at concurrency 1 a per-request rate 3.35 times that of plain decoding under Triton (957 against 286 tokens/s in x); among DFlash configurations block 4 leads at concurrency 128 (11,477 tokens/s, 8% above block 8 with FA4 draft attention).

The chosen arms (`bench/arms.toml`) all run without the prefix cache, which this workload does not reuse and whose state slots deeper drafts and replayed verification need. Arm A is plain decoding under FlashInfer, with a Triton twin as the matched baseline for Triton arms. Arm B is MTP with three steps and replayed verification, under FlashInfer for high concurrency and under Triton for low. Arm C is DFlash at block 8 with FA4 draft attention for every concurrency, at block 16 under Triton for low concurrency, capped at 64 running requests because its 16 verify states per request take about 0.8 GB, and at block 4 for high concurrency. Two of these choices change the target’s arithmetic: replayed verification computes its verify outputs with a chunked transform on TF32 tensor cores (Table 21), and Triton attention for the target replaces the target’s attention kernel. Both are exact up to rounding by the comparison below, as is replayed decoding, so every confirmed arm is stock or exact up to rounding. MTP with SGLang’s snapshot verification under FlashInfer is kept as a stock reference, and plain decoding with replayed decoding, the fastest configuration at concurrency 128 on the tuning split and the slowest at 1, as a variant.

Arm	Configuration	Prefix cache	y (tokens/s) at concurrency c				Accepted length
			1	8	32	128	
Plain	FlashInfer	on	282	1,982	6,087	13,421	–
	FlashInfer	off	282	2,004	excl.	13,844	–
	Triton	off	286	2,018	6,241	13,846	–
	FlashInfer, replayed decoding	off	244	1,807	5,909	14,981	–
MTP	2 steps	on	385	2,288	5,455	9,153	2.67
	3 steps	on	461	2,482	5,839	9,593	3.28
	4 steps	on	465	2,608	5,761	9,276	3.72
	5 steps	on	468	2,575	5,684	8,719 [†]	4.11
	3 steps	off	466	2,534	6,127	10,106	3.27
	5 steps	off	481	2,699	6,087	9,456	4.14
	7 steps	off	465	2,514	5,690	8,493	4.62
	3 steps, RV	on	446	2,646	6,469	11,941	3.28
	5 steps, RV	on	462	2,666	6,502	11,186	4.11
	3 steps, RV	off	456	2,667	6,925	13,011	3.29
	3 steps, RV, Triton	off	535	3,093	7,103	11,353	3.27
	4 steps, RV, Triton	off	552	3,104	6,958	10,804	3.73
	Adaptive depth, RV	off	474	2,603	5,968	10,010	1.97
DFlash	Block 4	off	498	2,682	6,380	11,477	3.28
	Block 8	off	686	3,443	6,727	10,226	4.75
	Block 8, FA4 draft attention	off	769	excl.	6,983	10,594	4.75
	Block 8, Triton	off	807	3,613	6,307	8,957	4.75
	Block 16 [‡]	off	excl.	3,398	5,386	6,737	5.71
	Block 16, Triton [‡]	off	851	3,639	5,199	6,397	5.72

Table 22. Configuration search on the tuning split: output throughput y (Equation (13)) from one launch per configuration with $\max(32, 4c)$ requests per point, and the accepted length at concurrency 32. FlashInfer attention unless Triton is named; MTP chains draft one token per step; RV: replayed GDN verification. “excl.”: excluded because other processes used more than two CPU cores on average, by a sampler that then overcounted, so these may be false positives. [†]Limited by the attention cache, which without a cap had shrunk to 54K tokens as the state buffers grew: at most 123 requests ran, with retractions. [‡]Capacity 64; the last throughput column is concurrency 64. Selection evidence, not results [E66].

Output equality Each arm that changes the target’s arithmetic was compared with a matched stock reference on the 320 divergence prompts of Appendix D.4, decoded greedily for 256 tokens with the top five log-probabilities at concurrency 1 and without the prefix cache, like every arm: plain levers against stock plain decoding, buffered MTP against stock MTP with three steps, and Triton DFlash at block 16 against stock DFlash at block 16, both DFlash runs at a capacity of 4 [E68]. The class rule was fixed after the first comparisons had been seen. It rests on the divergence classes of Appendix D.4, which predate them: an arm is exact up to rounding if every prompt’s first divergence is at an exact tie, within one BF16 step, or a near tie with both margins within two steps, and no output differs in length; it is lossy otherwise. Every arm passes, with no event above a near tie. Against its reference, buffered MTP diverges at 3.79 per 1,000 compared tokens, 1.11 times the batch-shape floor of 3.42 (plain decoding at concurrency 1 against 32 on one server with the prefix cache on and at most 16 running requests); its Triton twin at 4.05 (1.19), Triton DFlash at block 16 at 3.82 (1.12), Triton plain decoding at 3.89 (1.14) and replayed plain decoding at 3.40 (0.99). Every ratio’s 95% interval includes 1, and the ratios inform the class rather than decide it. The ratios depend on that floor’s cap: with at most 8 running requests and pinned pools the same pair diverges at 0.63 per 1,000 (Appendix D.4), while the classes, which count event types, do not depend on it [E28]. Replayed plain decoding was first classed lossy against the divergence study’s plain run, which has the prefix cache on: one prompt’s first divergence was large. At that position the prefix-cache-on run’s own token rests on a logit gap of 0.0625, which stock plain decoding without the prefix cache resolves the other way, and with the prefix cache on a request’s log-probabilities can depend on earlier requests (Appendix D.4). Against the reference without the prefix cache, replayed plain decoding first differs from it in that prompt at a tie further on, and its rate is the floor’s. The two stock plain references, with and without the prefix cache, differ in 96 of 320 prompts at 1.69 per 1,000, all rounding-level.

Stock speculation against the same plain reference diverges at 1.15 times the floor’s rate both for MTP with three steps (3.95 per 1,000; the ratio’s 95% interval is 0.93 to 1.43) and for DFlash at block 16 (3.92; 0.93 to 1.42). Against the prefix-cache-on run the ratios were 1.29 and 1.28, with intervals above 1, so that apparent excess came at least partly from

the reference’s prefix-cache setting, not from speculation: at the pin that setting changes the recurrent prefill of prompts longer than one 64-token chunk (Appendix D.4), which shows from the first output token; how far that holds across the 320 prompts, and with the overlap scheduler off, is [pending D11]. Against this floor, then, an excess of speculation is not detected, though one of up to about 40% is not excluded. Three limits apply to this check: the floor is a prefix-cache-on pair on one server, and a floor without the prefix cache was not run; the servers’ pools were sized from free memory, not pinned; and the DFlash runs used a capacity of 4 against plain decoding’s 16. Whether speculation shows an excess depends on the floor, however. With pinned pools and at most 8 running requests, both runs with the prefix cache on, plain decoding at concurrency 1 against 32 diverges at 0.63 per 1,000 while MTP with three steps against plain decoding at concurrency 1 diverges at 3.68, about six times that floor (derived), and every MTP configuration diverges from plain decoding at 3.5 to 4.0 per 1,000, about as often as it diverges from itself between concurrency 1 and 32 (3.1 to 3.5) [E28]. The speculative rate is about the same in both regimes, and what moves is plain decoding’s own batch-shape divergence.

Quality check Output length and throughput say nothing about whether a change preserved the model. The quality check is the full GSM8K test split (1,319 problems) scored with the GSM8K grader of the `sgl-eval` package, version 0.1.2 (a zero-shot prompt, the last boxed answer, symbolic equality) with thinking on, natural stopping at 16,384 tokens, and Qwen’s recommended thinking-mode sampling for precise tasks (temperature 0.6, top- p 0.95, top- k 20) under a fixed request seed. Two runs are compared problem by problem: the accuracy difference, an exact McNemar test on the discordant problems, and the share of identical final answers. Sampled accuracy tests equality in distribution for a lossless change and prices a lossy one; it cannot show that greedy outputs are unchanged, which the output comparisons of Appendix D.4 address. No arm differs detectably from plain decoding [E69]. Two launches of plain decoding score 89.99% and 90.30%, buffered MTP 89.54%, MTP with snapshot verification 88.93%, DFlash at block 8 89.69% and replayed plain decoding 91.05% (run while it was still classed lossy). Every paired difference against either plain run lies between -1.36 and $+1.06$ points, with exact McNemar $p \geq 0.18$. A non-significant test excludes nothing by itself. The paired 95% intervals (Wald intervals on the discordant problems, derived from the committed counts) have lower bounds between -0.7 and -3.3 points, the lowest for MTP with snapshot verification against the second plain run (-1.36 points, interval -3.27 to $+0.54$), so the data exclude losses larger than about 3.3 points for every arm and cannot exclude smaller ones. Two sampled runs of one arm agree on the final answer for only 77% of problems, so the fixed request seed does not make these runs reproducible, and 17 to 19% of outputs in every arm reach the 16,384-token limit.

Frontiers The baseline frontier is the upper envelope over arms A, B and C, since no arm need be best at every concurrency: speculation saves weight traffic, which matters most at low concurrency, and adds per-request state traffic, which matters most at high concurrency (Appendix E.4). Every arm carries an exactness class in `bench/arms.toml`: *stock* when it sets only flags known to leave the target’s arithmetic as in plain decoding with FlashInfer target attention and stock state handling, *exact up to rounding* by the comparison above, *pending* until compared, and *lossy* otherwise. All nine confirmed arms are stock or exact up to rounding, so the envelope over exact arms equals the envelope over all arms. The frontier over three confirmation sessions, with paired ratios against the matched plain baselines and the run-to-run variance, is committed [E67]; only points with three valid sessions rank, so replayed plain decoding, with one session, does not yet [pending D32]. Still pending are the before-and-after frontiers for the certified head in MTP drafting, verification and plain decoding, with output-equality checks at the same batch shape, which decide H4 [pending D34], and the exact and lossy stacks [pending D21].

Sensitivity to output length With a closed-loop client and equal output lengths, plain requests start and finish in synchronized waves, while speculative arms advance requests at rates that vary with acceptance, so their completions spread out into more, smaller prefill passes and a drain at the end of a point. A sensitivity workload, declared before any arm was timed on it, removes the equal lengths and nothing else: each confirmation prompt is given its own greedy completion length under plain decoding with natural stopping, capped at 2,048 tokens, so that every arm generates the same number of tokens per request. It runs at concurrency 32 and 128 with `max(64, 8c)` measured requests, the ordered prefix of the confirmation split (256 and 1,024 prompts). The arms follow a rule fixed before any sensitivity run: for each speculative family and concurrency, the arm with the highest mean y over the three confirmation sessions, invalid points excluded and only arms with a valid point in all three eligible, together with the family’s runner-up if it is within 2% of that mean. Each runs with its matched plain baseline, in three sessions. The fixed-length panel stays the primary result; where the sensitivity workload changes a ranking, both numbers are reported. Its results are [pending D33].

Measurement hazards Two effects outside the GPU shaped this protocol. At high concurrency plain decoding can be limited by the streaming path rather than the GPU; speculative arms stream several tokens per chunk and are less exposed, so a frontend bottleneck would inflate their measured advantage. A diagnosis with one plain server at client concurrency 256 on a quiet host measured the effect [E65]. With one-token chunks the tokenizer-manager process (HTTP and the OpenAI serving layer) ran at a full core and the client received about 85% of the server’s full-batch decode rate; streaming every four tokens brought that process to 0.35 cores and gave the same throughput as a non-streaming client, about 92% of the full-batch rate, the rest being the prefill of each new wave and the drain, which y includes by definition. More tokenizer and detokenizer workers made it worse, and more client workers changed nothing. Every row of that diagnosis is a single point; concurrency above 256 on a quiet host is [pending D35]. The same diagnosis attributes the low throughput of the earlier feasibility probes at concurrency 256 to 1,024 to CPU contention on the host, not to a frontend limit. That is the second effect: exclusive use of the GPU does not keep CPU-heavy jobs off the host, which is why every point records foreign CPU load and waits for a quiet machine.

F DRAFTING AND REPAIR HYPOTHESES

How much a verify pass is worth depends on the drafter in front of it: a cycle commits as many tokens as the target accepts plus one, and its cost includes the draft. The native MTP layer of Qwen3.5-4B drafts sequentially, paying one small layer plus the full 1.27 GB head per drafted token, and its acceptance decays with depth. A block drafter proposes a whole block in one forward pass. Every drafter that projects through a head pays for it at every draft step: Medusa and the EAGLE family (Cai et al., 2024; Li et al., 2024a;b; 2025), native multi-token prediction (Gloeckle et al., 2024; DeepSeek-AI, 2024) and DFlash (Chen et al., 2026a). This appendix describes the public block drafter for this target, how SGLang runs it on a hybrid model and what is measured about it, and then states the proposals on drafting and verification as hypotheses with their kill tests: a selector trained for rate (P6), exact repair of long draft windows (P2, P3), reuse of a cached window after a rejection (P9), and the treatment of the recurrent state in verification (P7b, P10, P11). P6 passes its first screen; P3 is refuted, because its anchored evaluator fails at the operator level; P2’s first test depends on the verify kernel, and with SGLang’s Triton kernel (exactness pending) P2 turns on drafting long, almost fully accepted blocks; P9’s oracle bound does not reject reuse with top-8 and top-16 candidate sets at concurrency 1; the remaining tests are pending.

F.1 The public block drafter

DFlash has a public drafter for exactly this target, `z-lab/Qwen3.5-4B-DFlash` at revision `9a1996cc`, under Apache-2.0 (Chen et al., 2026a; z-lab, 2026); `modal-labs/Qwen3.5-4B-DFlash` carries byte-identical weights. Read from the checkpoint rather than the card, it is a six-layer Qwen3-style transformer of width 2,560 (32 query and 8 key-value heads of dimension 128, MLP width 9,216) with no embedding or head of its own: it embeds with the target’s table and projects through the target’s tied head. It conditions on the target’s hidden states after layers 1, 5, 9, 13, 17, 21, 25 and 29, concatenated per token and projected by one linear map and an RMS norm to a single context vector. Five layers use sliding-window attention (window 4,096) and the last uses full attention. It has 634 million parameters, 1.27 GB in BF16, almost exactly the size of the target’s head (derived).

A cycle in SGLang’s DFlash worker (SGLang contributors, 2026c) works as follows (code reading at the pin). The block is the last committed token followed by fifteen copies of a mask token (id 248,077). Each draft layer’s keys and values for the committed context come from the projected target features, which the worker writes into the drafter’s own cache as tokens are committed. One forward pass over the sixteen positions yields sixteen hidden states; positions 1–15 go through the target head, and their argmaxes are the drafted tokens. The target verifies the block in one pass, the worker accepts the longest drafted prefix that matches the target’s argmax, commits it plus the target’s token at the first mismatch, and appends the committed tokens’ features to the drafter’s cache. The sliding layers are causal inside the block and the full layer is not; SGLang infers this from the layer types because the configuration sets `no_is_causal`, which matches the attention pattern of the DFlash training recipe in SGLang’s SpecForge repository (`sgl-project/SpecForge`).

On the hybrid target the verify pass has a cost the drafter’s parameters do not show. SGLang verifies a chain on the GDN layers by keeping the recurrent state after every block position, so that the state after the accepted prefix can be committed without recomputation. Each state is 50.3 MB per request, so a 16-token block writes about 805 MB per request per cycle, and each running request reserves space for sixteen of them (derived). That bounds block-16 capacity on this machine, and it is a property of the verifier, not of the drafter.

The card’s launch line targets a B200 with TRT-LLM target attention, FA4 draft attention and piecewise-compiled prefill

graphs. None carries over unchanged: TRT-LLM attention is Blackwell-only, FA3 and FA4 are unavailable on this aarch64 host, and the piecewise prefill graph fails to capture for this target at the pinned commit. The drafter is served with FlashInfer for target and draft attention, the FlashInfer GDN kernels, the default prefill graph and the card’s overlap planning stream, with decode, verify and draft CUDA graphs and the overlap scheduler on; the launch records of the acceptance runs below give the exact command, environment and resolved server settings [E48].

F.2 What a cycle costs

Counting weight bytes gives the shape of the trade before any timing. At batch one a DFlash cycle reads the drafter once (1.27 GB), the target head once for the fifteen drafted positions (1.27 GB), and the target once to verify (8.41 GB): drafting costs about 30% of the verify pass’s weight traffic and proposes fifteen tokens. An MTP chain of three steps reads the MTP layer and the full head three times (about 4.5 GB, 54% of the verify pass) and proposes three (derived). Activation traffic, the per-position GDN states, attention over the context and host time are not in these counts; the measured MTP cycle of Appendix E.2 shows how much the host adds. The split of the DFlash cycle at batch 1, 4, 16 and 64 into draft forward, draft head, verification, GDN commit, cache update and host time is [pending D24].

F.3 Acceptance by block position

Let h_j be the number of verify cycles in which exactly j drafted tokens were accepted, $N = \sum_j h_j$, and $S(k) = \sum_{j \geq k} h_j / N$ the fraction of cycles that accept at least k . The conditional acceptance of block position k is $\alpha_k = S(k) / S(k-1)$, and a cycle commits $\tau = 1 + \sum_{k \geq 1} S(k)$ tokens on average over cycles. SGLang reports h per request, so α_k and τ are measured rather than modelled. The model card instead averages each request’s tokens per cycle over requests; we call that quantity $\bar{\tau}_{\text{req}}$. It weights short responses more than τ does, and for these runs it is the larger of the two. The card reports mean accepted lengths of 5.9–7.7 at block 16 on a B200 for its five benchmarks, against 3.3–3.5 for MTP with three steps (z-lab, 2026). The same card reports, on one B200 with greedy decoding, 3.40–4.60× over autoregressive decoding at concurrency 1 with blocks of 16 and 2.15–2.61× at concurrency 32 with blocks of 8, against at most 2.31× and 1.80× for the MTP layer. These are the authors’ numbers on other hardware.

The measurement uses a panel of 32 MATH-500 problems and the first 16 chat, 16 code and 16 GSM8K prompts of the benchmark’s earlier confirmation split, decoded greedily with thinking on, stopping naturally at up to 2,048 new tokens, at concurrency 1, with blocks of 16 and 8 [E49]. The runs used an engine trace hook that synchronizes every cycle on a shared GPU, so acceptance and token counts are valid and timings are not; the panel contains benchmark test problems and measures acceptance only, never quality.

Block	Average	All	Chat	Code	GSM8K	MATH-500
16	τ , over cycles	6.18	3.98	7.63	6.72	7.55
16	$\bar{\tau}_{\text{req}}$, over requests (the card’s)	6.98	4.26	7.87	7.16	7.81
8	τ , over cycles	5.04	3.64	5.75	5.31	5.77
8	$\bar{\tau}_{\text{req}}$, over requests	5.34	3.80	5.76	5.45	5.84
Requests (verify cycles at block 16)		80 (21,274)	16	16	16	32

Table 23. Committed tokens per verify cycle for the public DFlash drafter on GH200 (greedy, thinking on, concurrency 1), pooled over cycles (τ) and averaged over requests as the model card does ($\bar{\tau}_{\text{req}}$). Measured in correctness runs [E49]; the survival form $1 + \sum_k S(k)$ agrees with the pooled τ to within 0.013 in every column.

A block of 16 commits 6.2 tokens per verify cycle and a block of 8 commits 5.0 (Table 23). Acceptance depends strongly on the domain: chat commits about four tokens per cycle, GSM8K 6.7, and code and MATH-500 about 7.6. The per-request MATH-500 average, 7.8, sits near the 7.48 the card reports for that benchmark in the same convention (z-lab, 2026), but that is a cross-hardware reference, not a reproduction: the card used a B200 and a 4,096-token cap and does not state how many problems it used, where this panel uses a GH200, a 2,048-token cap and 32 sampled problems. A gate with the card’s own settings on its MT-Bench prompts does reproduce its number within sampling noise [E51]: over the 80 MT-Bench first turns at block 16, greedy with thinking on, up to 4,096 new tokens and concurrency 1, the mean accepted length per request is 5.73 (bootstrap 95% interval 5.36–6.13; 5.25 pooled over cycles) against the card’s 5.93 on a B200, with 26 of the 80 responses reaching the token limit. The panel’s low chat acceptance comes from its OASST1 prompts (4.06 per request over 14 prompts) rather than its two MT-Bench prompts (5.70).

The per-position rates of Figure 8 show why a longer block still commits more tokens per cycle; whether it pays depends on the cost of a cycle, which is pending. Over all domains the conditional acceptance dips at the second position (0.80) and then rises toward 0.9, so the share of cycles that reach position 15 is still 13%. The rise is a selection effect: cycles that get that far are in predictable stretches of text, which is not evidence that the drafter is better deep in the block. Chat is lower throughout (0.69 at positions 2 and 3), and the dip at position 2 appears in every domain. Two further observations bound what training could change. Acceptance, pooled over tokens, is lower in the answer after the thinking text than in the thinking itself for chat (3.3 against 4.2 tokens per cycle) and higher for code and mathematics, so no single segment dominates. And of the 18,596 cycles with a rejection, the first rejected draft equals the target’s token at the previous position in 6% (1,082), a repetition-type failure, and is some other token in 94% [E50]. On a second panel (the first 16 chat, code and GSM8K-train prompts of the serving workload’s confirmation split, and the same 32 MATH-500 problems), the drafter commits 6.81 tokens per cycle per request (6.11 pooled) at block 16, and the native MTP layer with three draft steps 3.38 (3.34) [E52]. MTP accepts more at the first three positions (0.90, 0.86 and 0.86, against 0.86, 0.79 and 0.81), and the block drafter keeps drafting to position 15. Whether the longer blocks pay for their cost depends on the cycle split, which is [pending D24].

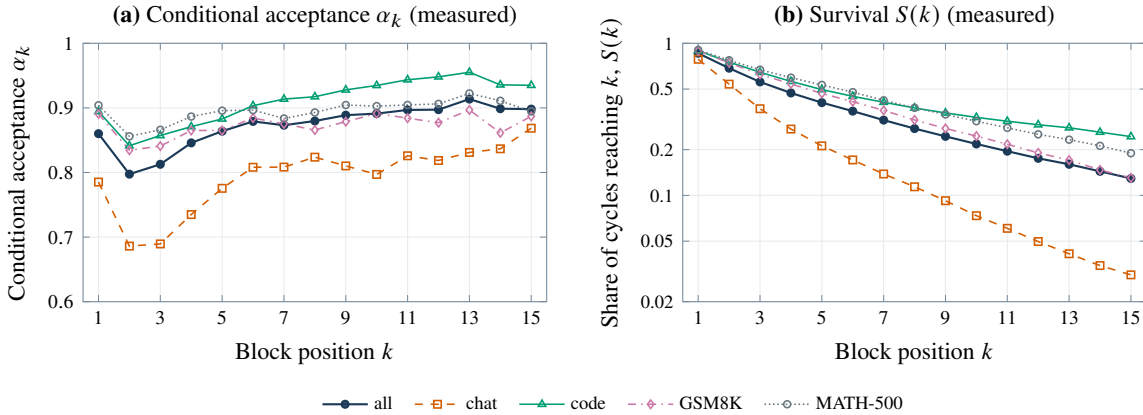


Figure 8. The public DFlash drafter by block position, block size 16, greedy with thinking on, per domain of the panel. (a) Conditional acceptance α_k ; (b) the share of verify cycles that accept at least k drafted tokens, on a logarithmic axis. Measured in correctness runs [E49]; the late rise in (a) is a selection effect of the few cycles that get that far.

F.4 Output equality and limits

Greedy speculative verification is exact in real arithmetic, but the verify pass computes the target’s logits at a different batch shape from plain decoding, so the stock noise floor of Appendix D.4 applies. On the second panel at concurrency 1, 77 of 80 outputs of the block drafter diverge from plain decoding somewhere in up to 2,048 tokens, 3.06 divergences per 1,000 compared tokens, and 77 of 80 MTP outputs, 2.50 per 1,000 [E53]. At each divergence the plain run’s two competing tokens were close: an exact BF16 tie at 47 of the 77 DFlash divergences and 40 of the 77 MTP ones, within one BF16 spacing at 28 and 37, and in 2 DFlash cases the speculative run’s token was outside the plain run’s top two. The classes are one-sided, because only the plain run recorded its top two log-probabilities, and the DFlash divergences have not been traced to a kernel; the rates are of the same order as the plain-decoding floor between concurrency 1 and 32 at a cap of 16 running requests (3.42 per 1,000). Two-sided margins and a trace to the first differing kernel are [pending D25]. Four limitations are candidates: acceptance that decays with block position or by domain; drafting cost dominated by the full-vocabulary head for fifteen positions; the verifier’s per-position state writes, which bound block size at high concurrency; and draft states far from the target’s in the head’s metric, which is what defeats transport on DFlash drafts (median relative drift 0.92, Section 5.3). The public drafter’s training data are not published, so none of these limitations can be traced to its data. A custom drafter is worth training only against one of them, judged by total serving time: fine-tuning on the target’s own outputs for acceptance, adding DFlash 2’s selector (Inco AI, 2026a;b) for acceptance, or an auxiliary loss on $\|h^t - h^d\|_2^2 / \|h^t\|_2^2$ if the roughly hundredfold drift reduction transport needs proves reachable (Appendix C.9). A training set of 18,000 prompts, disjoint from every benchmark and quality set by construction, is built and its manifest committed [E55]; which limitation the measurements support, and the decision on training, are [pending D26].

E.5 Training a selector for rate (P6, hypothesis)

A later proposal keeps the DFlash backbone and the verifier frozen and trains only a small predecessor-conditioned selector over the backbone’s $K = 16$ candidates per position, $q_i(b \mid a) = \text{softmax}_{b \in C_i} [\ell_i(b) + s_{\theta,i}(a, b)]$, so that one coherent path is proposed; DFlash 2’s selector architecture (Inco AI, 2026a), whose training objective the announcement does not state, is the baseline, not a novelty. The objective is emitted tokens per unit time, $\mathbb{E}[\sum R]/\mathbb{E}[\sum C]$ with $R = L + 1$ per cycle, trained as $R - \lambda C$ with λ set to the measured rate. If λ^* is the best achievable ratio in Equation (10), then $\mathbb{E}R - \lambda^* \mathbb{E}C \leq 0$ for every policy, with equality exactly for the policies that attain the ratio, because $\mathbb{E}C > 0$; so maximizing the difference at the right λ maximizes the rate. Two identities make the accepted-length term trainable exactly.

Proposition F.1 (Prefix value of a sampled draft policy). *Let the draft path Y be sampled position by position, $Y_i \sim q_i(\cdot \mid Y_{<i})$, and let L be the number of accepted drafted tokens. Against a greedy target with continuation g , $\mathbb{E}L = \sum_k \prod_{i \leq k} q_i(g_i \mid g_{<i})$, and $\nabla_{\theta} \mathbb{E}L = \sum_j (\sum_{k \geq j} S_k) \nabla_{\theta} \log q_j(g_j \mid g_{<j})$ with $S_k = \prod_{i \leq k} q_i(g_i \mid g_{<i})$. Against a sampled target with the standard rule, with $A_k = \prod_{i \leq k} \min(1, p_i(Y_i)/q_i(Y_i))$,*

$$\mathbb{E}L = \mathbb{E}_Y \left[\sum_k A_k \right], \quad \nabla_{\theta} \mathbb{E}L = \mathbb{E}_Y \left[\sum_j \mathbf{1}\{q_j(Y_j) < p_j(Y_j)\} \left(\sum_{k \geq j} A_k \right) \nabla_{\theta} \log q_j(Y_j) \right].$$

Proof. Greedy: $L \geq k$ exactly when $Y_i = g_i$ for $i \leq k$, which has probability S_k , and $\mathbb{E}L = \sum_k \Pr(L \geq k)$; differentiate the product. Sampled: given the path, position k is reached and accepted with probability A_k , so $\mathbb{E}L = \mathbb{E}_Y \sum_k A_k$. Its gradient has a score part and a pathwise part. In the score part, $\mathbb{E}[A_k \nabla \log q_j(Y_j)]$ vanishes for $k < j$, because A_k depends only on $Y_{<j}$ and the score has conditional mean zero, leaving $\sum_j \nabla \log q_j \sum_{k \geq j} A_k$. In the pathwise part, $\nabla \min(1, p_j/q_j)$ is zero where $q_j < p_j$ and $-\min(1, p_j/q_j) \nabla \log q_j$ where $q_j \geq p_j$, which contributes $-\mathbf{1}\{q_j \geq p_j\} \sum_{k \geq j} A_k \nabla \log q_j$. The two parts add to the stated expression. $\square$

Freezing the acceptance probabilities as a fixed reward keeps only the score part and so gives a biased gradient. The proposition is a direct calculation and we claim no novelty for it. Acceptance-aware training already optimizes the greedy expected accepted length directly, with a window total-variation loss for sampling (Xia et al., 2026); BV Loss trains on the exact expected acceptance length under block verification (Kim et al., 2026b); D-PACE, PARD-2 and VAT weight a cross-entropy by position or by the chance of reaching it (Wu et al., 2026; An et al., 2026; Gu et al., 2026); variational speculative decoding optimizes a bound on sequence acceptance (Zou et al., 2026); TIGER uses the realized accepted length as a reinforcement-learning reward (Vo et al., 2026); and DBLAST drafts blocks with explicit dependence between positions (Karimi et al., 2026). The only trained DFlash selector we know of, LiLiCorr’s, uses cross-entropy over candidates (Rusanovsky et al., 2026). What remains for P6 is the combination: a DFlash 2-style selector over a frozen backbone, trained for rate rather than for accepted length, with the sampled-target gradient above; whether that pays is the experiment. The verification width must be chosen from committed-prefix features before proposals are drawn, for the reason of Appendix B.7; choosing a draft-tree budget from a profiled verify-cost curve at inference, as CaDDTree does for DFlash marginals (Zhang et al., 2026a), and confidence-scheduled width, as in DSpark (Cheng et al., 2026), are the existing ways to set it.

The cheapest test needs no training. From ordinary greedy runs, log the candidate sets C_i and the completed greedy continuation g , and compute $U = \max\{k : g_i \in C_i \text{ for all } i \leq k\}$, an upper bound on the accepted length of any selector over those frozen candidates; if U barely exceeds DFlash’s actual acceptance there is little to gain. Only then is the rate objective compared, at fixed width, with the strongest matched objective (VAT’s weighting ranks first in its own ablation (Gu et al., 2026); D-PACE-style weighting and position-weighted cross-entropy are the other controls) using the same backbone, candidates, selector size, features, data and budget. The pre-registered effect size is 1.25× end to end over the strongest matched baseline, rejected if the upper confidence bound falls below it; a gain that vanishes against an identical selector trained with another objective supports the selector, not the objective.

The screen does not kill P6 [E54]. On the block-16 trace of the first panel the drafter’s block was recomputed offline at each verify cycle’s anchor; the offline drafter agrees with the engine’s drafted token at 97.4% of positions, and its own acceptance matches the engine’s (6.200 against 6.204 tokens per cycle). Over the top 2, 4, 8 and 16 candidates per position the bound $1 + \mathbb{E}U$ is 7.51, 8.47, 9.33 and 10.14 tokens per cycle, against the engine’s 6.20; for chat it rises from 3.99 to 7.17. It is an oracle bound per cycle along the stock trajectory, not a bound on another selector’s tokens per cycle, since a better selector would reach other states, and it excludes the 207 of 21,274 cycles whose block runs past the end of the output, which accept less than average and so bias it slightly upward. The matched-objective comparison is [pending D27].

F.6 Exact repair of long windows (P2, P3)

A block drafter proposes a whole block in one forward pass and the target verifies it in one pass; the public DFlash card reports a mean accepted length of 7.48 tokens per verify pass at block size 16 on MATH-500 on a B200 (z-lab, 2026). While decoding is limited by weight traffic, a verify pass over a wider block costs little more than one over a narrow block, so the question is whether the target can commit far more than about seven tokens per pass without giving up exactness. Two proposals are tested with oracle experiments designed to kill them cheaply: long-window repair of a DFlash window by the target’s own predictions (P2), and target-anchored residual decoding (P3), which repairs a window by updating the target’s previous computation with fixed low-rank corrections and then audits it exactly. Both keep exactness by the same device: only tokens that an exact audit pass of the target has verified are committed, approximate states never enter the committed cache, and the noise of a position is never resampled.

Prior work Reusing the verifier’s work on rejected positions is an active line. ReTrace conditions the next DFlash block on the rejected trajectory (Lin et al., 2026), DFlow feeds the verifier’s hidden states at rejected positions back to the drafter (Zhang et al., 2026c), Carryover Drafting reuses those states as extra draft context (Koo et al., 2026), and CURE repairs uncertain positions of block-parallel drafts locally (Liu et al., 2026a); token recycling reuses candidate tokens from earlier steps (Luo et al., 2025), and CARD and H-Spec restructure parallel speculation around cached queries and drafter state (Zhou et al., 2025; Jiang et al., 2026). Jacobi-style iteration has been applied to translation, to feedforward computation, to image generation with probabilistic acceptance and to causal language models trained for it (Santilli et al., 2023; Song et al., 2021; Teng et al., 2025; So et al., 2026; Hu et al., 2026), and speculative speculative decoding overlaps drafting with verification (Kumar et al., 2026). P3’s anchored update is closest to computation reuse across similar inputs, which DeltaCNN, Eventful Transformers and CacheBlend apply to video frames, vision tokens and cached retrieval contexts (Parger et al., 2022; Dutson et al., 2023; Yao et al., 2025). None of these, as far as we found, commits a long window through an exact audit of a cheap residual evaluator of the target itself; that combination is what the kill tests below examine.

Setting and cost model Greedy decoding solves a triangular system: with $F_i(y_{<i})$ the target’s argmax after the committed prefix and $y_1, \dots, y_{i-1}$, the continuation satisfies $y_i = F_i(y_{<i})$. A verify pass evaluates F at every position of a candidate block from the committed prefix and commits the longest prefix with $y_i = F_i(y_{<i})$ plus the target’s token at the first mismatch. The commit is exact for any candidate, because every committed token is the target’s own decision on a verified prefix and the committed recurrent state and attention cache are those the target computed for that prefix. The Jacobi map $y_i^{(r+1)} = F_i(y_{<i}^{(r)})$ fixes at least one more position per application and reaches the solution within the block length; this is Jacobi iteration applied to decoding (Song et al., 2021; Santilli et al., 2023), and lookahead and consistency-trained decoders pursue related fixed-point iterations (Fu et al., 2024; Kou et al., 2024). For sampling, fixed per-position random inputs held across repairs give the target law, as in Theorem 4.5.

Let DFlash commit A_D tokens per cycle of cost C_D (draft, verification, state commit and control). A repair scheme that commits A_R tokens per attempt at cost $C_R = C_{\text{init}} + C_{\text{anchor}} + k C_{\text{repair}} + C_{\text{audit}} + C_{\text{state}} + C_{\text{control}}$, averaged over all attempts including failures and fallbacks, speeds decoding up by $S_{\text{decode}} = A_R C_D / (A_D C_R)$. Prefill and scheduling take a fraction f of a request’s time that repair does not change, so end to end $S_{\text{e2e}} = 1 / (f + (1 - f) / S_{\text{decode}})$; a $5\times$ end-to-end gain needs $S_{\text{decode}} = 5.44$ at $f = 2\%$ and 9 at $f = 10\%$ (derived). Two oracle bounds use a perfect candidate of width B (the reference continuation, or a forced full acceptance). With $V(B)$ the target verify pass and $C_{\text{state}}(B)$ the work needed to continue from the accepted boundary,

$$S_a(B) = \frac{B C_D}{A_D (V(B) + C_{\text{state}}(B))}, \quad S_b(B) = \frac{B C_D}{A_D (C_{\text{anchor}}(B) + C_{\text{audit}}(B) + C_{\text{state}}(B))}. \quad (14)$$

S_a is an ideal drafter paying one verify pass per block of B tokens; S_b is P3’s two-pass design with free repair, $C_{\text{anchor}}(B)$ a full pass that also writes the anchor cache and $C_{\text{audit}}(B)$ a full pass. If S_b cannot reach the target end to end, the two-pass design is dead whatever the repair does; if even S_a cannot, better drafting alone cannot reach it with this verifier. A concrete repair pays only if $A_R / C_R > A_D / C_D$, and it is rejected outright if a lower bound on its cost gives $C_R^{\text{lower}} / (B + 1) \geq C_D / A_D$, since no attempt commits more than $B + 1$ tokens.

What a repair must achieve Local low rank does not imply useful repair of a whole model. Three things must hold separately: the fixed cheap operators stay accurate after candidate tokens change, not only near the anchor; their errors

stay below the decision margins after attention, nonlinearities, the recurrent state and later layers; and each sweep resolves enough positions to pay for itself and for the audit. A sufficient condition links these quantities. Let m_i be the exact top-two logit margin at position i , ε_i a bound on the logit error the approximate evaluator makes at i when its inputs are exact, b_j a bound on the error in the current guess’s effect at an earlier position j ($b_j = 0$ once j is correct), and C_{ij} the influence of position j on the logits at i . If $2(\varepsilon_i + \sum_{j < i} C_{ij} b_j) < m_i$, position i is correct after the next sweep, by the triangle inequality. Two consequences follow. A persistent approximation error η in a recurrence with contraction factor $a < 1$ leaves an error floor of about $\eta/(1 - a)$. And a faithful evaluator can advance only as fast as exact Jacobi iteration, which on a copy chain $F_i(y) = y_{i-1}$ gains one position per sweep even with exact rank-one operators; a non-faithful map can jump ahead, as consistency-trained models do (Kou et al., 2024), but that would be better drafting, not repair. Expected accepted length is best read through hazards: with h_i the conditional failure rate at position i given an accepted prefix, $\mathbb{E}A = \sum_a \prod_{i \leq a} (1 - h_i)$. The decisive control is a small target-conditioned drafter given the same anchor information and matched in compute, memory traffic, training data, verifier width and target calls; if it does as well, the supported claim is better target-conditioned drafting, not cheap repair of a trajectory. This is analysis that fixes the measurements, not a result.

Test 1: perfect continuations at width B $V(B)$ and $C_{\text{state}}(B)$ are measured inside SGLang’s DFlash worker in the DFlash model card’s serving configuration (FlashInfer attention and GDN kernels, CUDA graphs, overlap scheduling, concurrency 1) at block widths $B = 16, 32, 64, 128, 256$, forcing every verify pass to accept the whole block; the committed tokens are then meaningless, but a pass’s cost does not depend on token values. CUDA events around each phase of the cycle give GPU times without host synchronization. SGLang’s GDN verify kernels write one FP32 state per verified position and the commit copies the accepted one back. At the pin the GDN verify kernel follows the decode backend unless overridden, so this configuration verifies with FlashInfer’s kernel, while SGLang’s default linear-attention backend, which the served arms of Appendix E.8 keep, verifies with a Triton kernel (code reading). The replay protocol, which keeps per-token inputs and folds the accepted prefix at commit, does not start with DFlash on this GDN model at the pin. The per-position state writes are measured at $B = 16, 64$ and 256 by a verify pass that drops them, and bounded by their bytes at a 3.0 TB/s write rate at the other widths. Feeding the reference continuation instead of forcing acceptance shows how often a verify pass at width B rejects the plain-decoding continuation; such rejections are the stock noise floor of Appendix D.4, not drafting errors. The measurement [E56] ran eight MATH-500 problems of 2,048 tokens at concurrency 1, with verification forced to accept the whole block through the engine’s simulated-acceptance setting, in the configuration of the drafter’s acceptance runs; foreign CPU load stayed below 0.6 cores. On the same problems DFlash at block 16 takes $C_D = 7.457$ ms per cycle and commits $A_D = 7.675$ tokens, and prefill and scheduling take $f = 2.28\%$ of a request, so a $5\times$ end-to-end gain needs $S_{\text{decode}} = 5.51$. The verify pass takes $V(B) = 4.78, 7.30, 15.62, 28.10$ and 44.82 ms at $B = 16, 32, 64, 128$ and 256 , and the state commit $0.11\text{--}0.14$ ms at every width (medians; the p10 and p90 cycle periods are within 1.3% of the median). An ideal free drafter would reach $S_a = 3.17, 4.19, 3.95, 4.41$ and 5.53 in decode, and $5\times$ end to end only at $B = 256$ (5.01); paying DFlash’s own drafting cost at width B , perfect blocks reach 4.68 end to end there. For P3, a ceiling that charges the anchor pass a single read of the 8.41 GB of weights at the measured 3.83 TB/s plus writing the anchor cache, and the audit its measured $V(B)$ and commit, reaches 5.24 in decode and 4.78 end to end at $B = 256$; an estimate that charges the anchor pass $V(B)$ less its measured per-position state writes, plus writing the anchor cache, gives 2.89 and 2.77. These are derived from the measured times.

With the audit charged the measured verify pass, P3’s two-pass design misses the $5\times$ target at every width even with free repair, so by its pre-registered rule it is refuted for this verifier, and P2 meets its target only at the edge of this test, at $B = 256$ with a drafter that costs nothing and every block accepted. Both verdicts are statements about the FlashInfer verify kernel, and two further runs at $B = 16, 64$ and 256 separate its parts [E57]. Dropping the per-position FP32 state writes, with an engine patch used for timing only, shortens the pass by 0.26, 1.26 and 4.10 ms, 5.3, 8.1 and 9.1% of $V(B)$, close to their bytes at 3.0 TB/s. These savings compare a pass from a later session with the Stage A session’s $V(B)$; the draft phase, which neither change touches, ran 1.8–2.6% shorter in the later session, so at $B = 16$ the difference is within that drift. An audit that dropped those writes and continued from the accepted boundary by an uncharged replay would therefore lift P3’s ceiling to 5.18 end to end at $B = 256$, where the target needs a saving of 2.33 ms; such a verifier would also make the DFlash baseline cheaper, since a block-16 cycle writes 16 per-position states. The larger cost is the FlashInfer kernel itself: selecting SGLang’s Triton GDN verify kernel instead, a stock option, makes the whole pass $2.06\times$ faster at $B = 64$ and $2.34\times$ at $B = 256$ at one request ($V(B) = 4.55, 7.58$ and 19.18 ms).

The Triton kernel is a different numerical path. DFlash at block 16 with it commits 7.603 tokens per cycle against 7.675, so its greedy outputs differ from the FlashInfer configuration’s, and every figure that uses it is pending an exactness classification against plain decoding. With the DFlash baseline on the same kernel, 7.302 ms per cycle and $f = 2.39\%$, so that $5\times$ end to

end needs 5.54 in decode, the derived speedups change both verdicts (Table 24). Perfect blocks reach $5\times$ end to end from $B = 64$, 5.21 paying DFlash’s drafting cost and 6.84 with a free drafter, and P3’s two-pass estimate passes at $B = 256$ (6.21), its ceiling already at $B = 64$. The baselines on the two kernels are single runs from two sessions, so the baseline’s own gain from the switch is not established; the wide blocks gain about two times. The Stage A verdicts therefore depend on the verify kernel. With the Triton kernel P2 turns on drafting, since a $5\times$ gain needs blocks of 64 or more tokens accepted almost entirely and the repair mechanisms tested here do not produce them (Tests 2 and 3); P3 stays refuted by Test 3 whatever the verifier. In per-position terms, a cycle drafted by DFlash at width B would have to commit 61.1 of 64 tokens at $B = 64$ or 132.2 of 256 at $B = 256$, which for a drafter with the same conditional acceptance at every position means at least 0.9985 or 0.9941, against 0.80 to 0.91 for the public drafter at block 16; with the FlashInfer kernel none of the measured widths, 16 to 256, suffices even with every block accepted (derived) [E58]. The drafter’s top-16 candidate sets still contain the target’s token at a conditional rate of 0.91 to 0.92 from the fourth position, but that rate is measured only at the cycle starts of the stock block-16 trajectory, so it bounds neither a selector, which would start its cycles elsewhere, nor a wide-block drafter. Past $B = 16$ the FlashInfer pass grows by about 0.167 ms per extra token, roughly five times what that token’s GEMM work and its 48 MiB per-position state write account for. A kernel trace locating the rest inside the pass, a same-session control of the three verify variants, and the acceptance of the reference continuation per width are [pending D28].

B	S_a , free drafter	Perfect blocks, DFlash drafting	S_b estimate	S_b ceiling
16	3.30 / 3.12	2.10 / 2.05	1.71 / 1.69	2.23 / 2.17
64	7.98 / 6.84	5.81 / 5.21	4.30 / 3.99	6.17 / 5.49
256	12.72 / 9.94	10.73 / 8.71	7.11 / 6.21	11.28 / 9.06

Table 24. Stage A with SGLang’s Triton GDN verify kernel, over DFlash at block 16 on the same kernel: speedup in decode / end to end. Derived from measured verify-pass times; the S_b estimate bounds this kernel’s state writes by their bytes. Pending an exactness classification of the kernel switch [E57].

Test 2: Jacobi repair from DFlash windows The second test asks how close exact repair from real DFlash windows gets, with no reference tokens. Every DFlash block $y^{(0)}$ is verified, and then k further full target passes run from the same committed prefix: *recycle* sweeps apply the Jacobi map, and *keep* sweeps replace only the first mismatching token by the target’s token and keep the rest of the draft. The pass on the original draft is then re-run and committed, so the trajectory is plain DFlash and each sweep’s accepted length is measured on real windows. Re-running a verify pass on the same prefix needs the committed GDN convolution window restored first, because the verify kernel shifts it in place, and the probe checks that the re-run reproduces the first pass’s argmax exactly. A sliding variant drafts each new block from the previous pass’s suffix predictions and fills the rest with DFlash.

Its one-step form has been measured offline on the drafter’s block-16 trace [E59]. At the 18,525 cycle boundaries after a rejection, a next block drafted from the previous pass’s target predictions accepts 0.52 drafts on average, against 4.44 for the fresh DFlash draft the engine used; keeping the previous draft’s tail gives 1.11, and choosing the best of the three with hindsight 4.55, 2.5% more than fresh drafting. At block 8 the figures are 0.45 and 1.15 against 3.21 (3.30). The target’s prediction after a wrong draft token matches the continuation 14% of the time once that token is corrected, against 84% for the drafter’s first drafted token. Recycling the target’s suffix predictions therefore does not improve on fresh drafting. Full recycle sweeps advance at the rate of the copy chain above: on the 80 held-out blocks of Test 3, exact Jacobi from the DFlash draft accepts 2.98 drafts before any sweep and 4.33, 5.35, 6.38, 7.34 and 8.30 after one to five full target passes, about one token per pass [E60]. Accepted drafts against the number of full sweeps in the engine, recycle and keep, per block width, and P2’s verdict are [pending D29].

Test 3: anchored residual evaluation For every linear operator $z = Wx$, P3 keeps the anchor $(\bar{x}, \bar{z} = W\bar{x})$ from a full pass over a DFlash block and repairs a changed block with $\tilde{z} = \bar{z} + (WU)U^T(x - \bar{x})$, where the orthonormal basis U is fixed per operator input and WU is precomputed, so a repair sweep streams WU instead of W ; a basis computed from the current correction would stream W again. Nonlinearities, normalizations, the causal convolution, the GDN recurrence from the exact committed state and attention over the exact prefix run exactly on the repaired values. The measurement [E60] ran the Hugging Face implementation of the target (BF16, pinned revision) block by block from an exact prefix cache; its argmax agrees with the engine’s verify argmax at 99.1% of the development blocks’ positions. Blocks are real block-16 DFlash drafts with a rejection, from the drafter’s shared trace: 120 development blocks from even-indexed requests, on which each operator’s basis is fitted as the leading principal components of its exact input changes, and 80 held-out blocks from odd-indexed requests. Every linear operator is repaired (the GDN input and output projections, the attention projections, the

MLP and the tied head) at ranks 16 to 512; rank 0, which reuses the anchor outputs unchanged, is the initializer without repair. Each held-out block is replayed twice against the original anchor, $y^{(1)} = F(y^{(0)})$ and $y^{(2)} = F(y^{(1)})$, and at every position the test records whether $\arg \max \tilde{z}_i = \arg \max z_i$ and the ratio $R_i = 2\|\tilde{z}_i - z_i\|_\infty / m_i$ ($R_i < 1$ guarantees the same decision). A free-running repair from $y^{(1)}$ is audited exactly after every sweep and compared with exact Jacobi iteration from the same anchor.

The first of the three conditions fails, and the other two fail with it. On held-out exact changes the median relative error $\|W(I - UU^\top)\Delta x\| / \|W\Delta x\|$ of the first projection that reads each operator input is 0.76–0.90 at rank 16, 0.64–0.81 at rank 128 and 0.51–0.73 at rank 512, by operator class, and even on the development blocks rank 128 captures a median of only 51–64% of the change energy: the changes that token corrections cause are not confined to a small fixed subspace. On the first replay the repaired argmax equals the exact one at 35% of the changed positions at rank 0, 37% at rank 128 and 44% at rank 512, and at the corrected position itself at 11–14%; on the second replay at 24–35%. $R_i < 1$ at 43 of the 921 positions of the first replay where it is defined (4.7%; the other 41 are exact ties of the exact logits, with margin 0), the same positions at every rank including rank 0; these have large exact margins (median 12 logits, against 1.3 over all replayed positions), where even the anchor’s logits stay within half the margin, so the repair earns no certificates, and the median R_i is 13–16. Free-running repair accepts 4.33 drafts before its first sweep and 4.33–4.44 after four sweeps at every rank, at most 0.11 more than no repair, while exact Jacobi from the same anchor gains about one token per sweep (Test 2), which also bounds any faithful cheap evaluator.

The economic gate compares committed tokens per unit cost with DFlash at block 16, charging repair sweeps a lower bound that counts only their bytes and the audit its measured $V(16)$ and commit (derived). The anchored evaluator reaches at most 0.28 of DFlash’s rate at any rank and sweep count. Exact Jacobi with each sweep charged only the bytes of reading the anchor cache and the GDN state, a faithful evaluator at that cost, reaches 0.27, 0.47, 0.66, 0.85 and 1.03 after zero to four sweeps, so even it would at best match DFlash after four sweeps. The bound of the cost model, the audit’s cost over the most tokens an attempt can add, does not reject at $B = 16$; the measured progress does. The pre-registered control against a compact drafter with the same storage and compute was not run: the evaluator gains 0.11 accepted drafts over no repair, which leaves nothing for that control to explain. P3 is refuted at the operator level, whatever the verifier.

E.7 Reusing a cached window after a rejection (P9)

P9 proposes, after a rejection, to condition a program compiled from the first cycle’s cached candidate sets on the corrected prefix and to propose the rest of the old window from it, at most once, instead of drafting afresh. Its pre-registered rule compares the two over two cycles,

$$\Delta = \mathbb{E}[G_2^R - G_2^F] - r_F \mathbb{E}[A + T_2^R - T_2^F], \quad r_F = \frac{\mathbb{E}[G_1 + G_2^F]}{\mathbb{E}[T_1 + T_2^F]}, \quad (15)$$

with G the tokens a cycle commits, T its time, R and F reuse and fresh drafting in the second cycle and A the extra cost reuse adds to the first, and rejects the reuse at the tested configuration if the upper end of Δ ’s 95% interval is at most 0.

Its first test is a support oracle at concurrency 1 [E61]. On the drafter’s block-16 trace (21,067 cycles of 80 requests), block 16 being the fastest DFlash block at concurrency 1 on the serving workload (Table 22), there are 18,274 boundaries after a rejection. P9’s program conditions on the whole corrected prefix through the frozen candidate sets of the old block, so reuse is possible when those top- K sets contain every token of the corrected prefix and some of the old window remains, which is known at decision time; the oracle then proposes the true token wherever the old candidate set contains it, charges a reused cycle the measured fresh cycle less its 2.33 ms draft phase, that is a full block-16 verify padded beyond the remaining positions, and takes A , the cost of conditioning and the draft-cache catch-up as zero, so it bounds from above P9’s program over the same candidate sets when that program verifies a padded block-16 and reuses at every boundary where reuse is possible. With the top-16 sets the corrected prefix is supported at 88.8% of the boundaries, the oracle commits 5.08 tokens in the second cycle against 5.55 for fresh drafting on those boundaries, and $\Delta = +0.99$ tokens per boundary (95% interval 0.89 to 1.07). With the top-8 sets $\Delta = +0.22$ (0.12 to 0.32), and with the top-1, top-2 and top-4 sets the rule rejects reuse. Keeping the old draft’s tail without conditioning, the unchanged cached control, gives $\Delta = -1.93$ and is rejected. These verdicts, the rejections included, hold for P9’s program over the recomputed candidate sets, with a padded block-16 verify and reuse at every boundary where reuse is possible. A program whose support starts at the correction, conditioning on the suffix alone, is a different class and is not covered; with recomputed sets it could also reuse at boundaries where the recomputation drops an already accepted token. The engine’s own sets, whose top-1 token differs from the recomputation’s at 2.6% of positions, were not captured, and a program over them is not covered. At concurrency 1 the oracle bound therefore does not reject

reuse with top-8 and top-16 sets. It is an upper bound, not a served speed-up: the candidate sets are an offline recomputation whose top-1 token matches the engine’s draft at 97.4% of positions, the costs are one run’s phases at concurrency 1, and a real program must select the token, which the old draft’s own choices fail to do (the cached control), while its compile, conditioning, retention and draft-cache catch-up costs come out of the same margin.

Three further bounds over the same boundaries and bootstrap resamples widen the class of programs, and none of them is a result about a real program [E61]. A program that verifies only the remaining positions at a smaller width could pay less for its second cycle than the padded 4.73 ms verify the oracle charges. Charging the reused verify nothing, its lower bound, and every other phase at its block-16 cost gives $\Delta = +0.01$ (0.00 to 0.02), +0.89 (0.81 to 0.98), +1.88 (1.75 to 2.05), +2.89 (2.71 to 3.10) and +3.86 (3.64 to 4.11) with the top-1, top-2, top-4, top-8 and top-16 sets. This bounds from above P9’s program reusing at every boundary where reuse is possible, at any verify width whose other phases cost at least their block-16 values in the measured fresh block-16 run (0.395 ms per cycle beyond draft and verify), and it rejects none of the tested sets, so the rejections of the top-1, top-2 and top-4 sets do not extend to a narrower verify. A narrower cycle can also cut those phases, to 0.384 ms at block 8, so it is not a bound for every implementation. On the point estimates the Δ of the top-1, top-2 and top-4 sets stays negative only while a reused verify takes more than 0.89, 2.71 and 3.87 ms, and the only verify measured below width 16 at concurrency 1, 4.16 ms at width 8, does not settle it. A program that chooses between reuse and fresh drafting at each boundary could avoid the boundaries where reuse loses. An oracle gate that knows fresh drafting’s outcome and takes the better of the two at each boundary where reuse is possible has a Δ of at least 0, and at least the Δ of always-reuse, which reuses at every such boundary, by construction, so it rejects nothing: +0.01 (0.01 to 0.01), +0.45 (0.42 to 0.49), +0.91 (0.85 to 0.98), +1.44 (1.35 to 1.53) and +2.05 (1.93 to 2.18) with the same five sets. It bounds from above P9’s program with any gate over the same sets with a padded verify, and its Δ is the bound to quote for such programs. Its excess over always-reuse, 1.06 to 1.33 tokens per boundary with the top-2, top-4, top-8 and top-16 sets, is only the gain from gating the oracle’s reuse arm: a real program’s reuse arm is no better than the oracle’s, so gating it can gain more, and the excess does not bound that gain. The same gate with the reused verify free bounds P9’s program with any gate at any verify width, under the same condition on the other phases: +0.04 (0.03 to 0.05), +1.47 (1.35 to 1.61), +2.54 (2.35 to 2.76), +3.47 (3.24 to 3.73) and +4.35 (4.09 to 4.64). A real gate decides before the second cycle without seeing fresh drafting’s outcome, so it pays off only if what is known at decision time (the correction position, the number of old positions left and the old sets’ probabilities at them) predicts the boundaries where its own reuse arm loses to fresh drafting; the length of the supported suffix is not known then, because it depends on the greedy tokens after the correction. A conditioned program and a matched autoregressive refiner over the same cached information, each against Equation (15) with real progress and all costs charged, the verify time at widths 2 to 16 and the draft shares at concurrency 8 and 16 are [pending D30].

E.8 Verifying the recurrent state (P7b, P10, P11)

Three proposals concern how a verify pass treats the recurrent state. P7 has two parts. P7a, the five exactness contracts and the witnesses that separate them, frames every exactness claim in this paper (Appendix D.1). P7b would verify a draft block’s GDN outputs in parallel, through the triangular form of the chunked update instead of the sequential recurrence, emit an output where its sound enclosure lies in one BF16 rounding cell and recompute it otherwise, and commit the accepted state by the rounding-preserving replay of Appendix E.5. Its first rejection test is negative for SGLang’s chunked kernel, which is not a usable fast path: it disagrees with the recurrent outputs in most BF16 words and is slower at every width (Appendix E.4; synthetic inputs, one request). The certified path itself, against strict replay on a captured trace, is [pending D18].

P11 would predict after the first target layers how many draft positions each request will accept, run the remaining layers only on that prefix, and resume the same block when the prediction was short; the predictor would control work, never tokens. It was dropped by its author in favour of P10, because verify kernels keep the recurrent state in registers, so shortening the verified suffix does not remove per-token full-state reads (reasoning). P10 instead replays the previous cycle’s accepted tail inside the next verify kernel and checkpoints the state after the anchor token. Its first test, the share of the speculative cycle spent in SGLang’s exact GDN fold at concurrency 8 and 16, is [pending D19]; P10 is built only if that share is at least 9.09%.

E.9 Selector and depth analysis not tested here

Three pieces of analysis bear on the proposals above; none is under test.

Executing the DFlash 2 selector At the inspected implementation adjacent candidate scores have the form

$$S_t(a, b) = U_t(b) + \sum_{r=1}^{d_s} A(a)_r H(h_t)_r B(b)_r, \quad (16)$$

with a gate that depends on the draft hidden state (Inco AI, 2026a; SGLang contributors, 2026c). A greedy walk needs only the realized row $S_t(x_{t-1}, \cdot)$, which reduces score work from $O(BLK^2 d_s)$ to $O(BLK d_s)$ at the price of a dependent sequence of small contractions. Alternatively the lattice can emit, for every predecessor, its winning successor $f_t(a) = \arg \max_b S_t(a, b)$, after which the path is $x_t = (f_t \circ \dots \circ f_1)(x_0)$.

Proposition F.2 (Finite-map execution). *Given the same row winners, serial map walking and prefix composition produce the same path. A work-efficient scan of length L over K -entry maps uses $O(LK)$ map work and $O(\log L)$ composition stages, excluding the score producer.*

Proof. The t -th prefix composition applies exactly the first t transitions to the anchor, and function composition is associative. A work-efficient tree scan uses $O(L)$ compositions of $O(K)$ indexed evaluations each, with logarithmic depth. $\square$

Function scanning is standard; the open question would be where it pays. The exact rational reference implements the simpler $O(LK \log L)$ Hillis–Steele scan and checks path equivalence only (Table 6).

Prefix value on a fixed lattice With a substochastic surrogate $r_t(a, b)$ for the probability of matching the next target token, $J_r(x) = \sum_{j=1}^L \prod_{t \leq j} r_t(x_{t-1}, x_t)$ is maximized by the recursion $V_t(a) = \max_b r_t(a, b)(1 + V_{t+1}(b))$ with $V_{L+1} = 0$. If the estimate $\hat{r}$ also lies in $[0, 1]$ and edge errors are uniformly bounded by ϵ_t , telescoping gives $|J_r - J_{\hat{r}}| \leq \delta = \sum_t (L - t + 1)\epsilon_t$ and optimizer regret at most 2δ . LiLiCorr’s finding that an exact best-path search over a different score decodes worse than the greedy rule (Rusanovsky et al., 2026) is the relevant warning.

Choosing depth under a cost model If predicted progress $F_i(d)$ is known and cost depends only on the total optional depth m , the depth assignment for a batch is solved by

$$G_i(m) = \max_{0 \leq d \leq L_i} \{G_{i-1}(m - d) + F_i(d)\}, \quad \max_m \frac{G_B(m)}{C(m; z)}, \quad (17)$$

which is exact only under that cost hypothesis. Published policies choose depth in several ways: OPT-Tree searches for the tree that maximizes expected acceptance (Wang et al., 2025), Sequoia sizes trees for the hardware (Chen et al., 2024), SpecDec++ stops drafting with a learned acceptance head (Huang et al., 2025), DSpark schedules verification from confidences and a profiled throughput curve (Cheng et al., 2026), D-cut prunes draft blocks against a profiled cost table aligned with the engine’s CUDA-graph shapes (Liu et al., 2026b), and an open SGLang pull request ports the DSpark scheduler to DFlash 2 with greedy benchmarks (SGLang contributors, 2026b). CUDA-graph tiers make the cost discontinuous: with rewards (1, 1.9, 2.7) and costs (1, 2, 2) at depths (0, 1, 2), depth one looks worse than depth zero while depth two is best, so stopping at the first local decline fails. Autotuning a kernel that mutates state must restore that state between trials (Triton contributors, 2026). Any such policy must choose depth from committed-prefix information when sampling (Appendix B.7).

G EVIDENCE REGISTER, PENDING RESULTS AND REPRODUCTION

G.1 Evidence register

Every committed result the paper cites is listed below with the claims it supports, the files that hold it and the program that produced them. All paths are relative to the repository at <https://github.com/SamMausberg/verified-progress>. The exact command behind each evidence file is in the README.md of the file’s directory, and the files record the commits of this repository and of the engine that produced them where the generator records them. The text cites an entry as [En]. Proved statements are not listed; their proofs are in the text and Appendix A.10 says which are machine-checked.

The Work a Verifier Needs

Entry	What it supports; files and the program that produced them
<i>The certified head on captured engine states</i>	
E1	Captured plain-decode head inputs reproduce the engine’s tokens; R-real and R-stock differ at about 0.5% of greedy positions (Section 3.1) evidence/head_geometry/alignment_plain4b.json; produced by experiments/head_geometry/validate_alignment.py.
E2	Candidate sets, envelope checks, head-input norms, batch unions, FP32 re-scoring and cascades on 6,005 held-out plain-decode positions (Section 4.4, Appendix A.7) evidence/head_geometry/selfevidence_plain4b.json; produced by experiments/head_geometry/analyze_selfevidence.py.
E3	Candidate counts per cheap head, envelope, accumulation model and decision evidence/head_geometry/selfevidence_plain4b.csv; produced by experiments/head_geometry/analyze_selfevidence.py.
E4	Share of positions needing at least k candidate rows (Figure 3) evidence/head_geometry/selfevidence_plain4b_ccdf.csv; produced by experiments/head_geometry/export_candidate_ccdf.py.
E5	Row norms, top-two margins, top-token mass, envelope width against realized error evidence/head_geometry/stats_plain4b.json; produced by experiments/head_geometry/analyze_stats.py.
E6	Rows and batches left to the stock kernel by rule and error model; no certified token differed from the engine’s (Table 4) evidence/head_geometry/rstock_plain4b.json; produced by experiments/head_geometry/analyze_rstock.py.
E7	Candidate sets on DFlash-4B verify and draft rows evidence/head_geometry/selfevidence_dflash4b.json, selfevidence_dflash4b.csv, selfevidence_dflash4b_ccdf.csv; produced by experiments/head_geometry/analyze_selfevidence.py.
E8	Candidate sets and fallback shares on 4,010 MTP-4B verify rows and 4,024 draft rows (Appendix A.7) evidence/head_geometry/selfevidence_mtp4b.json, selfevidence_mtp4b.csv, selfevidence_mtp4b_ccdf.csv; produced by experiments/head_geometry/analyze_selfevidence.py, export_candidate_ccdf.py.
E9	Kill test of the certified decoder tail, P1 (Appendix A.9) evidence/head_geometry/tail_killtest.json; produced by experiments/head_geometry/tail_killtest.py.
<i>Transport</i>	
E10	Weight-only constants and the crossover thresholds δ^* (Table 7) evidence/precision/head_constants.json; produced by experiments/precision_head_constants/head_constants.py.
E11	Drift ratio on 40,000 DFlash-4B pairs, by outcome, position, domain and context, with the per-row thresholds (Figure 4, Table 8) evidence/head_geometry/rho_dflash4b.json, rho_dflash4b_quantiles.csv, rho_dflash4b_pairs.csv; produced by experiments/head_geometry/analyze_rho.py.
E12	Transport on DFlash-4B by bound family and tiling: skips, partition widths, P_7 , batch unions, drift scaling (Tables 5, 10 and 11) evidence/head_geometry/transport_dflash4b.json, transport_dflash4b.csv; produced by experiments/head_geometry/analyze_transport.py.
E13	The same on 16,016 MTP-4B pairs, with their drift evidence/head_geometry/transport_mtp4b.json, transport_mtp4b.csv; produced by experiments/head_geometry/analyze_transport.py.
E14	Drift ratio on 40,000 aligned MTP-4B pairs, with the per-row thresholds (Figure 4) evidence/head_geometry/rho_mtp4b.json, rho_mtp4b_quantiles.csv, rho_mtp4b_pairs.csv; produced by experiments/head_geometry/analyze_rho.py.
E15	Draft and target head-input statistics; alignment of the DFlash-4B capture (Appendix C.5) evidence/head_geometry/stats_dflash4b.json, alignment_dflash4b.json; produced by experiments/head_geometry/analyze_stats.py, validate_alignment.py.

The Work a Verifier Needs

Entry	What it supports; files and the program that produced them
E16	Draft and target head-input statistics; alignment of the MTP-4B capture and its outputs against plain decoding (Appendix C.5) evidence/head_geometry/stats_mtp4b.json, alignment_mtp4b.json; produced by experiments/head_geometry/analyze_stats.py, validate_alignment.py.

Exact references and formal checks

E17	Exact floating-point reference: 21 test methods, 40,015 checks, the R-bf16, split- K and noise-stream witnesses (Appendix A.10) evidence/precision/precision_tests.json, precision_tests.log; produced by tests/test_precision.py.
E18	Lean 4.19.0 elaboration of the two formal files (Appendix A.10) formal/STATUS.md, evidence/precision/lean_check.log; produced by scripts/check_lean.sh.
E19	Exact rational reference for decisions and races (Table 6); the replay witness of Proposition D.2 evidence/decision_tests.json, race_tests.json and their logs; produced by tests/test_decisions.py, tests/test_races.py, via scripts/verify_artifact.py.
E20	Work counts on constructed heads (Figure 5) data/synthetic_drift.csv, evidence/synthetic_drift.json; produced by experiments/synthetic_drift.py.
E21	Witnesses that separate the exactness contracts; the regrouped recurrence; the transition norm (Appendix D.1) evidence/contracts/contract_witnesses.json, contract_witnesses.log; produced by tests/test_contracts.py.
E22	Witnesses for the structure of the recurrent state and for sharing across requests (Appendix E.5) evidence/state_structure/state_structure_tests.json, state_structure_tests.log; produced by tests/test_state_structure.py.

The stock engine

E23	First differing module outputs and divergence classes under both error models (Figure 6, Table 14) evidence/state_safety/mechanism_plain_c1_vs_mtp_s3_c1.json, mechanism_plain_c1_vs_c32.json, first_difference_by_module.csv, divergences.csv; produced by experiments/state_safety/mechanism.py, analyze_all.sh, compare.py.
E24	Divergence rates, BF16 top-two ties, same-server and fresh-server repeats and the deterministic-mode checks; the server settings of every run evidence/state_safety/noise_floor.csv, noise_floor.json, run_meta.json; produced by experiments/state_safety/run_all.sh, compare.py.
E25	Positive controls of the bit-exact tap; the method; the tap-changed prompts and their module signature in two tapped sessions evidence/state_safety/tap_control_down9.json, tap_control_core13_decode.json, tap_control_core13.json, tap_check.json, tap_signature.json, README.md; produced by experiments/state_safety/mechanism.py, analyze_all.sh, tap_signature.py.
E26	Cache-level checks with the cache-hashing tap at batch 1: plain decoding against MTP with three steps (40 prompts), the two history pairs at the level of the KV, the prefix cache on against off (40 prompts), one prompt's same-server repeats with and without a flush, and the tapped runs against untapped ones (Appendices D.4 and D.5) evidence/state_safety/cachecheck_v4_plain_c1_vs_mtp_s3_c1.json, cachecheck_v4_plain_c1_radix_vs_noradix.json, history_v4_mt_bench-0056.json, history_v4_humaneval-0008.json, repeats_v4_h44_flush.json, repeats_v4_h44_noflush.json, README.md; produced by experiments/state_safety/mechanism.py, from the tapped sessions of run_tap_v4.sh (tap_runs.py).
E27	The pools each server allocated for every comparison of Appendix D.4, with whether the two servers were the same and the pools identical evidence/state_safety/pools.json, run_meta.json, noise_floor.csv; produced by experiments/state_safety/pools.py, analyze_all.sh.
E28	The matrix with pinned pools: divergence rates and classes for plain and MTP pairs with identical pools, self-consistency, the pools and token counts of every run, divergences per fragile position by the previous cycle's commit length with the first cycle after prefill, and the pinned against unpinned batch-1 comparison (Appendices D.4 and D.5) evidence/state_safety/noise_floor_pinned.json, noise_floor_pinned.csv, divergences_pinned.csv, run_meta_pinned.json, cycles_mtp_s1_pinned.json, cycles_mtp_s3_pinned.json, cycles_mtp_s5_pinned.json, cycles_mtp_tree_pinned.json, cross_regime.json, README.md; produced by experiments/state_safety/analyze_all.sh, compare.py, cycles.py.

The Work a Verifier Needs

Entry	What it supports; files and the program that produced them
E29	Targeted state tests: truncation and stop tokens inside MTP verify cycles and for the tree and plain decoding, warm against cold continuation, and history dependence through the prefix cache (Appendices D.4 and D.5) evidence/state_safety/targeted.json; produced by experiments/state_safety/run_targeted.sh, targeted.py, summarize_targeted.py.
<i>Where the time goes</i>	
E30	Attribution of the plain step and the MTP cycle by component; unprofiled throughput; the state-slot reservation (Tables 17, 18 and 19) evidence/profiles/tables.md, breakdown.csv, step_share.csv, label_structure_check.json, attribution/, windows/; produced by experiments/profiling/run_profiles.py, attribute.py, summarize.py.
E31	The head chain’s share of a plain step and of an MTP cycle; the head microbenchmark and the kernels cuBLAS selects evidence/profiles/head_microbench.json, head_microbench_kernels.json, attribution/; produced by experiments/profiling/attribute.py, head_microbench.py.
E32	Read bandwidth: 3.83 TB/s over 4 GiB for the best of nine launch configurations, 3.79 TB/s over the head’s footprint with that configuration evidence/profiles/hbm_bandwidth.json; produced by experiments/profiling/hbm_bandwidth.py.
E33	Bytes per step by component, checked against measured kernels; bytes per output token of plain decoding and MTP over the batch size evidence/profiles/bytes_per_step.json, bytes_per_step_sweep.csv, bytes_per_step_sweep_wide.csv; produced by experiments/profiling/bytes_model.py.
E34	Host functions running while the GPU idles in the MTP cycle evidence/profiles/diagnostics/; produced by experiments/profiling/host_gaps.py, pyspy_summary.py.
E35	Share of layer 0’s GDN input projections in a plain step (P5) evidence/profiles/p5_layer0_in_proj.json; produced by experiments/profiling/layer0_share.py.
E36	Derived ceilings per lever stack under both execution models (Table 20, Figure 7) evidence/moonshot/ceilings.json, ceilings.csv; produced by experiments/moonshot/ceilings.py.
E37	First served lever measurements, single runs evidence/moonshot/lever_sweeps_quick.csv; produced by experiments/moonshot/lever_sweep.py, levers.py, summarise.py.
E38	First engine-only decode-step times by state precision evidence/moonshot/decode_ceiling_try1.csv; produced by experiments/moonshot/decode_ceiling_sweep.py.
E39	Engine patches for the reduced draft head, the relaxed greedy rule and rounding-preserving replay, with their tests; code, not results engine/sglang/patches/moonshot/, tests/test_moonshot_levers.py, tests/test_gdn_exact_replay.py.
E40	Rounding-preserving replay (P4) against the packed decode, per word, and its kernel time by ring length (Appendix E.5) evidence/moonshot/gdn_exact_replay_check_L4.json, gdn_exact_replay_check_L16.json, gdn_exact_replay_check_L4_layer20.json, gdn_exact_replay_bench.json; produced by experiments/moonshot/gdn_exact_replay_check.py.
E41	P4’s served test: the design, metric and decision rule declared before its first run, the prompt check corrected after it, the three void attempts, the cause of their 127-request plateau checked against every admission wave, and the amendments declared before each further run (declarations and a diagnosis, not results) evidence/moonshot/README.md, p4_admission_plateaus.csv; produced by experiments/moonshot/run_p4b.sh, run_p4_admission.sh, validate_p4_ab.py, output_probe.py, check_admission.py, admission_plateaus.py.
E42	Triton attention for the MTP cycle’s target and draft at concurrency 1 and 4, single runs evidence/moonshot/host_levers.csv, README.md; produced by experiments/moonshot/lever_sweep.py, levers.py, summarise.py.

The Work a Verifier Needs

Entry	What it supports; files and the program that produced them
E43	Backbone weight GEMMs and RMSNorm in isolation: the kernels cuBLAS runs per projection and M , faster kernels reading the same weights and their bitwise agreement with cuBLAS, the packed GDN input projection, the norm and SiLU prologues, layer skeletons and each run’s foreign CPU load; the opt-in engine patch series is code, not a result (Appendix E.6) evidence/backbone/gemm_microbench.json, merge_microbench.json, norm_microbench.json, chain_microbench.json, tables.md, README.md, hostload/, engine/sglang/patches/backbone/; produced by experiments/backbone/gemm_bench.py, summarize.py, make_table.py.
E44	Host-side idle in the MTP and DFlash cycles: stock host traces, the derived untraced idle, plan and attention-output equivalence of the sync-free plans, in-engine validation, greedy output equality with stock, and untraced cycle times before and after (Appendix E.6) evidence/hostgap/README.md, stock_traces_hold1.json, cycle_profiles.json, plan_equivalence.json, equality/, engine/sglang/patches/hostgap/; produced by experiments/hostgap/profile_arms.sh, gap_analysis.py, hold_equality_profiles.sh, plan_equivalence.py, equality.py, summarize.py.
E45	SGLang’s chunked GDN kernel as a block-parallel verify path: BF16 disagreement and time by block width evidence/moonshot/p7_fast_verify_check.json, p7_verify_width_bench.json; produced by experiments/moonshot/gdn_fast_verify_check.py.
E46	Coverage of hot draft-vocabulary maps, built on the tuning split, on the confirmation split’s plain outputs evidence/moonshot/token_map_coverage.csv, README.md; produced by experiments/moonshot/token_map_coverage.py, bench/token_map.py.
E47	Byte and FLOP decomposition of the MTP cycle at concurrency 128, from the tuning points (derived) evidence/moonshot/README.md, evidence/bench/tuning/points.csv.
<i>Drafting and serving</i>	
E48	Command, environment and resolved settings of the drafter’s acceptance runs evidence/drafter/launch/; produced by experiments/drafter/run_trace.sh.
E49	DFlash acceptance by block position and domain (Table 23, Figure 8) evidence/drafter/acceptance_summary.csv, acceptance_by_position.csv; produced by experiments/drafter/summarize_acceptance.py.
E50	Acceptance by text segment and the class of the first rejected draft evidence/drafter/trace_segments_b16.csv, trace_segments_b16_failures.json (and the block-8 files); produced by experiments/drafter/trace_analysis.py.
E51	Card-reproduction gate on the 80 MT-Bench first turns with the card’s settings; chat acceptance by prompt source evidence/drafter/card_gate_mtbench.json, README.md, evidence/drafter/launch/zlab_b16_card_gate.json; produced by experiments/drafter/run_card_gate.sh.
E52	DFlash and three-step MTP acceptance on the second panel, by position and domain evidence/drafter/panel_v2/acceptance_summary.csv, acceptance_by_position.csv, evidence/drafter/launch/panel_v2_zlab_b16.json, panel_v2_mtp3.json, panel_v2_plain.json; produced by experiments/drafter/run_equality.sh, summarize_acceptance.py.
E53	Divergences of DFlash and MTP outputs from plain decoding, with the plain run’s margins evidence/drafter/panel_v2/equality_zlab_b16.json, equality_mtp3.json; produced by experiments/drafter/run_equality.sh, compare_outputs.py.
E54	The P6 candidate-support screen: oracle bound per cycle for the top 2 to 16 candidates evidence/drafter/support/zlab_b16_panel_v1_summary.json, zlab_b16_panel_v1_survival.csv; produced by experiments/drafter/support_screen.py.
E55	A training prompt set disjoint from every benchmark and quality set evidence/drafter/data/prompts-v2.manifest.json; produced by experiments/drafter/build_train_prompts.py.
E56	Repair Stage A: verify-pass, commit and draft times per block width with forced full acceptance; the DFlash baseline; derived oracle speedups S_a and S_b (Appendix F.6) evidence/repair/stage_a_timing.json, stage_a_oracle.json, stage_a_oracle.csv, README.md; produced by experiments/repair/analyze_timing.py, stage_a.py, experiments/repair/runs/stage_a.sh.

The Work a Verifier Needs

Entry	What it supports; files and the program that produced them
E57	Verify-pass decomposition at $B = 16, 64$ and 256 : per-position state writes dropped, SGLang’s Triton GDN verify kernel, and Stage A recomputed on it (pending exactness classification) evidence/repair/stage_a_timing.json, stage_a_oracle_triton.json, stage_a_oracle_triton.csv, stage_a_oracle.csv, README.md; produced by experiments/repair/runs/decomposition.sh, experiments/repair/analyze_timing.py, stage_a.py.
E58	The per-position acceptance a block drafter would need for $5\times$ end to end at concurrency 1, from Stage A’s cycle periods under both verify kernels and the block-16 trace’s acceptance and top-16 support by position (derived, no new runs; Appendix F.6) evidence/drafter/drafting_requirement.json, README.md; produced by experiments/drafter/drafting_requirement.py.
E59	One-step recycling of the target’s suffix predictions against fresh DFlash drafts evidence/repair/one_step_recycling.json, draft_source_accuracy.csv; produced by experiments/repair/analyze_jacobi.py.
E60	Repair Stage B: the anchored residual evaluator on real block-16 DFlash blocks (operator error, decision agreement, R_t , progress per sweep, exact Jacobi) and the economic gate evidence/repair/residual_eval_b16.json, residual_eval_b16.agree_by_distance.csv, p3_gate_b16.json, README.md; produced by experiments/repair/runs/residual_b16.sh, experiments/repair/residual_eval.py, summarize_residual.py, p3_gate.py.
E61	P9 support oracle at concurrency 1: supported rates, oracle and fresh second-cycle commits and Δ with bootstrap intervals per candidate-set size, the unchanged cached control, the bound with a free reused verify and the omniscient-gate oracle with a padded and a free verify (upper bounds, not served results) evidence/repair/p9_support_oracle.json, README.md; produced by experiments/repair/p9_support_oracle.py.
E62	The frozen serving workload: sources, revisions, splits and token statistics bench/workloads/mixed-v2/manifest.json; produced by bench/build_workload.py.
E63	Where greedy outputs stop on 576 tuning prompts evidence/bench/workload/natural_lengths_tune.json, natural_requests_tune.csv; produced by bench/sweep.py, bench/lengths.py.
E64	Feasibility probes of capacity and draft trees (what launches, not performance) evidence/bench/probes/; produced by bench/campaigns/capacity_probes.sh, bench/pareto.py.
E65	Frontend diagnosis at concurrency 256 on a quiet host (single points) evidence/bench/frontend/; produced by bench/campaigns/frontend_diagnostic.sh, bench/pareto.py.
E66	Configuration search on the tuning split, one launch per configuration, and the chosen arms (Table 22); derived snapshot traffic evidence/bench/tuning/points.csv, frontier.csv, launches.csv, evidence/bench/README.md, bench/arms.toml; produced by bench/campaigns/tuning_depth.sh, tuning_knobs_dflash.sh, tuning_backend_dflash.sh, bench/pareto.py.
E67	Confirmed serving frontier: nine tuned arms on the confirmation split, three sessions, paired ratios, run-to-run variance and the envelope over points with at least three sessions evidence/bench/confirm/frontier.csv, envelope.csv, envelope-all.dat, pairs.csv, points.csv, launches.csv, evidence/bench/README.md; produced by bench/campaigns/confirm.sh, confirm_supplementary.sh, confirm_evidence.sh.
E68	Greedy outputs of every arm that changes the target’s arithmetic, and of stock MTP and DFlash, against matched stock references without the prefix cache: first-divergence classes, rates and ratios to the batch-shape floor, each server’s pools, and the first classification against the prefix-cache-on plain run (Appendix E.8) evidence/bench/equality/classes.json, report.json, divergences.csv, table.csv, pools.csv, evidence/bench/README.md; produced by bench/campaigns/equality_tuned.sh, bench/divergence.py, experiments/state_safety/run_matrix.py, compare.py.
E69	GSM8K test accuracy of six runs (two of plain decoding, buffered MTP, MTP with snapshot verification, DFlash at block 8, replayed plain decoding) with paired exact McNemar tests evidence/bench/quality/comparisons.json, evidence/bench/quality/; produced by bench/campaigns/quality.sh, quality_compare.py.

Entry What it supports; files and the program that produced them

Literature

E70 Literature review and citation audit behind Section 6
sources/literature_review.md, citation_audit.md, source_manifest.json.

A file name listed without a directory is in the directory of the entry’s first file. Raw captures, traces and per-row arrays are not in the repository; each evidence README gives the command that regenerates them on the same hardware.

G.2 Status of the questions

Table 26 lists each question of the programme (H1–H7) and each later proposal with a pre-registered kill test that has a result or a decision (P1–P13; P7a and P7b are the two parts of P7), with the test that decides or could damage it and its status. A failed hypothesis stays in the table with its mechanism.

Table 26: Questions and proposals, the test that decides or could damage each, and the status. Pending entries are listed in Appendix G.3.

Question or claim	Deciding or damaging test	Status
H1: the head is a large share of decode and MTP-speculation time	Nsight attribution at several batch sizes (Appendix E.2)	Measured: the head chain is 10.3–6.1% of a plain step ($B = 1$ –128), below its 15.1% share of the step’s weight bytes, and 17.2–12.4% of an MTP cycle ($B = 1$ –32), against about 38% of the cycle’s bytes at $B = 1$ [E31, E33]
H2: transport gives useful certificates on real hidden states, tighter than self-evidence and the static screen	Drift ratio against δ^* ; certification on aligned pairs (Section 5.3)	Refuted on DFlash-4B and MTP-4B for every bound family and tiling tested [E11, E12, E14, E13]
H3: a low-precision head with a rigorous envelope certifies the stock decision while reading fewer weight bytes	Candidate sets on replayed rows, then a GPU kernel against the stock head chain (Section 4)	Envelope and contract proved; candidate sets and fallback shares measured on replayed rows [E2, E6], including MTP-4B verify and draft rows [E8]; runtime [pending D7]
Certified tokens equal stock tokens	Same-shape token equality; measured cuBLAS error against the modelled G_i	No disagreement on 6,005 replayed rows [E6]; kernel error [pending D2]
Fewer bytes read means less time	Head-path time against the stock chain at each served shape	[pending D7]
Per-row sparsity reduces weight traffic	Batch unions of candidates; measured traffic	Candidate unions on capture batches measured [E2]; traffic [pending D6]
Certificates save target work	Count rows, re-scoring and completion, not only decision cost	A cheap decision is not a cheap verification on constructed heads [E20]
H4: the certificate improves the served frontier without changing outputs	Before-and-after sweeps with output-equality checks (Appendix E.8)	[pending D34]
H5: SGLang’s hybrid state stays correct under speculative verification	Targeted tests at every rejection position, aborts and prefix reuse (Appendix D.5)	Truncation and stop tokens at every index of an MTP verify cycle leave the output token-identical to the untruncated prefix (397 and 400 cases; log-probabilities and state not compared), and for the tree and plain decoding identical in tokens and log-probabilities (198 and 200 cases) [E29]; with identical pools, MTP’s divergences from plain decoding at concurrency 1 per fragile position by the previous cycle’s commit length are compatible with one common rate (a failure to reject, positions treated as independent) and none falls at a non-fragile position [E28]; the declared first-cycle test [pending D13]; the other tests [pending D12]

The Work a Verifier Needs

Question or claim	Deciding or damaging test	Status
H6: other layers of the stack leave measurable headroom	Attribution and controlled ablations (Appendices E.2 and E.6)	Measured shares: FP32 state 41.6% of a plain step at $B = 128$; host gaps about 21% of an unprofiled MTP cycle at $B = 1$ [E30, E34]; removing blocking planning reads shortens the MTP cycle by 2.7–4.8% in held batches with token-identical outputs [E44]; faster backbone GEMM kernels would save at most 5.7% of a plain step at $M = 1$ (derived from isolated microbenchmarks [E43]; served [pending D22]); removals [pending D14]
H7: reformulations and declared approximations multiply gains well beyond tuning	Bytes per step, quality–speed curves, end-to-end sweeps (Appendix E)	Ceilings derived [E36]; first single runs [E37]; the rest [pending D21]
Adaptive sampled depth stays exact	Tiny laws with candidate-dependent stopping	Settled by prior art and Theorem 4.5 (Appendix B.7)
P1: a certificate on the decision can skip the final layer’s MLP and the head	Oracle-radius replay at the cut (Appendix A.9)	Rejected by its kill test [E9]
P2: long-window exact repair from a DFlash window beats optimized DFlash substantially	Perfect-continuation oracle, then real windows (Appendix F.6)	With SGLang’s FlashInfer GDN verify kernel, perfect blocks meet the 5× target only at $B = 256$ with a free drafter (5.01×; 4.68× with DFlash’s drafting cost); with its Triton kernel (exactness pending), from $B = 64$ (5.21×) [E56, E57]; with the Triton kernel a block drafter would need to commit 61.1 of 64 tokens per cycle at $B = 64$, a per-position acceptance of at least 0.9985 if it were the same at every position, against 0.80–0.91 measured at block 16 (derived) [E58]; one-step recycling does not help, and exact sweeps gain about one token each [E59, E60]; sweeps in the engine [pending D29]
P3: target-anchored residual repair is much cheaper than recomputation	Oracle and real-correction kill tests (Appendix F.6)	Refuted: rank-128 bases leave a median relative error of 0.64–0.81 on held-out changes, repaired decisions agree at 37% of changed positions, and four sweeps add at most 0.11 accepted drafts [E60]; the two-pass design also misses the target with the FlashInfer verify kernel (ceiling 4.78× at $B = 256$), though it would pass with the Triton kernel (exactness pending) [E56, E57]
P4: writing the recurrent state less often, bit-exactly, gives at least 1.10× decode throughput at batch 128	Bit-exactness check, then paired timing (Appendix E.5)	Bit-exact at kernel level on synthetic activations; kernel 1.22× at batch 128, derived about 1.08× per decode step, below the target [E40]; served test [pending D17]
The recurrent state compresses exactly	Exact witnesses; rank audit of captured keys	Witnesses committed [E22]; audit [pending D16]
P5: exact computation can be shared across unrelated requests	Analysis, exact witnesses, measured share of the one structural candidate	Rejected in analysis and by its pre-registered criterion (at most 0.60% of a step) [E35]
P6: a draft selector trained for rate beats the same selector with the strongest matched objective by 1.25×	Candidate-support screen, then a matched-objective comparison (Appendix F.5)	Survives the screen (oracle bound 10.14 against 6.20 tokens per cycle) [E54]; comparison [pending D27]
P7a: every exactness claim names its reference and one of five contracts	Witnesses that separate the contracts (Appendix D.1)	Adopted throughout; witnesses committed [E21]
P7b: certified block-parallel GDN verification beats strict replay	Captured-trace arms: reference, strict replay, certified fast, unchecked fast (Appendix F.8)	SGLang’s chunked kernel is not a usable fast path (synthetic inputs, one request) [E45]; the certified path [pending D18]
P9: reusing a cached DFlash window once after a rejection beats fresh drafting	Support oracle on the post-rejection boundaries, then the cost-charged two-cycle test with a real program and a matched refiner (Appendix F.7)	For P9’s program, conditioned on the whole corrected prefix, over offline-recomputed candidate sets, with a padded block-16 verify and reuse at every possible boundary at $c = 1$, the oracle bound does not reject reuse with top-8 and top-16 sets ($\Delta = +0.22$ and $+0.99$ tokens per boundary; an upper bound, not a speed-up) and rejects it with top-1, top-2 and top-4 [E61]; the bounds with a free reused verify and with an oracle gate reject none of the tested sets; real program [pending D30]

The Work a Verifier Needs

Question or claim	Deciding or damaging test	Status
P11: restartable prefix-demand verification removes verify work	None run (Appendix F.8)	Dropped by its author in favour of P10: verify kernels keep the recurrent state in registers, so a shorter suffix does not remove per-token full-state reads (reasoning); P10’s first test [pending D19]
Results transfer	New prompts, a second model or a second GPU	[pending D36]

G.3 Pending results

Each entry is designed work without a committed result, with the place its evidence is to be committed. The text marks it [pending D*n*] and says beside the marker what the result will decide; no number stands in for it.

Entry	Pending result (where its evidence is to be committed)
D1	The certified-head implementation and its tests (src/certified_head/, tests/)
D2	The stock head GEMM’s kernel per batch size and its measured accumulation error against both error models (evidence/certified_head/stock_invariance.json)
D3	Row-wise and column-subset bitwise invariance of the stock head GEMM (evidence/certified_head/stock_invariance.json)
D4	Undecided rows and batches by error model and fallback mode on replayed decode steps (evidence/certified_head/replay_decisions.json)
D5	Cost of the invariance-checked row-wise fallback at $M = 32$ to 256 (evidence/certified_head/micro_head.json)
D6	Times of the int8, integer and BF16 passes per M , stage times, effective bandwidth, primitive costs and batch-union traffic (evidence/certified_head/micro_head.json, head_primitives.json)
D7	Head-path time against the stock chain for $M = 1$ to 256 , per error model (evidence/certified_head/head_path_time.csv)
D8	The seeded race’s slack derivation; agreement with the stock seeded sampler on replayed steps; acceptance against the stock speculative sampler (evidence/certified_head/)
D9	Work counts per mechanism converted to predicted time with the kernel’s primitive costs (evidence/head_geometry/work_model.json)
D10	Race candidates on speculative states at $T = 1$ and 0.7 ; mass enclosure and $P_?$ of the standard rule on real drafts; decisions resolved before re-scoring; fixed-noise against standard acceptance (evidence/head_geometry/, evidence/certified_head/)
D11	Caches entering the first differing module for the concurrency pair, for the 127 plain-versus-MTP prompts not rerun with the cache-hashing tap and for the four tap-changed prompts the history test does not reproduce; the pools’ part in plain-versus-MTP divergences after the first; the prefix-cache-off history control for MTP; the remaining pinned pairs (Appendix D.4) (evidence/state_safety/)
D12	Checkpoint reuse, aborts with slot reuse, chunked prefill and run-to-run repeats (H5) (evidence/state_safety/targeted.json)
D13	The declared first-cycle test on 960 fresh prompts: supported, inconclusive or void (evidence/state_safety/first_cycle_fresh.json)
D14	Repeated engine-only and served lever measurements up to $B = 1,024$, the paired FP16-state rerun, and the mechanisms of levers that fail (evidence/moonshot/)
D15	Replayed verification at concurrency 128 to 1,024; KL, divergence rate and GSM8K by state precision (evidence/moonshot/lever_sweeps_hc.csv, lever_quality.csv)
D16	Rank and observability audit of captured GDN traces (evidence/moonshot/)
D17	P4’s server output probe and served paired timing at batch 128 (evidence/moonshot/)
D18	P7b: certified block-parallel GDN verification against strict replay on a captured trace (evidence/moonshot/)
D19	P10’s first test: the exact GDN fold’s share of the speculative cycle at concurrency 8 and 16 (evidence/drafter/)
D20	Measured effect of the reduced draft head and the relaxed greedy rule, and the rule’s quality curve (evidence/moonshot/)
D21	The interaction matrix and the exact and lossy frontiers, with the lossy stack’s quality (evidence/moonshot/interactions.csv, frontier_exact.csv, frontier_lossy.csv)
D22	The exactness class of each backbone switch against stock plain decoding at concurrency 1, and paired serving runs against the tuned plain and MTP arms at concurrency 1 to 128 (evidence/backbone/)
D23	Interleaved serving sweeps of the host-gap patches against stock for MTP and DFlash at concurrency 1 to 32; validation of the two later patches; stock host traces retaken with the same window driver (evidence/hostgap/)
D24	The DFlash cycle split at batch 1, 4, 16 and 64, from an untraced run (evidence/drafter/cycle_split.csv)
D25	Two-sided margins at the DFlash and MTP divergences and a trace of the DFlash divergences to their first differing kernel (evidence/drafter/panel_v2/)
D26	Which limitation of the drafter the measurements support, and the decision on training (evidence/drafter/)
D27	P6’s rate-trained selector against the strongest matched objective at fixed width (evidence/drafter/)
D28	Verify-pass kernel trace at $B = 64$ and 256 , a same-session control, the Triton verify kernel’s exactness class and the reference continuation’s acceptance per width (evidence/repair/)
D29	Accepted drafts against full sweeps in the engine, recycle and keep, per width; P2’s verdict (evidence/repair/jacobi_progress.csv)

The Work a Verifier Needs

Entry	Pending result (where its evidence is to be committed)
D30	P9’s cost-charged two-cycle test with a real program and a matched refiner; verify time at widths 2 to 16; draft shares at concurrency 8 and 16 (evidence/repair/)
D31	Capacity limits per arm at default settings (evidence/bench/)
D32	Two more confirmation sessions of replayed plain decoding, which it needs to rank on the frontier (evidence/bench/confirm/)
D33	The natural-length sensitivity workload at concurrency 32 and 128 (evidence/bench/sensitivity/)
D34	Before-and-after frontiers for the certified head, with output equality at the same batch shape (H4) (evidence/bench/)
D35	Frontend behaviour above concurrency 256 on a quiet host, and repeats of the diagnosis (evidence/bench/frontend/)
D36	Transfer to new prompts, a second model or a second GPU; not scheduled

G.4 Reproduction

The CPU evidence (the exact references, the contract and state-structure witnesses, the synthetic experiment and the Lean check) is reproduced from the repository root with Python 3.13, NumPy and, for the formal files, Lean 4.19.0:

```
python -m venv .venv && . .venv/bin/activate
python -m pip install -r requirements-cpu.txt -r requirements-dev.txt
python scripts/verify_artifact.py          # rational references, synthetic drift, Lean check
python tests/test_precision.py             # exact floating-point reference
python tests/test_contracts.py             # contract witnesses
python tests/test_state_structure.py        # recurrent-state witnesses
python -m pytest tests/                   # every CPU test; GPU tests skip without CUDA
```

The GPU experiments need the hardware and engine of Appendix G.5: one GH200 and the engine at the pin with the patches under `engine/sglang/patches/`; `engine/sglang/README.md` gives the commands that apply each series. Every evidence directory’s `README.md` lists the exact command behind each file, and `RUNBOOK.md` collects the procedures. Timed runs were taken with exclusive use of the GPU and with the CPU load of other processes recorded for every point. The paper builds with `latexmk -pdf paper.tex` in `paper/` (pdfLaTeX and BibTeX, no shell escape); the build fetches the MLSys style file from the official kit and checks its SHA-256 digest (`paper/template/README.md`), and a figure whose data file is absent builds as a marked placeholder.

G.5 Hardware, software and model pins

Every measurement in the paper was taken with the pins below; the precise command, commits and resolved server settings of each run are recorded next to its evidence.

Item	Pin
GPU	One NVIDIA GH200 480GB: 96 GB HBM3, compute capability 9.0 (sm_90), 132 SMs, 60 MB L2; clocks 1,980 MHz (SM) and 2,619 MHz (memory) during the profiles
Host	64-core Grace (aarch64); driver 570.195.03 with CUDA 13.0 forward-compatibility libraries
Engine	SGLang <code>bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824</code> (0.5.21.dev861), PyTorch 2.13.0+cu130, FlashInfer 0.6.18, Triton 3.7.1; FlashAttention 3 and 4 are unavailable on this aarch64 host
Target model	Qwen/Qwen3.5-4B at revision <code>851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a</code> , BF16; its MTP layer loads from the same checkpoint
Block drafter	<code>z-lab/Qwen3.5-4B-DFlash</code> at revision <code>9a1996ccf887b79ab3af4fcbf8c1d1f4b5658bcf</code> (weights SHA-256 <code>1eb221d3...; byte-identical to modal-labs/Qwen3.5-4B-DFlash</code>)
Engine patches	Head-input capture (<code>engine/sglang/patches/geometry/</code>), divergence tap and verify KV split (<code>state/</code>), lever arms (<code>moonshot/</code>), acceptance trace hook (<code>drafter/</code>), repair probes and a timing-only verify switch (<code>repair/</code>), backbone GEMM routing and folded norms (<code>backbone/</code>), sync-free attention planning (<code>hostgap/</code>); each is off unless its flag or environment variable is set
Serving client	AIPerf 0.13.0, closed-loop concurrency, streaming chat completions
Profilers	Nsight Systems 2025.3.2, Nsight Compute 2025.3.1
CPU references	Python 3.13 (3.13.5 for the rational reference, 3.13.15 for the floating-point reference and the witnesses; each evidence file records its version), NumPy 2.3.5; Lean 4.19.0
Prompts	320 capture prompts (<code>evidence/head_geometry/prompt_manifest.csv</code>), 320 divergence prompts (<code>evidence/state_safety/prompt_manifest.json</code>), the serving workload [E62], each with its sources and SHA-256 digests

G.6 Attribution and disclosure

Samuel Mausberg is the author of this paper and is responsible for its claims. The earlier draft of the manuscript, its derivations and its reference code were prepared with GPT-6 Astra Pro (OpenAI). This version, including the literature review, theory, measurements, implementation and text, was produced with Claude Code using Claude Opus 5.5 agents, with an independent agent reviewing the changes, and the author directed the work and reviewed it as it progressed, including in discussions of specific points with Claude and with OpenAI models. No AI system is an author. The confidential assignment that motivated the work is not reproduced or quoted, and no model weights are redistributed.

H GLOSSARY

Table 1 in Section 2 defines the terms the main text relies on. The table below defines, in alphabetical order, those that appear only in the appendices, with the place where each is introduced.

Term	Definition
Accepted length	Committed tokens per verify cycle, the bonus token included: τ pooled over cycles, $\bar{\tau}_{\text{req}}$ averaged over requests as the DFlash model card does (Appendix F.3).
Batch size B , concurrency c	B is the number of requests in one decode step; c is the number of requests the client keeps in flight, which the server may cap below c .
Batch union	The set of vocabulary tiles or rows that at least one row of a batch still needs; a kernel that shares weight reads across the batch must read the whole union (Appendix C.8).
Bound family, tiling	The geometry of a transport bound (scalar, coordinate, grouped, low-rank radii, or their minimum) and the partition of the vocabulary into tiles (contiguous, random or k -means; 64 to 1,024 rows) (Appendix C.1).
Capture and replay	Head inputs recorded bit for bit from the running engine and re-evaluated offline (Section 3.1).
Certified row	A head input whose R-stock token the enclosures fix for every accumulator value the error model admits; every other row runs the fallback.
Commit rule, sound set	Commit a decision when every logit vector in a set known to contain the reference logits gives the same decision (Definition D.1).
Common mass α^*	The largest probability with which an exact sampler can emit a token before more is known, $\sum_i \inf_p p_i$ over the compatible laws (Proposition B.5).
Dead, deferred, soundly enclosed	The three classes of skipped work: no remaining consumer; a later consumer with a reconstruction path; or bounded well enough for every consumer that remains (Appendix A.9).
Demand frontier	Suppression of head work after the first certified rejection in a chain (Proposition B.8).
Divergence, exposure, noise floor	The first position at which two runs choose different tokens; the positions compared before it; divergences per 1,000 exposed tokens between stock configurations that ought to agree (Appendix D.4).
Drift scaling s	A model extrapolation that replaces the draft input by $h^t - s\Delta$, scaling every drift term by s ; not a measured drafter (Appendix C.9).
Emitted and materialized prefix	The tokens a cycle has returned and the tokens already consumed into every cache; the bonus token is emitted before it is materialized (Appendix D.5).
Enclosure $[lo_i, hi_i]$	An interval that contains the real logit z_i , computed with outward rounding (Lemma A.3).
Exact and lossy lever, exact and lossy stack	A lever is exact when every emitted token remains the target’s decision under the stock-kernel contract, lossy otherwise; the lossy stack admits lossy levers under a quality budget declared before measuring (Appendices E.6 and E.7).
First differing module output	In a bit-exact tap of two runs, the first position and, in execution order, the first module whose output bits differ; cache state is checked separately (Appendix D.4).
Fixed-noise verification	Sampled verification that accepts a drafted token exactly when it wins the target’s race under the keyed noise field (Theorem 4.5).
Frontier coordinates x, y	Mean per-request output rate including the time to first token, and output tokens per second on the GPU (Equation (13)).
Guards, P_γ	Interval tests that decide the standard acceptance rule from enclosures of the numerator and the partition function; P_γ is the probability that neither fires (Theorem B.2).
H1–H7, P1–P13	The programme’s questions and the later proposals with pre-registered kill tests (Table 26); P7a and P7b are the two parts of P7, and a proposal is listed once it has a result or a decision.
Head chain	The stock head GEMM, the FP32 copy of its logits and the argmax or top-1 that follows it (Appendix E.2).
Integer pass	A certified pass that quantizes the head input as well and accumulates exactly in int32 (Theorem A.1).
Keyed noise field	Gumbel noise $G_{t,i}$ that is a pure function of (seed, absolute position t , token i), as in the engine’s seeded sampler.

The Work a Verifier Needs

Term	Definition
Layer-serial and overlapped ceilings	Throughput bounds under two execution models: per-request kernels after the weight GEMMs, as the engine runs, or overlapped with them (Equations (11) and (12)).
Oracle radii	The realized per-tile deviations used as transport radii; a ceiling that no certified geometry reaches (Appendix C.7).
Partition interval $\log(Z^+/Z^-)$	The width, in nats, of an enclosure of the partition function; it decides how often the acceptance guards fire (Appendix C.8).
Point-mass proposal	A deterministic draft token, SGLang’s default for sampled verification; its residual is a race over the untried tokens (Appendix B.4).
Replayed and snapshot verification	Two ways SGLang verifies a draft chain on the GDN layers: snapshot verification writes one FP32 state per draft token and copies the accepted one back at commit; replayed verification (<code>--enable-linear-replayssm-spec</code>) keeps each draft token’s inputs and folds the accepted prefix into the state at commit (Appendix E.4).
Residual race	An exact sample from the residual law $(p - q)_+$ by a certified race under a fixed priority field (Theorem B.3).
Rounding flip, order flip, ambiguous	How a divergence reaches the token: BF16 rounding merges an unchanged accumulator order into a tie; the accumulator order reverses; or the order cannot be read from the data under the named error model (Appendix D.4).
Rounding-preserving replay	Decoding that keeps a dense state anchor and the recent FP32 operands and replays them in the stock kernel’s order, writing the state less often (P4, Appendix E.5).
Static screen	Tile bounds $\langle \mu_c, h \rangle + r_c \ h\ $ evaluated at the target state without any draft (Proposition C.5).
Survival $S(k)$, conditional acceptance α_k	The share of verify cycles that accept at least k drafted tokens, and $S(k)/S(k - 1)$ (Appendix F.3).
Tuning and confirmation splits	Disjoint prompt sets of the serving workload: configurations are chosen on the first and reported on the second (Appendix E.8).
Verify pass, draft step, cycle	The target’s forward pass over a drafted block; one drafter forward; a draft, its verification and the state commit (Appendices E.2 and F).